

Transducers

Mikołaj Bojańczyk

May 25, 2026

Contents

Introduction	4
Part A: Mealy machines	8
A.1 Mealy machines	9
A.1.1 Equivalence, compositions and continuity	10
A.2 The Krohn-Rhodes Decomposition Theorem	14
A.2.1 Map lifting	15
A.2.2 State transformations	17
A.2.3 Aperiodic Mealy machines	21
Part B: Rational functions	25
B.1 Rational relations	26
B.1.1 Composition and continuity	28
B.1.2 Undecidable equivalence	29
B.2 Rational functions	32
B.2.1 Bimachines	32
B.2.2 Decomposition into primes	37
B.2.3 Mealy machines as a subset of the rational functions	37
B.3 Rational relations and weighted automata	40
B.3.1 Weighted automata	40
B.3.2 Decidable equivalence	42
B.3.3 Decidability of equivalence for weighted automata over a field	45
B.4 Machine independent characterisations	47
B.4.1 Mealy machines	47
B.4.2 Sequential functions	50
B.4.3 Subsequential functions	51
B.4.4 Rational functions	56
Part C: Regular functions	59
C.1 The prime regular functions	60
C.2 Two-way transducers	64
C.2.1 Continuity	65
C.2.2 Closure under composition	68
C.2.3 Decidability of equivalence	71
C.2.4 Decomposition into prime functions	71
C.3 Streaming string transducers	78
C.3.1 Equivalence with regular functions	80
C.4 Logic	85
C.4.1 Monadic second-order logic	85
C.4.2 Rational functions in terms of logic	88

C.4.3	Regular functions in terms of logic	90
C.4.4	The first-order fragment	97

Part D: Polyregular functions **103**

D.1	For-transducers	106
D.1.1	Equivalence with polyregular functions	108
D.2	Pebble transducers	113
D.2.1	Continuity	114
D.2.2	Equivalence with for-transducers	117

Introduction

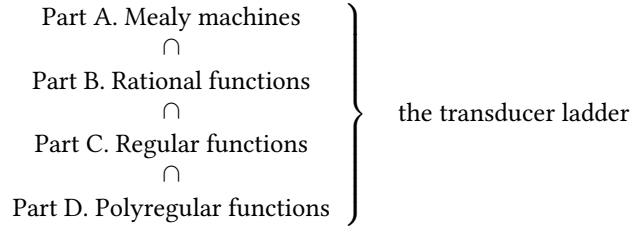
This book is a collection of lecture notes for a course on transducers, taught at the University of Warsaw in 2024 and 2026. Unlike the study of regular languages, which can be viewed as functions with Boolean outputs

$$L : A^* \rightarrow \text{Bool} = \{\text{yes}, \text{no}\},$$

this course is about string-to-string functions

$$f : A^* \rightarrow B^*,$$

where A and B are some finite alphabets. In this course, the functions should be computed by models that are finite-state in some way; such models will be called transducers. There will be four main transducer models treated in this book, which are ordered in a “transducer ladder” in increasing order of expressive power as follows:



Before delving into the details of these models, we begin by explaining some general properties that we would like our transducer models to have.

Continuity. The transducer ladder does not include Turing machines, or pushdown automata, suitably generalised to compute string-to-string functions. Why?

The reason is that, in our understanding of “finite-state”, the regular languages are finite-state, but pushdown automata and Turing machines are not. This is because we want finite-state models to correspond to regular languages. While the distinction between regular languages and non-regular ones is well understood, the same distinction for string-to-string functions is less clear. Although one of the transducer models in the transducer ladder is called the “regular” functions, this might well be only a historical artefact, and cogent arguments can be made to justify the name “regularity” for any of the other models in the transducer ladder, and also for other models that are not in the ladder. Summing up, we do not really know what is the right analogue for regular languages in the case of string-to-string functions.

Nevertheless, there are some models that we can reject out of hand, because they violate a criterion that we call *continuity*. This criterion requires a function to be compatible with regularity in the string-to-Boolean case, as formalised in the following definition.

Definition .0.1 (Continuous string-to-string functions). *A function*

$$f : A^* \rightarrow B^*$$

is called continuous if for every regular language $L \subseteq B^*$ over the output alphabet, the inverse image $f^{-1}(L)$ is also regular.

For example, if we take a string-to-string function that is computed by a Turing machine, or by a pushdown automaton with some kind of output mechanism, then the function will typically not be continuous. One of the salient properties of continuity is that for functions with two possible outputs “yes” and “no”, continuity is the same as regularity in the usual string-to-Boolean sense (this observation is valid when we see the outputs as one-letter words, and also as multi-letter words).

The name continuity is loosely inspired by the topological notion of continuity, in which the inverse image of an open set is open. This is not a perfect analogy, since open sets in topology should be closed under possibly infinite unions, while regular languages are only closed under finite unions. A more in-depth discussion of this analogy can be found in the exercises, where we show that Definition .0.1 is the same as uniform continuity with respect to a certain metric on strings.

As we will see, all functions in the transducer ladder are continuous. Unfortunately, continuity on its own is not enough to guarantee a reasonable transducer model. In fact, as the following example shows, there are uncountably many continuous functions.

Example 1. Consider an output alphabet B and any sequence of words

$$w_1, w_2, \dots \in B^*$$

which has the following *Cauchy property*: every regular language $L \subseteq B^*$ gives the same answer on all but finitely many words in the sequence. By a simple extraction process, one can show that every infinite sequence contains a subsequence with the Cauchy property. A more explicit example of a Cauchy sequence is the sequence of words obtained by repeating an input letter a factorial number of times

$$b^1, b^{2!}, b^{3!}, \dots \quad \text{where } b \in B.$$

Let us prove that this sequence has the Cauchy property. Consider some regular language, and a corresponding deterministic finite automaton (DFA) that recognises this language. (We assume that the reader is familiar with DFA.) When reading a string that uses only letter b , the run of the automaton looks like a lasso, with some prefix of length k followed by some loop of length ℓ . Therefore, if $n > k$, the acceptance of an input string b^n depends only on the remainder $n - k \bmod \ell$. If n is furthermore a sufficiently large factorial – and hence divisible by ℓ – then this remainder is always the same, namely $\ell - k \bmod \ell$. Hence, from some point on the automaton will give the same answer to all strings in the sequence, thus proving that the sequence is Cauchy.

Fix a sequence with the Cauchy property, and consider any function

$$f : A^* \rightarrow B^*$$

where all outputs are in the sequence, and furthermore each output is used for finitely many inputs. For every regular language $L \subseteq B^*$, the language will give the same

result on all but finitely many words in the Cauchy sequence, and therefore the inverse image $f^{-1}(L)$ will be either finite or cofinite. Since languages that are finite or cofinite are regular, it follows that the inverse image will necessarily be regular, thus proving that the function is continuous. Since a Cauchy sequence can be constructed in uncountably many ways, and the function f can also be constructed in uncountably many ways, there are uncountably many continuous functions of this form. $\square$

Despite the issues raised in the above example, continuity will be a good – if not perfect – criterion for checking if a transducer model is reasonable. There are, however, other criteria.

Composition. Another desirable property of transducer models is closure under composition, i.e. if we have two functions

$$A^* \xrightarrow{f} B^* \xrightarrow{g} C^*$$

that can be defined in some transducer model, then the same should be true for their composition. All transducer models in this book will have this property. Arguably, continuity itself can be seen as a special case of composition, since taking the inverse image is the same as composing with a string-to-Boolean function:

$$A^* \xrightarrow{f} B^* \xrightarrow{L} \text{Bool}.$$

One of the advantages of composition is that functions can also be decomposed, i.e. presented as compositions of simpler functions. This will be a fundamental theme in this book, starting with the Krohn-Rhodes decomposition of Mealy machines, and continuing with similar decomposition theorems for models that are higher up in the transducer ladder.

Decidable algorithmic problems. The above conditions are necessary, but not sufficient, for a good transducer model. For example, we could take the class of all continuous functions: this class is easily seen to be closed under composition, but it contains many pathological examples, as we have seen in Example 1. An important further condition is that the model should be thoroughly decidable, similarly to finite automata. There are many algorithmic problems to consider, including:

- **evaluation:** given a transducer and an input string, compute the output string;
- **computing inverse images:** given a transducer and a regular property of output strings, compute the inverse image;
- **composition:** given two transducers, compute a transducer for their composition;
- **equivalence:** given two transducers, decide if they compute the same function.

The second and third problems can be seen as effective versions of continuity and closure under composition. For all transducer models in this book, all the above problems will be decidable, with one exception: we do not know if equivalence is decidable for polyregular functions.

Is this all? The properties that we have discussed above are not exhaustive. For all we know, there could be infinitely many transducer models that satisfy all of these properties. In this book, we try to give complete characterisations of some of the models in the transducer ladder, in a way that is machine independent. However, as we will see, this gets harder and harder as the models become more expressive.

Exercises

Exercise .0.1. Show continuity for the reversal function $a_1 \cdots a_n \mapsto a_n \cdots a_1$.

Exercise .0.2. Show continuity for the duplication function $w \mapsto ww$.

Exercise .0.3. Show continuity for the squaring function $w \mapsto w^{|w|}$.

Exercise .0.4. Show continuity for the factorial function $w \mapsto w^{|w|!}$.

Exercise .0.5. Show that the class of continuous functions is closed under composition.

Exercise .0.6. For words over an alphabet A , define a metric so that the distance between two strings $w \neq w'$ is

$$\frac{1}{\text{minimal number of states in a DFA that accepts } w \text{ but not } w'}.$$

Show that this is a metric, and in fact it satisfies the stronger inequality

$$d(w_1, w_3) \leq \max(d(w_1, w_2), d(w_2, w_3)).$$

Exercise .0.7. Show that all string-to-string functions are continuous with respect to the above metric.

Exercise .0.8. Show that the continuous functions, in the sense of Definition .0.1, are exactly the uniformly continuous functions with respect to the above metric.

Part A: Mealy machines

In this part, we study the simplest transducer model, namely Mealy machines. Mealy machines are barely different from deterministic finite automata, and in fact the early literature on automata theory did not even distinguish between the two. From this perspective, Mealy machines are a good starting point for the study of transducers, since many of the results that we care about – such as continuity, composition and equivalence – are easy to prove, which we do in Section A.1. There is one exception though, which is the Krohn-Rhodes decomposition theorem from Section A.2: of the various decomposition theorems that we will see in this book, this is possibly the most difficult one.

A.1 Mealy machines

A **Mealy machine** is a deterministic finite automaton in which transitions are labelled by output letters. Also, there is no accepting set of states, since the purpose of the machine is to produce an output string instead of accepting or rejecting. Here is a formal definition.

Definition A.1.1 (Mealy machine). A Mealy machine is a tuple consisting of

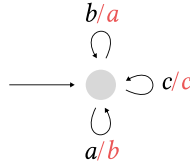
$$\underbrace{A}_{\substack{\text{input} \\ \text{alphabet}}} \quad \underbrace{B}_{\substack{\text{output} \\ \text{alphabet}}} \quad \underbrace{Q}_{\substack{\text{state} \\ \text{space}}} \quad \underbrace{q_0 \in Q}_{\substack{\text{initial} \\ \text{state}}} \quad \underbrace{\delta : Q \times A \rightarrow Q \times B}_{\substack{\text{transition} \\ \text{function}}}.$$

The semantics of the machine is a function

$$f : A^* \rightarrow B^*,$$

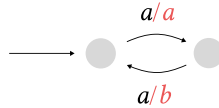
which is defined as follows. We consider the deterministic finite automaton which is obtained from the **Mealy machine** by ignoring the output letters. We run this automaton on the input string, and then we label each position by the output letter of the corresponding transition. This function is a *letter-to-letter* function, which means that the input and output have the same length.

Example 2. [Letter-to-letter homomorphism] We begin with the most trivial case of a **Mealy machine**, in which there is one state only. Here is a picture of such a machine, using the convention that transitions are labelled by pairs (input letter / output letter):



This particular machine swaps a with b , and leaves c unchanged. More generally, a one-state **Mealy machine** is the same as a *letter-to-letter homomorphism*, i.e. we apply some fixed letter-to-letter function independently to each input letter. $\square$

Example 3. [Alternating a and b] In this example, we have a two-state **Mealy machine** whose input alphabet is $\{a\}$ and whose output alphabet is $\{a, b\}$. Here is a picture of this machine:



This machine outputs letters a and b in alternation, as in the following picture:

☐

The output alphabet, call it $A + 1$, is the same as the input alphabet, except that it has an extra letter for the undefined output in the first position. The **Mealy machine** has states $A + 1$, i.e. the same as the input alphabet. The initial state is the extra letter, which stands for the undefined output in the first position. The transition function is

which means that the input letter is stored in the state, and the output is the previous state. Here is a picture of this machine, in the case when the input alphabet is $\{a, b\}$:



Theorem A.1.2. *The equivalence problem for Mealy machines is decidable.*

Proof. This is an easy reduction to the equivalence problem for regular languages, which is decidable. For a function

$$f : A^* \rightarrow B^*$$

computed by a **Mealy machine**, and an output letter $b \in B$, consider the language

$$\underbrace{\{ w \in A^* \mid \text{last output letter of } w \text{ is } b \}}_{\text{call this the "ends with } b \text{ language"}}$$

This is easily seen to be a regular language; the corresponding automaton is the same as the one in the **Mealy machine**, except that it remembers in its state whether the last output letter was b . (If we could use an extended variant of automata, in which the accepting set is a subset of transitions and not states, then we would not even need to modify the automaton in any way.) Since the function f is letter-to-letter, it is completely determined by the family of its “ends with b ” languages. Therefore, two **Mealy machines** f and g are equivalent if and only if the corresponding “ends with b ” languages are the same for every output letter b . $\square$

The above proof is trivial because there is a perfect alignment between input and output letters. For more advanced transducer models that will be studied later in the book, there will not be such an alignment, which will make the equivalence problem more difficult. This issue would arise already in the mild extension of **Mealy machines**, in which transitions produce not letters, but output strings of arbitrary (and possibly empty) length. As we will see in the next chapter, the equivalence problem will remain decidable for this extension, but the proof will be more complicated.

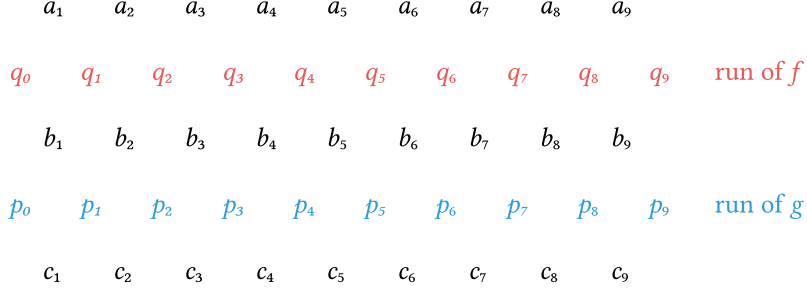
Compositions. We now establish closure under composition for **Mealy machines**. In this book, we use a dot to represent left-to-right composition of functions, i.e. $f \cdot g$ is the function where we first apply f , and then we apply g to the output of f . To denote the opposite order of composition, we will write $f \circ g$, which is the same as $g \cdot f$. In order to avoid confusion, we try to avoid the $\circ$ notation altogether.

Theorem A.1.3. ***Mealy machines** are closed under composition, i.e. if*

$$A^* \xrightarrow{f} B^* \xrightarrow{g} C^*$$

*are **Mealy machines**, then so is their composition $f \cdot g$.*

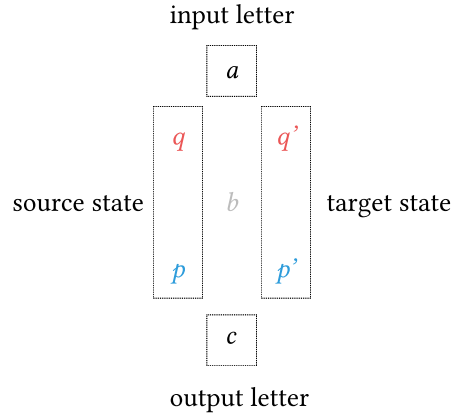
Proof. The proof is a straightforward product construction. Consider the composition $f \cdot g$ applied to some input string, with the runs of the two transducers shown in the following picture:



As the state space of the composition $f \cdot g$ we use the product

$$(\text{state space of } f) \times (\text{state space of } g).$$

The initial state is the pair of initial states. The transition function is easily understood by looking at the following diagram:



If we know the pair of source states (q, p) and the input letter a , then we can compute the intermediate letter b , and then use it to compute the pair of target states (q', p') and the output letter c . Hence, one can define a transition function in the product automaton which gives the desired composition. $\square$

Continuity. We finish this section by establishing continuity for [Mealy machines](#).

Theorem A.1.4. *Mealy machines are continuous.*

Proof. This can easily be proved using a direct product construction similar to Theorem A.1.3, but we prefer an indirect proof, which leverages the similarities of composition and continuity that were explained in the introduction. For a language $L \subseteq B^*$, define its *letter-to-letter lifting* to be the function

$$\text{lift}(L) : B^* \rightarrow \{0, 1\}^*,$$

where each position is labelled by 0 or 1, depending on whether the corresponding prefix of the input string belongs to the language. It is easy to see that the letter-to-letter lifting gives a bijective correspondence between regular languages and **Mealy machines** with output alphabet $\{0, 1\}$. In terms of this correspondence, inverse images become compositions, i.e.

$$\text{lift}(f^{-1}(L)) = f \cdot \text{lift}(L).$$

Therefore, continuity for **Mealy machines** becomes a corollary of closure under composition from Theorem A.1.3. $\square$

Exercises

Exercise A.1.1. Give an example of a function that is continuous, letter-to-letter (i.e. input length is the same as output length), and is not computed by a **Mealy machine**.

Exercise A.1.2. Show that a composition of n **Mealy machines** might require a number of states that is exponential in n .

Exercise A.1.3. We say that a **Mealy machine**

$$f : A^* \rightarrow A^*$$

is *invertible* if there is some **Mealy machine**

$$f^{-1} : A^* \rightarrow A^*$$

such that the composition $f \cdot f^{-1}$ is the identity function. Show that one can decide if a given **Mealy machine** is invertible.

Exercise A.1.4. Consider the group of invertible **Mealy machines** of type $A^* \rightarrow A^*$, with the group operation being composition. If we take a finite set of generators, is the generated subgroup necessarily finite?

Exercise A.1.5. Give an algorithm which inputs a **Mealy machine** $f : A^* \rightarrow B^*$ and decides if the following is bounded by a polynomial in n :

$$n \in \mathbb{N} \mapsto |\{ f(w) \mid w \in A^* \text{ has length at most } n \}|.$$

Exercise A.1.6. We say that a regular language $L \subseteq A^*$ is *regular-complete under Mealy reductions* if for every regular language $K \subseteq B^*$, there exists a **Mealy machine** $f : B^* \rightarrow A^*$ such that

$$w \in K \text{ iff } f(w) \in L \quad \text{for every nonempty } w \in B^*.$$

(We use nonempty strings because a **Mealy machine** cannot change the empty string.) Show that there is a language L with this property.

Exercise A.1.7. Give an algorithm, which checks if a given regular language is regular-complete under Mealy reductions. The algorithm can even be made polynomial, assuming that the input regular language is given by a deterministic automaton.

A.2 The Krohn-Rhodes Decomposition Theorem

As we have shown in Theorem A.1.3, Mealy machines can be composed. This motivates the idea of *decomposing* Mealy machines into simpler ones. For example, if we compose the delay function from Example 4 with itself, then we get the *double delay* function, which shifts the input letters by two steps to the right, as in the following picture:

b	c	a	b	c	b	a	a	c	b	input
$\perp$	$\perp$	b	c	a	b	c	b	a	a	output

Therefore, the double delay function can be decomposed into two copies of the delay function. The main result of this section is the Krohn-Rhodes Decomposition Theorem, which shows that every Mealy machine can be decomposed into a composition of certain special Mealy machines, which we call *prime Mealy machines*. Later on in this book, we will prove similar decomposition theorems for more advanced transducer models. Interestingly enough, the decomposition theorem for Mealy machines is arguably the most difficult one to prove.

Primes. We begin by defining the prime Mealy machines. These prime machines are defined in terms of the state transformations that they can perform. Here, the *state transformation* of an input letter is the function $Q \rightarrow Q$ which shows how this letter updates the state (the output of the machine is not relevant for this definition).

Definition A.2.1. *There are two kinds of prime Mealy machines:*

- *Reversible: all state transformations of all letters are permutations.*
- *Flip-flop: for each letter, its state transformation is either: (a) the identity transformation; or (b) a constant (i.e. all states are mapped to the same one).*

The definition of state transformations can be extended from individual letters to input strings, by composing the state transformations of the individual letters. We could modify the above definition to talk about state transformations of input strings, instead of input letters, and the result would be the same. This is because composing constants gives a constant, and composing permutations gives a permutation.

Example 5. If a Mealy machine has only one state, then it is both reversible and flip-flop, since all state transformations are the identity transformation. The machine with output $ababa \dots$ from Example 3 is a reversible machine, since the state transformation of each letter is a permutation. The delay machine from Example 4 is a flip-flop machine, since the state transformation of each letter is a constant. $\square$

Theorem A.2.2 (Krohn-Rhodes Theorem). *Every Mealy machine f admits a decomposition*

$$f = f_1 \cdot f_2 \cdots f_n$$

where each Mealy machine $f_1, \dots, f_n$ is either reversible or flip-flop.

In some decomposition results in mathematics, such as the decomposition of a number into its prime factors, the decomposition is unique. We make no such uniqueness claims in the above theorem. Since Mealy machines are closed under composition, it follows from the above theorem that the class of functions computed by Mealy machines is the same as the class of compositions of functions computed by prime Mealy machines. This can be written as:

$$\text{Mealy} = (\text{Reversible} \cup \text{Flip-flop})^*,$$

where the Kleene star on the right-hand side stands for closure under composition. The rest of Section A.2 is devoted to the proof of this theorem. There are two steps in the proof. First, in Section A.2.1, we show an auxiliary result about map lifting, which shows that a decomposition into primes can be applied in parallel to a list of input strings. Then, in Section A.2.2, we finish the proof.

A.2.1 Map lifting

In the first step of the proof, we introduce a construction on transducers that we call the map lifting, and which will be used many times in this lecture. The idea is to apply a transducer to a list of input strings, which are separated using some separator.

Definition A.2.3 (Map lifting). *Let*

$$f : A^* \rightarrow B^*$$

be a string-to-string function. The map lifting of f , denoted by

$$\underbrace{(A + 1)^*}_{\substack{\text{alphabet obtained from } A \\ \text{by adjoining a fresh letter } \#}} \xrightarrow{\text{map } f} (B + 1)^*,$$

is the function defined by

$$\underbrace{w_1 \# w_2 \# \dots \# w_n}_{\substack{\text{the strings } w_1, \dots, w_n \\ \text{do not use the separator } \#}} \mapsto f(w_1) \# f(w_2) \# \dots \# f(w_n).$$

The following lemma shows that map lifting is compatible with decompositions into primes.

Lemma A.2.4. *If a Mealy machine decomposes into prime Mealy machines, then the same is true for its map lifting.*

Proof. Map lifting commutes with composition of functions, i.e.

$$\text{map}(f \cdot g) = (\text{map } f) \cdot (\text{map } g).$$

Therefore, it is enough to show the lemma in the case when f is a prime Mealy machine. If f is a flip-flop, there is nothing to do: we reset its state to the initial state

whenever the separator symbol is read. The interesting case is when f is reversible. The difficulty is that resetting the state would break reversibility.

For an input string $w \in (A + 1)^*$ that may contain the separator symbol, we consider two state transformations:

δ_w this is the state transformation in the automaton which corresponds to the map lifting, i.e. the separator symbol resets the state to the initial one;

γ_w this is the state transformation in a variant of the above automaton, in which the separator symbol does nothing, i.e. it does not change the state.

The first state transformation is the one that we care about, but the underlying automaton is not reversible, since it involves resetting the state. The second state transformation seems less useful, but it arises from a reversible automaton. This usefulness of the second state transformation is demonstrated by the following cancellation law:

$$\delta_w = \underbrace{\left(\gamma_u \right)^{-1} \cdot \gamma_w}_{\substack{\text{a permutation, and} \\ \text{hence can be inverted}}} \quad \underbrace{\text{where } u \text{ is the prefix of } w \text{ that ends in the last } \#}_{\text{call this the } \# \text{-prefix of } w} \quad (1)$$

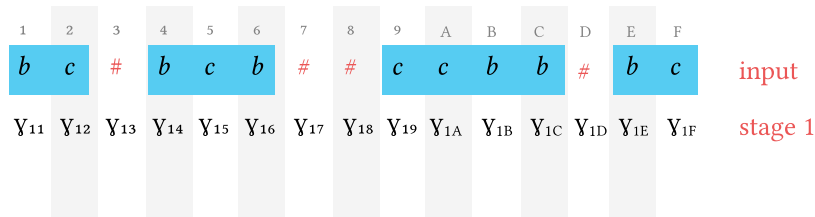
(In the edge case when w does not contain the separator symbol $\#$, the prefix u is empty, and its state transformation γ_u is the identity permutation.) Using the cancellation law, we will decompose the map lifting of f into prime Mealy machines. The idea is that for each prefix of the input string, we will compute the two state transformations that appear on the right-hand side of the cancellation law. This is done in two stages.

1. In the first stage, we label each prefix w of the input string with the state transformation γ_w , i.e. the one in which the separator symbol is ignored. This is done using a Mealy machine of type

$$(A + 1)^* \rightarrow (\text{permutations of } Q)^*,$$

where Q is the state space of the original Mealy machine. After reading an input word w , this machine stores γ_w in its state, and its last output letter is also γ_w . This machine is reversible, since the original one was reversible and the only new state transformation is the identity transformation.

Here is a picture of the first stage. In the picture, we use hexadecimal for indexing positions (so that we can have more single digit indexes) and we write δ_{ij} for the state transformation which corresponds to the infix of the input string that begins in the i -th position and ends in the j -th position, including both endpoints:



2. We now proceed to the second stage of the construction, in which we compute the state transformation for the $\#$ -prefixes of the input string, as in the following picture:

1	2	3	4	5	6	7	8	9	A	B	C	D	E	F	
b	c	#	b	c	b	#	#	c	c	b	b	#	b	c	input
Y_{11}	Y_{12}	Y_{13}	Y_{14}	Y_{15}	Y_{16}	Y_{17}	Y_{18}	Y_{19}	Y_{1A}	Y_{1B}	Y_{1C}	Y_{1D}	Y_{1E}	Y_{1F}	stage 1
			Y_{13}	Y_{13}	Y_{13}	Y_{13}	Y_{17}	Y_{18}	Y_{18}	Y_{18}	Y_{18}	Y_{18}	Y_{18}	Y_{1E}	stage 2

This is done using a flip-flop machine which is a variant of the delay machine from Example 4. The states of this machine are permutations of Q , plus an extra initial state for an undefined value. When we read a position with a separator symbol, we store the output of the first stage in the state. For other input positions, the state is unchanged. The output is always the incoming state.

3. By applying the cancellation law to the outputs of the first and second stages, we can get for each input position i the state transformation δ_{1i} . Using reversibility and the i -th input letter, we can also get the state transformation $\delta_{1(i-1)}$. From this state transformation, we can get the state just before reading the i -th position for the map lifting, and from this we can get the output of the map lifting. All of these operations use only the current position and its labels (in the input string and the first two stages), and hence they can be implemented using a one-state Mealy machine.

Since each of the three stages above is a prime Mealy machine, we get the conclusion of the lemma. Although this is not important for the statement of the lemma, we observe that the third stage can be merged with the second stage, since prime Mealy machines are closed under post-composition with letter-to-letter homomorphisms. Therefore, the map lifting of f is a composition of two prime Mealy machines, a reversible one followed by a flip-flop. $\square$

A.2.2 State transformations

In the second step of the proof, we will use the map lifting from the previous section to conclude the proof of the Krohn-Rhodes Theorem. As was the case for the map lifting, we will be mainly interested in the state transformations of the Mealy machine. The relevant information will thus be:

$$\underbrace{A}_{\text{input alphabet}} \quad \underbrace{Q}_{\text{state space}} \quad \underbrace{\{\delta_a : Q \rightarrow Q\}_{a \in A}}_{\text{state transformations}}. \quad (2)$$

We use the name *pre-automaton* for the above; it is the same as a DFA without designated initial and accepting states. For a pre-automaton, define its *state transformation*

transducer to be the Mealy machine of type

$$A^* \rightarrow (Q \rightarrow Q)^*,$$

in which the n -th letter of the output string is the state transformation for the first n input letters. The main lemma in the proof of the Krohn-Rhodes Theorem is the following.

Lemma A.2.5. *For every pre-automaton, its state transformation transducer is a composition of prime Mealy machines.*

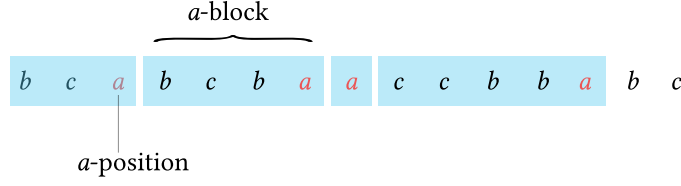
Before proving the lemma, we use it to conclude the proof of the Krohn-Rhodes Theorem. Take a Mealy machine, and consider its underlying pre-automaton. By applying the lemma, we get a composition of prime Mealy machines, which computes for each input position the state of the underlying pre-automaton after reading this position. To get the output of the Mealy machine, we can use a delay machine as in Example 4 to get the state from the previous position, and then we can get the output letter using a letter-to-letter homomorphism that is based on the transition function. It remains to prove the lemma.

Proof of Lemma A.2.5. We prove the lemma by induction on two parameters of the pre-automaton $\mathcal{A}$: (a) the size of the state space Q ; and (b) the size of the input alphabet A . These parameters are ordered lexicographically, so that we first compare the size of the state space, and if there is a tie, then we compare the size of the input alphabet. In particular, we can make progress in the induction by decreasing the number of states but increasing the size of the input alphabet.

Induction basis. The induction basis is when there is one state and one input letter. We can easily cover a generalisation of the induction basis, namely when the pre-automaton is reversible, i.e. all state transformations are permutations (which trivially holds when there is one state only). In this case, there is nothing to do, since the natural Mealy machine for state transformations is already reversible.

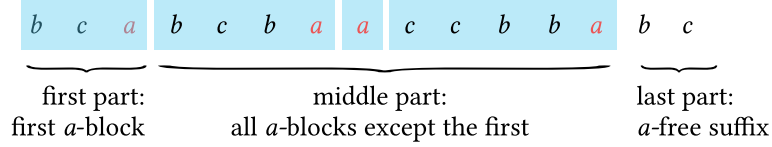
Induction step. We are left with the induction step, under the assumption that the pre-automaton is not reversible. This means that there is some input letter $a \in A$ with a state transformation that is not a permutation. This means that the image of this state transformation is some proper subset $P \subset Q$ of the state space. Fix this letter a for the rest of the proof.

In the proof, we will be interested in a -positions, i.e. positions in the input string that are labelled with the letter a , and a -blocks, i.e. pieces of the input string that go from one a -position to the next one, including the latter a -position but not the former one. Here is a picture:



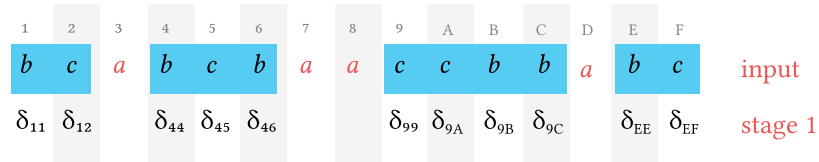
The a -blocks are disjoint and partition the input string, except for some suffix which does not use the letter a . The general idea in the induction step is that an a -block is subject to the assumption on a smaller input alphabet, since a appears only once, while the a -blocks can be seen as individual letters for a pre-automaton with a smaller set of states. This will be made precise below.

To compute the state transformation, we will be interested in a decomposition of the input word into three parts, as depicted in the following picture:



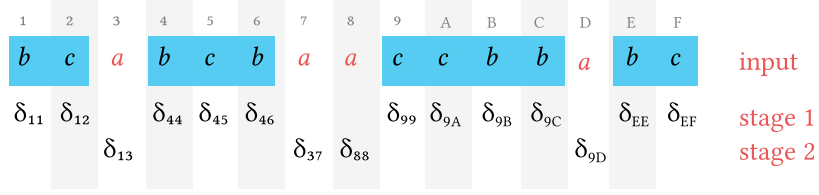
We refer to this decomposition as the *tripartite decomposition*. We now describe a sequential composition of prime Mealy machines, in which all state transformations in the tripartite decomposition are computed. This decomposition proceeds in five stages. We assume that in each stage, we have access to the outputs of the previous stages, since a prime Mealy machine can always be adapted so that it includes a copy of the input string in its output.

1. In the first stage, we compute the state transformation for the a -free suffixes, i.e. the last of the three parts in the tripartite decomposition. Here is a picture of this stage, using the same conventions as in the proof of Lemma A.2.4:



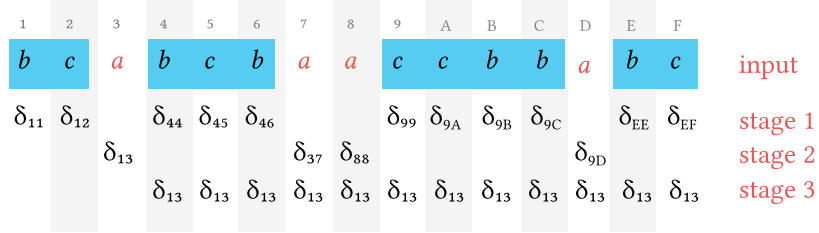
This stage is implemented by removing the letter a from the input alphabet in the pre-automaton, applying the induction assumption for a smaller input alphabet, and then applying map lifting to the result by Lemma A.2.4.

2. In the second stage, we compute the state transformations of individual a -free blocks, and we store them in the a -positions that end them, as in the following picture:



To implement this, we first use a delay machine as in Example 4, which is a flip-flop machine, to shift the outputs of the first stage by one letter to the right. Next, for positions that are not a -positions, we use a blank letter as the output, and for a -positions we take the value from the preceding position and post-compose it with the state transformation of a . This stage is a flip-flop machine followed by a letter-to-letter homomorphism, and hence it is a flip-flop machine.

3. In the third stage, we compute the first part of its tripartite decomposition, i.e. the first a -block. To do this, we use a flip-flop machine to copy the state transformation in the first a -block to the positions that are not in the first a -block, as in the following picture:



This way, each position outside the first a -block has access to the state transformation in the first a -block.

4. After the first three stages, we have access to the state transformations of the first and last parts of the tripartite decomposition. We are left with the middle part, which is the union of all a -blocks except for the first one. To compute this, we will use the induction assumption on the smaller set of states $P \subset Q$ which is the image of the state transformation of a . Consider a pre-automaton with states P as follows. The input alphabet is the alphabet used to store the original input word together with the results of the preceding stages, while the transition function is defined as follows:

- If the position is an a -position that is not the first one (which can be detected by looking at the input letter and the third stage), then apply the state transformation stored in the second stage.
- Otherwise, do not change the state.

We can apply the induction assumption to this pre-automaton, since it has a smaller set of states. The state transformation of this new automaton is the state transformation of the original automaton for the middle part of the tripartite decomposition.

5. In the stages so far, we have computed the state transformations for all three parts in the tripartite decomposition. These can be combined, using a letter-to-letter homomorphism, to get the state transformation for the entire prefix, as desired in the lemma.

□

A.2.3 Aperiodic Mealy machines

In the Krohn-Rhodes Theorem, we have two kinds of prime Mealy machines, reversible and flip-flop. What about functions that decompose into only one kind of primes? In other words, what are the classes

$$(\text{Reversible})^* \quad \text{and} \quad (\text{Flip-flop})^*?$$

For the first class, there is not much to say, since reversible Mealy machines are closed under composition, and hence the first class is just the class of reversible Mealy machines.

Lemma A.2.6. *Reversible Mealy machines are closed under composition.*

A much more interesting class is the second one, namely compositions of flip-flop machines. To capture this class, we will use the following notion of aperiodicity, which says that the machine cannot have periodic behaviour.

Definition A.2.7 (Aperiodic). *A string-to-string function is called aperiodic if for all input strings u, v, w , there is some output letter b such that*

$$b = \text{last letter of } f(uv^n w) \quad \text{for all sufficiently large } n.$$

In the definition above, we only talk about the last letter, but aperiodicity also implies that the last k letters of $f(uv^n w)$ are eventually fixed for every k , by the arbitrary choice of w .

Example 6. [Aperiodic and periodic functions] Consider the function

$$f : \{a\}^* \rightarrow \{0, 1\}$$

where the last letter says if the string is nonempty:

$$aaaaaaaa \mapsto 01111111.$$

This function is clearly aperiodic, since the last letter of any sufficiently long input string will always be 1. On the other hand, if we change the function so that it indicates the parity of the length, as in the following example:

$$aaaaaaaa \mapsto 010101010,$$

then the aperiodicity condition will be violated by repeating the letter a . In this example, we did not need the words u and w in $uv^n w$. To see that they are necessary, one can consider the function

$$f(w) = \begin{cases} \text{parity of } n & \text{if } w = \#a^n\# \text{ for some } n \\ \perp & \text{otherwise.} \end{cases}$$

□

Theorem A.2.8. *Consider a string-to-string function f that is computed by a Mealy machine. Then f is aperiodic if and only if it is computed by a composition of flip-flop Mealy machines. Moreover, this property can be decided, given a Mealy machine that computes f .*

Proof. We will begin with the implication

$$\text{composition of flip-flop} \implies \text{aperiodic}.$$

Then, before proving the converse implication, we will prove the decidability part of the theorem, as it will introduce notions that will be useful in the converse implication.

From composition of flip-flop to aperiodicity. To see that a composition of flip-flops is aperiodic, we will show that flip-flops are aperiodic, and that aperiodicity is preserved under composition. The first part is easy, since the last letter of $f(uv^n w)$ depends only on: (1) the last letter of w (or v if w is empty); and (2) the last letter in uvw that has a constant state transformation. Neither of these values depends on n . To see that aperiodicity is preserved under composition, it will be more convenient to use the following equivalent characterisation of aperiodicity.

Claim A.2.9. *A function f computed by a Mealy machine is aperiodic if and only if for all input strings u, v, w there are output strings x, y, z and a number k such that*

$$f(uv^{n+k}w) = xy^n z \quad \text{for all } n \geq 0.$$

Proof. The right-to-left direction is easy, since the last letter of $xy^n z$ does not depend on n . For the left-to-right direction, we use the remark after the definition of aperiodicity, which says that the last k letters of the output are eventually fixed. If we apply this to outputs of the form $f(uv^n)$ and the last $|v|$ letters, we see that from some point on, the output ends with a repetition of some fixed string y of length $|v|$. If we apply this to outputs of the form $f(uv^n w)$ and the last $|w|$ letters, we get the part z . □

The condition in the above claim is easily seen to be compatible with composition, since the number k for the composition $f \cdot g$ can be taken to be the sum of the corresponding numbers for f and g . This completes the proof of the implication from composition of flip-flop to aperiodicity.

Decidability. As announced at the beginning of the proof of the theorem, before proving the converse implication, we prove the decidability part. To check if function f is aperiodic, we will check if the corresponding Mealy machine satisfies a certain aperiodicity condition on the state transformations. However, in order for the latter check to be meaningful, we will need to use a machine that avoids unnecessary states. Since Mealy machines are very close to regular languages, we can use a minimisation procedure that is similar to the classical Myhill-Nerode minimisation theorem. Define a *derivative* of a string-to-string function f to be any function of the form

$$f(w_)\stackrel{\text{def}}{=} v \mapsto f(wv),$$

where w is some input string.

Lemma A.2.10 (Myhill-Nerode for Mealy machines). *A string-to-string function is computed by a Mealy machine if and only if: (1) it has finitely many derivatives; (2) it is letter-to-letter; (3) the n -th output letter depends only on the first n input letters.*

Proof. The left-to-right direction is easy: the derivative $f(w_)$ is uniquely determined by the state of the Mealy machine after reading w , and hence can be chosen in only finitely many ways. Let us now prove the right-to-left direction. The corresponding Mealy machine has states for the derivatives, with the initial state being the derivative corresponding to the empty string. The transition relation is defined by

$$f(w_)\xrightarrow{a/\text{last letter of } f(wa_)} f(wa_).$$

This transition function is well-defined, since the output letter and target states depend only on the source state $f(w_)$ and not the particular choice of w . Using a simple induction on the length of w , one can show that after reading an input string w , this machine is in state $f(w_)$ and the output that it has produced so far is $f(w)$. $\square$

The machine constructed in the above proof is called the *minimal machine* for the function f . The minimal machine can be easily constructed from any other Mealy machine, by computing which states define the same derivative, and merging them. To complete the proof of the theorem, it remains to show that aperiodicity can be decided based on the minimal machine.

Lemma A.2.11. *A function computed by a Mealy machine is aperiodic if and only if its minimal Mealy machine satisfies the following condition: (*) for every state transformation $\delta : Q \rightarrow Q$ that arises from some input string, the following sequence of state transformations eventually stabilises on a single state transformation:*

$$\delta^1, \delta^2, \dots$$

Proof. In the minimisation process, we merge states. Therefore, if two strings give the same state transformation in the original machine, then they will also give the same state transformation in the minimised machine. In particular, condition (*) is preserved under minimisation. Finally, if a Mealy machine satisfies (*), then the function it computes will be aperiodic in the sense of Definition A.2.7, since the state after reading wv^n will eventually stabilise. $\square$

The above lemma gives a decision procedure for checking if a function computed by a Mealy machine is aperiodic. We first compute all state transformations that arise from input strings, and for each one we check if its powers induce a non-trivial cycle. To compute the set of all state transformations, we start with the state transformations of individual letters, and we keep on adding state transformations by composing them, until we can add no new state transformation. This procedure terminates, since there are only finitely many possible state transformations. This completes the decidability part of the theorem.

From aperiodicity to composition of flip-flops. We are left with the implication

$$\text{aperiodic} \implies \text{composition of flip-flop.}$$

Consider a function f that is computed by a Mealy machine and is aperiodic. By Lemma A.2.11, we can assume that the Mealy machine satisfies condition (*) from the lemma. We now go through the proof of the Krohn-Rhodes Theorem for this particular machine, and we observe that in the induction process, all machines will continue to satisfy condition (*). This is because in the induction step, every new machine will only use state transformations that arise from the original machine, and these state transformations satisfy condition (*). In particular, a reversible Mealy machine with more than one state will never arise, since such a machine would violate condition (*). Therefore, all machines in the decomposition will be flip-flops, as desired. $\square$

Exercises

Exercise A.2.1. Show that every flip-flop machine can be obtained as a sequential composition of two-state flip-flop machines.

Exercise A.2.2. Show that the delay function from Example 4 is not a composition of reversible Mealy machines.

Exercise A.2.3. Show that the function from Example 3 is not a composition of flip-flop Mealy machines.

Exercise A.2.4. Is every invertible Mealy machine, as defined in Exercise A.1.3, necessarily reversible? The other way round?

Exercise A.2.5. Show that if $f : A^* \rightarrow B^*$ is continuous, then the same is true for its map lifting.

Part B: Rational functions

In this part, we move one step up the transducer ladder, to the rational functions. A Mealy machine produces its output deterministically in a left-to-right manner, and each input letter generates exactly one output letter. The rational functions lift both of these restrictions. There are intermediate models, which lift one restriction but not the other; however, we do not discuss them here in detail, in the interest of not proliferating transducer models.

Before defining the rational functions, in Section B.1 we take a detour, and define a more general class, which is called the rational relations. These are binary relations and not functions, because the underlying model is non-deterministic, and the same input string might generate multiple outputs, even infinitely many. Many results about rational functions are easily proved in the more general setup of relations, and so it is instructive to begin with the relational case. Nevertheless, there is a price to pay: the equivalence problem for rational relations is undecidable.

Next, in Section B.2, we define the rational functions as those rational relations which are semantically functional, i.e. each input produces exactly one output. Although the definition uses a nondeterministic model, we also identify an equivalent model that is deterministic, which is called bimachines, although the determinism is not of the left-to-right kind that we are used to from Mealy machines. Finally, we show how restricting to functions allows us to recover decidability of equivalence.

B.1 Rational relations

We begin with a nondeterministic version of the rational functions, in which the underlying model is an NFA equipped with an output mechanism. One of the advantages of this model is that it does not treat the input and output differently.

Definition B.1.1 (NFA with output). A nondeterministic automaton with output *consists of the following ingredients*:

- input and output alphabets A and B ;
- a finite set of states Q ;
- distinguished initial and final subsets $I, F \subseteq Q$;
- a transition relation, which is a finite set

$$\delta \subseteq Q \times A^* \times B^* \times Q.$$

In other words, such an automaton is a directed graph, where the vertices are states, and each edge is labelled by a pair of input/output words. A run of this automaton is a path that begins in an initial state and ends in a final state. The input string of the run is obtained by concatenating the input strings on the transitions, and likewise we define the output string. The semantics of the automaton is the relation

$$R \subseteq A^* \times B^*,$$

which consists of pairs of input/output strings, ranging over accepting runs of the automaton. The relations that arise this way are called rational, as stated in the following definition.

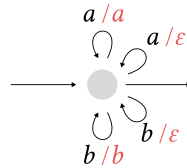
Definition B.1.2 (Rational relation). A rational relation is any relation that can be defined by a nondeterministic automaton with output.

Every Mealy machine is a special case of a rational relation, since it corresponds to a nondeterministic automaton with output in which: (a) every transition is labelled by a single input letter and a single output letter; (b) the automaton is deterministic; and (c) every state is accepting. Here are some other examples of rational relations, which are not Mealy machines.

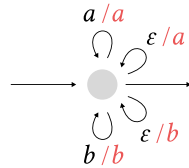
Example 7. [Subword relation] For every alphabet A , the subword relation

$$\{ (w, v) \in A^* \times A^* \mid v \text{ can be obtained from } w \text{ by deleting some letters} \}$$

is a rational relation. Here is a picture of the automaton when $A = \{a, b\}$:

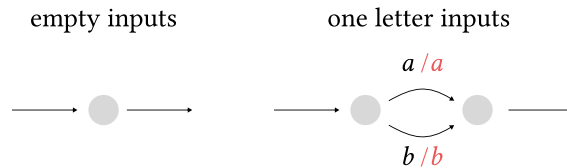


An important property of rational relations is that they are input/output symmetric. Therefore, the inverse of the subword relation, i.e. when letters can be added, is also rational, with the corresponding automaton looking like this:

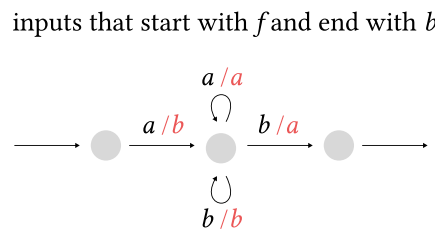


In particular, thanks to transitions that have ε on the input part, a rational relation can have infinitely many outputs for a single input. $\square$

Example 8. [Swap first and last] Here is an example of a rational relation which is a function, i.e. every input string has exactly one output string: the output string is obtained from the input string by swapping the first and last letters (if the input string has at most one letter, it is unchanged). To implement this function, we use a union of six nondeterministic automata. (In the pictures below, we use an alphabet with two letters.) The first two automata deal with inputs that have at most one letter:



The remaining four automata deal with the possible combinations of (first letter, last letter), with one of them being:



This function is not computed by a Mealy machine, even though it is letter-to-letter. The reason is that for a Mealy machine, the first output letter depends only on the first input letter. $\square$

Why the name rational? We finish this section by explaining the naming convention. The idea is that we can try to identify the “regular” languages in any monoid, and not just the free monoid A^* . To formalise regularity in an abstract monoid, two

ideas come to mind, which are based on (two-sided) Myhill-Nerode equivalence and regular expressions, respectively, as explained in the following definition.

Definition B.1.3. *[Rational and recognisable subsets of a monoid]*

- **The recognisable subsets.** A subset L of a monoid M is called recognisable if it is a union of finitely many equivalence classes for some equivalence relation $\sim$ on M that has finite index and is a congruence, i.e.

$$m_1 \sim m'_1 \wedge m_2 \sim m'_2 \quad \Rightarrow \quad m_1 \cdot m_2 \sim m'_1 \cdot m'_2.$$

- **The rational subsets.** A subset L of a monoid M is called rational if it can be obtained from finite subsets of M by applying a finite number of times the following operations: union, product, and Kleene star

$$L^* \stackrel{\text{def}}{=} \bigcup_{n \in \{0,1,\dots\}} L^n.$$

In the free monoid A^* , the two notions coincide, but in other monoids they do not coincide. In the monoid $A^* \times B^*$, the rational relations, as defined in Definition B.1.2, are exactly the rational subsets from the above definition, which can be proved by converting an NFA with output into a regular expression. The recognisable subsets of $A^* \times B^*$ are much weaker; these are relations that can be recognised by separately testing some regular properties of the input and output strings, as explained in Exercise B.1.1.

We have still not explained why the name “rational” is used in Definition B.1.3. The motivation comes from power series.

B.1.1 Composition and continuity

We now show that rational relations enjoy some of the important properties that we have identified in the introduction, namely closure under composition and continuity.

Theorem B.1.4. *Rational relations are closed under relational composition, i.e. if*

$$R \subseteq A^* \times B^* \quad \text{and} \quad S \subseteq B^* \times C^*$$

are rational relations, then the same is true for the composition

$$R \cdot S = \{ (u, v) \in A^* \times C^* \mid uRw \text{ and } wSv \text{ for some } w \in B^* \}$$

Proof. This is essentially the same product construction that we used for Mealy machines in Theorem A.1.3. By possibly splitting transitions into sequences of transitions, we can assume that in the automata for R and S , each transition produces at most one letter of input or output. Once this has been done, we synchronise the two automata using a product construction, in which two transitions

$$q \xrightarrow{u,w} q' \quad p \xrightarrow{w,v} p'$$

in the automata for R and S respectively are combined into a new transition

$$(q, p) \xrightarrow{u, v} (q', p')$$

in the automaton for the composition. The rest of the construction is as usual in a product construction, i.e. initial states are pairs that are initial on both coordinates, and likewise for final states. $\square$

We now show that rational relations are continuous. Since we are talking about relations and not functions, the notion of continuity from Definition .0.1 needs to be adapted, by taking inverse images under relations.

Theorem B.1.5. *If $R \subseteq A^* \times B^*$ is a rational relation and $L \subseteq B^*$ is a regular language, then the inverse image*

$$\{ w \in A^* \mid wRv \text{ for some } v \in L \}$$

is also a regular language.

Observe that rational relations are symmetric with respect to input and output, and therefore the above theorem also shows that forward images of regular languages under rational relations are also regular.

Proof. One proof is by repeating a product construction, similar to the one used for composition. However, we prefer to deduce continuity from composition directly, since this highlights how the two notions are related. Consider a rational relation with an empty output alphabet. There is only one possible output string, namely the empty string, and hence the relation can be described by saying which input strings can produce the unique output. All regular languages over the input alphabet arise this way, i.e. the map

$$R \mapsto \{ w \in A^* \mid wR\varepsilon \}$$

is a bijective correspondence between rational relations with an empty output alphabet and regular languages over the input alphabet.

Using the above bijective correspondence, we can prove the continuity property using closure under composition. Indeed, take some R and L as in the statement of the theorem. By Theorem B.1.4, the composition

$$R \cdot \{ (w, \varepsilon) \mid w \in L \}$$

is a rational relation. The corresponding language is exactly the inverse image of L under R , and hence it is regular. $\square$

B.1.2 Undecidable equivalence

The trouble with rational relations is that nondeterminism makes the equivalence problem undecidable.

Theorem B.1.6. *The equivalence problem $R = S$ is undecidable for rational relations.*

Proof. We will show undecidability already for a special case of the equivalence problem, namely the universality problem: is a rational relation equal to the full relation $A^* \times B^*$? We will do this via a reduction from the Post Correspondence Problem, which is known to be undecidable: given two string homomorphisms $g, h : A^* \rightarrow B^*$, decide if there is some input string where both homomorphisms produce the same output. (Recall that a string homomorphism is a function that applies some function of type $A \rightarrow B^*$ to each input letter. This is like the letter-to-letter homomorphisms, except that letters can also be erased, or replaced by longer strings.) The main observation is that the complement of a homomorphism can be defined by a rational relation.

Claim B.1.7. *If $h : A^* \rightarrow B^*$ is a homomorphism, then its complement defined by*

$$R = \{ (w, v) \in A^* \times B^* \mid v \neq h(w) \}$$

is a rational relation.

Proof. The automaton guesses a prefix of the input word for which the homomorphism is correctly applied. After reading this prefix and producing the correct output, it insists on making an error. This means that for the next letter a after the correct input prefix, it does one of the following: (i) outputs a string that is a proper prefix of $h(a)$ and then produces no more output; or (ii) outputs a string that is incomparable with $h(a)$ under the prefix relation and then produces any output. $\square$

Using the complements from the above claim, we will recast the Post Correspondence Problem as an instance of universality for rational relations. Consider an instance of the Post Correspondence Problem:

$$\exists w \in A^* h(w) = g(w).$$

The negation of this problem is

$$\forall w \in A^* \forall v \in B^* (w, v) \in \underbrace{(\text{complement of } g) \cup (\text{complement of } h)}_{\text{a union of two rational relations}}.$$

Since rational relations are closed under union, we have shown that the complement of the Post Correspondence Problem is the same as an instance of the universality problem for rational relations, and hence the latter problem must be undecidable. $\square$

The above problem is one of the reasons why we avoid relations in this book. The undecidability and related complications would get only worse for the more advanced models of transducers that we will be looking at later.

Exercises

Exercise B.1.1. Show that the recognisable subsets of $A^* \times B^*$ are exactly the unions of finitely many products of regular languages, i.e. unions of the form

$$K_1 \times L_1 \cup \dots \cup K_n \times L_n$$

where each K_i is a regular language over the input alphabet and each L_i is a regular language over the output alphabet.

B.2 Rational functions

After the brief detour into relations and nondeterminism, we return to the main focus of this book, which is about functions and deterministic models. As we will see, this will allow us to work around the undecidability result from Theorem B.1.6. We define rational functions semantically, as the functional fragment of the rational relations.

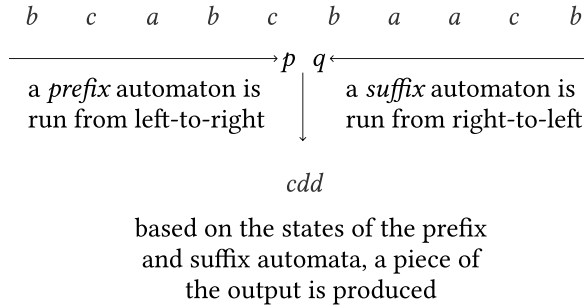
Definition B.2.1 (Rational function). *A rational function is the special case of a rational relation, in which each input string is related to exactly one output string.*

An advantage of the above definition is its simplicity, but a disadvantage is that the underlying automaton model is nondeterministic, and only happens to have unique outputs (possibly arising from different runs with the same output). The disadvantage will be overcome in Section B.2.1 below, where we introduce a deterministic model, called bimachines, which defines exactly the same class of functions.

Rational functions are closed under composition and are continuous, since these properties are inherited from rational relations. As we will see in the next chapter, on weighted automata, rational functions have decidable equivalence.

B.2.1 Bimachines

A bimachine is based on two automata (called the prefix and suffix automata), which are deterministic, but which will be used in opposite directions: the prefix automaton will be used in a left-to-right way, but the suffix automaton will be used in a right-to-left way. For each gap between positions in the input string, it produces a piece of the output according to the following picture:



Here is a more formal definition.

Definition B.2.2 (Bimachine). *A bimachine consists of the following ingredients:*

- *input and output alphabets A and B ;*
- *two deterministic automata with input alphabet A , which are called the prefix and suffix automata (the accepting states in these automata will be irrelevant, so they can be omitted);*

- an output function of type

$$(\text{states of prefix automaton}) \times (\text{states of suffix automaton}) \rightarrow B^*$$

A bimachine computes a function of type $A^* \rightarrow B^*$, which is defined as follows. Suppose that the input string is $a_1 \cdots a_n$. For each $i \in \{0, 1, \dots, n\}$, we consider the partition of the input word as

$$\underbrace{a_1 \cdots a_{i-1}}_{\text{prefix}} \quad \underbrace{a_i \cdots a_n}_{\text{suffix}}.$$

We run the prefix automaton on the prefix, and the suffix automaton on the reverse¹ of the suffix, yielding a pair of states. To this pair of states, we apply the output function, yielding a piece of the output string. The $n + 1$ pieces obtained this way are concatenated, in increasing order of i , thus yielding the output string.

We now show that bimachines compute exactly the rational functions. Along the way, we give another characterisation of the rational functions, which uses unambiguous nondeterministic automata with output, i.e. nondeterministic automata that have exactly one accepting run for each input string.

Theorem B.2.3. *The following are equivalent for a string-to-string function:*

1. *it is a rational relation which happens to be functional;*
2. *it is computed by an unambiguous nondeterministic automaton with output;*
3. *it is computed by a bimachine.*

Proof. We prove the implications in the order $3 \Rightarrow 1 \Rightarrow 2 \Rightarrow 3$.

From bimachines to functional. We begin with the easiest inclusion

$$\text{bimachines} \subseteq \text{functional rational relations}.$$

Consider some bimachine. The states of the corresponding nondeterministic automaton are pairs (states of prefix automaton, state of suffix automaton), plus one extra state that we will denote by $\perp$. The essential idea is that the nondeterministic automaton guesses the runs of both the prefix and suffix automaton. More formally, we have transitions of the form

$$(p, q) \xrightarrow{a/w} (p', q'),$$

¹Using the reverse of the suffix yields a pleasing symmetry in the execution of the bimachine, since both automata run in the direction of the gap. It will also be useful in the proof of Theorem B.2.3. However, running the suffix automaton in the forward direction would give the same class of functions, since the only role of the suffix automaton is to partition the suffixes into finitely many regular languages, and the reverse of a regular language is regular.

where a is an input letter, w is the output string produced by the pair (p, q) and the following are transitions of the prefix and suffix automata:

$$\underbrace{p \xrightarrow{a} p'}_{\text{in prefix automaton}} \quad \underbrace{q' \xleftarrow{a} q}_{\text{in suffix automaton}}.$$

Since we have used the output string for (p, q) and (p', q') , the resulting automaton outputs for each input letter the output of the bimachine that corresponds to the gap on the left of the letter. This accounts for all gaps except the last one, which is where we use the extra state $\dashv$, for which we have transitions

$$(p, q) \xrightarrow{\varepsilon/w} \dashv,$$

such that w is the output on (p, q) and q is the initial state of the suffix automaton. The initial states in the automaton are pairs (p, q) where p is an initial state of the prefix automaton and q is any state of the suffix automaton. There is only one final state, namely $\dashv$. It is easy to see that the resulting automaton computes the same function as the bimachine.

From functional to unambiguous. We now prove the inclusion

$$\text{functional rational relations} \subseteq \text{unambiguous rational relations}.$$

In the construction, and also later in this book, it will be convenient to eliminate ε -transitions, i.e. transitions with empty input. There are two caveats to eliminating ε -transitions, which is that they are essential for two roles: (a) producing an output for the empty input string; and (b) producing infinitely many outputs for one input string. To deal with issue (a), we will discuss the empty input separately. To deal with issue (b), we allow *extended transitions*, where instead of an output string, a transition is labelled by a regular language of output strings. Subject to these two caveats, we can eliminate ε -transitions, as the following lemma shows.

Lemma B.2.4 (Elimination of ε -transitions). *Every rational relation can be computed by an NFA with output and extended transitions, where each accepting run has the following properties:*

1. *if the input string is nonempty, then each transition inputs exactly one letter;*
2. *if the input string is empty, then the run has exactly one transition.*

Furthermore, if the rational relation is such that every input string has finitely many outputs, then extended transitions are not needed.

Proof. Without loss of generality, we can assume that: (a) there is no state that is both initial and final; and (b) every transition in the automaton consumes zero or one input letters. Both of these assumptions can be achieved by using ε -transitions. Furthermore, we assume that (c) every state appears in at least one accepting run.

Having made the above assumptions on the original automaton, we can apply the usual construction for eliminating ε -transitions. We remove all transitions from the

automaton, and replace them as follows. For each input letter a and states p and q , we create a transition of the form

$$q \xrightarrow{a/L} p,$$

where L is the set of outputs that can be produced by a run in the original automaton which starts in q , consumes the letter a and uses any number of ε -transitions, and ends in p . This set is easily seen to be a regular language. As a result, the new automaton has the same outputs for all nonempty inputs strings. Finally, in order to take care of the empty input string, we add a fresh initial state $\vdash$ and fresh final state $\dashv$, together with a transition

$$\vdash \xrightarrow{\varepsilon/L} \dashv,$$

in which L is the set of outputs that can be produced by the original automaton on an empty string.

Finally, if the original automaton had the property that every input string has finitely many outputs, then the regular languages L that label the transitions in the new automaton are finite, and therefore we can replace each transition by a finite number of transitions that produce exactly one output string. $\square$

Using the above lemma, we show that a functional rational relation can be computed by an unambiguous automaton with output. In fact, we will show a stronger result, which we call uniformisation: if a rational relation is total (i.e. every input string produces at least one output string), then it contains an unambiguous rational relation. In the case of a functional rational relation, the totality assumption is satisfied, and the contained unambiguous rational relation must be the same function.

Lemma B.2.5. *[Uniformisation] If a rational relation is total, then it contains an unambiguous rational relation.*

Proof. We begin by eliminating ε -transitions, using the previous lemma. As a result, the automaton might use extended transitions, but this will not be an issue. We call this the original automaton, and our goal is to define a new unambiguous automaton.

Choose some arbitrary linear order on the state space of the original automaton. The new unambiguous automaton will use the run of the original automaton, which is lexicographically minimal with respect to the chosen linear order on states. This new automaton is defined as follows.

- **States.** We have the two special states $\vdash$ and $\dashv$, which will be used for the empty input, similarly to the construction in Lemma B.2.4. Apart from these two states, the new automaton has a state of the form $P \ni p$ for every subset P of states in the original automaton and every chosen element p of this subset. The idea is that P is the set of states that can reach acceptance in the future, and p is the chosen state used in the run with lexicographically minimal profile.
- **Initial states.** The special state $\vdash$ is initial. A state of the form $P \ni p$ is initial if and only if p is initial in the original automaton, and P does not contain any smaller initial states of the original automaton.

- **Final states.** The special state $\dashv$ is final. A state of the form $P \ni p$ is final if and only if p is final in the original automaton.
- **Transitions.** For the two special states $\vdash$ and $\dashv$, there is a unique transition

$$\vdash \xrightarrow{\varepsilon/w} \dashv$$

where w is some arbitrarily chosen output that can be produced by the original automaton on an empty input. Next, we create transitions of the form

$$P \ni p \xrightarrow{a,w} P' \ni p'$$

under the following conditions:

1. P is the set of all states that can reach P' by reading input a ;
2. p' is the minimal state in P' that can reach p by reading input a ; and
3. w is some chosen output on a transition from p to p' that inputs a .

In the above, the chosen output in item 3 is unique for the automaton, which means that once the source and target states and the input letter have been fixed, it is impossible to have two different outputs.

Let us now argue that this new automaton is unambiguous. The first component of the state $P \ni p$, which is the subset P , is chosen deterministically from right-to-left. Therefore, all runs of the new automaton will agree on this component. Once the first component has been fixed, the second component is deterministic from left-to-right, and therefore all runs will also agree on this component. Finally, the output string is determined uniquely by the states and the input letter. It is also not hard to see that if the original automaton had an accepting run (which is guaranteed by the totality assumption), then the new automaton also has an accepting run, and that the output string is the same. $\square$

From unambiguous to bimachines. The final step in the theorem is the inclusion

$$\text{unambiguous rational relations} \subseteq \text{bimachines}.$$

Consider an unambiguous automaton with output $\mathcal{A}$ that computes some function. We can assume without loss of generality that this automaton has the following property: every accepting run uses exactly one ε -transition (i.e. with empty input), and this is the last transition of the run. This property was ensured by the construction in the previous step, by the proof of Lemma B.2.5. We now design a bimachine that computes the same output.

The bimachine produces the output as follows. For each gap in the input string (i.e. a factorisation into prefix and suffix), the following output is produced: (a) if the gap is not the last one, i.e. the suffix begins with some input letter, then we produce the part of the output that corresponds to the transition of the original automaton which consumes this letter; and (b) if the gap is the last one, i.e. the suffix is empty, then

we produce the part of the output that corresponds to the ε -transition at the end of the run of the original automaton. It remains to define the prefix and suffix automata so that the above can be implemented. The prefix automaton computes the states that can be reached from an initial state by reading the prefix. The suffix automaton computes a little more information, namely: is the suffix empty (i.e. are we at the last gap), and if the suffix is nonempty, then:

- what is the first letter of the suffix; and
- which states can reach an accepting state via the suffix minus its last letter.

Using this information we can determine the output that should be produced at the gap, as described above. $\square$

B.2.2 Decomposition into primes

Using bimachines, we can prove a variant of the Krohn-Rhodes decomposition theorem for rational functions. In fact, most of the decomposition work was already done in the case of Mealy machines. In the theorem, we use right-to-left Mealy machines, which are defined the same way as Mealy machines, except that the input string is processed in a right-to-left manner, with the initial state being used after the rightmost position.

Theorem B.2.6. *A string-to-string function is rational if and only if it can be obtained by composing the following functions (which we call the prime rational functions):*

1. *the prime Mealy machines, i.e. reversible and flip-flop Mealy machines;*
2. *the right-to-left variants of the prime Mealy machines;*
3. *string homomorphisms;*
4. *the function $w \mapsto w\#$ which appends a fresh separator symbol to the input string.*

Proof. Clearly all prime functions are rational, and rational functions are closed under composition, which gives us the implication $\Leftarrow$. Let us now prove the implication $\Rightarrow$.

By Theorem B.2.3, we can assume that f is computed by a bimachine. We first apply the function $w \mapsto w\#$. Next, we run the past and future automata on the string, and label the input string with the states of these automata. The state is stored in the position to its right, which is why we use the separator symbol $\#$ to store the state at the end of the string. The past and future automata are Mealy machines, with the future one being a right-to-left Mealy machine. Finally, we apply a homomorphism, which produces the desired output string. $\square$

B.2.3 Mealy machines as a subset of the rational functions

Mealy machines are a special case of rational functions. The following theorem explains exactly which special case they are.

Theorem B.2.7. Consider a rational function $f : A^* \rightarrow B^*$. This function is computed by a Mealy machine if and only if it satisfies the following two properties:

- Letter-to-letter. For every input string, the output string has the same length;
- Determinism. If two input strings agree on the first n input letters, then the corresponding output strings also agree on the first n output letters.

Exercises

Exercise B.2.1. Show that rational relations are not closed under intersection.

Exercise B.2.2. Show that it is undecidable if two rational relations have nonempty intersection

Exercise B.2.3. Show that there is a function

$$f : A^* \rightarrow B^*$$

which is not rational, but has the property that for every rational function

$$g : B^* \rightarrow \underbrace{1^*}_{\text{words over a one-letter alphabet}},$$

the composition $f \cdot g$ is rational.

Exercise B.2.4. Consider two rational functions f and g , possibly with different input and output alphabets. We say that f factors through g if one can decompose f as

$$f = g_1 \cdot g \cdot g_2$$

for some rational functions g_1 and g_2 . Show an algorithm that decides this property, given rational functions f and g .

Exercise B.2.5. A set I of rational functions is called an ideal if

$$I = \text{Rational} \cdot I \cdot \text{Rational},$$

i.e. functions that factor through the ideal must stay in the ideal. Show that there are finitely many ideals.

Exercise B.2.6. Consider a rational relation $R \subseteq A^* \times B^*$, such that for every input string there is at least one output. Show that there is a rational function $f : A^* \rightarrow B^*$ such that $wRf(w)$ holds for every input string.

Exercise B.2.7. Show that if a rational function $f : A^* \rightarrow B^*$ is surjective, then it has a one-sided inverse, i.e. a rational function $g : B^* \rightarrow A^*$ such that $g \cdot f$ is the identity on B^* .

Exercise B.2.8. Show that the following problem is undecidable: given a rational function f , we want to know if it is injective, i.e. different input strings are mapped to different output strings.

Exercise B.2.9. Show that the following problem is undecidable: given a rational function $f : A^* \rightarrow A^*$, we want to know if it generates finitely many functions under composition, i.e. if the following set is finite:

$$\{ f^n \mid n \in \{0, 1, 2, \dots\} \}.$$

Exercise B.2.10. Show that the following two conditions are equivalent for a rational relation:

1. for every input string, all output strings have the same length as the input;
2. it is computed by an NFA with output, in which for every transition, the length of the input and output strings are the same.

Hint: for two states p and q in an NFA with output, consider the set

$$\Delta_{pq} = \{ (|w| - |v|) \mid \text{there is a run } p \xrightarrow{v/w} q \}.$$

$$\begin{array}{c|c}
x + (y + z) = (x + y) + z & x \times (y \times z) = (x \times y) \times z \\
x + y = y + x & 1 \times x = x \\
x + 0 = x & 0 \times x = 0 = 0 \times x \\
x \times (y + z) = x \times y + x \times z & \\
(x + y) \times z = x \times z + y \times z &
\end{array}$$

Figure 1: The semiring axioms.

B.3 Rational relations and weighted automata

In this section, we describe the connections of rational relations (including their functional fragment) with another automaton model, called weighted automata. Using this connection and a certain decidability result for weighted automata, we will show that equivalence of rational functions is decidable. This connection will also be used in the future, to decide equivalence for a more powerful transducer model, namely the regular functions.

B.3.1 Weighted automata

We begin by defining weighted automata. For the equivalence algorithm, we will be mainly interested in weighted automata that have outputs in a field, but for other results, we will use the more general variant with outputs in an arbitrary semiring. Intuitively speaking, a semiring is like a field, except that we do not require any kind of inverses (both additive and multiplicative), and multiplication can be non-commutative. Here is the formal definition.

Definition B.3.1 (Semiring). *A semiring is a set $\mathbb{S}$ equipped with two binary operations $+$ and $\times$, and two distinguished elements 0 and 1 , subject to the axioms in Figure 1.*

Instead of carefully reading the semiring axioms, for our applications it will be more useful to see the examples.

Example 9. [Fields and rings] Every field, such as the field of rationals $(\mathbb{Q}, +, \times)$, is a semiring. This is true also for rings, such as the ring of integers $(\mathbb{Z}, +, \times)$. Finally, if we restrict the integers to the natural numbers, the result $(\mathbb{N}, +, \times)$ is still a semiring. $\square$

Example 10. [Boolean semiring] In this semiring, the underlying set is $\{0, 1\}$, the semiring addition is $x \vee y$, and the semiring multiplication is $x \wedge y$. Both operations are commutative, but neither of them has an inverse. $\square$

Example 11. [Semiring of regular languages] In this semiring, the underlying set is the regular languages, semiring addition is union $L \cup K$, and semiring multiplication is concatenation $L \cdot K$. $\square$

Definition B.3.2 (Weighted automaton). *A weighted automaton over a semiring $\mathbb{S}$ is defined in the same way as an NFA with output, except that:*

1. instead of output strings, transitions use elements of the output semiring; and
2. we require that for every input string, there are finitely many accepting runs.

The semantics of a weighted automaton is the function which maps an input string w to the sum

$$\sum_{\rho} \text{output of } \rho,$$

where the sum ranges over accepting runs over the input string, and the output of a run is the multiplication (in the semiring) of the weights of its transitions. If multiplication in the semiring is non-commutative, then it is important to multiply the transition weights in the order in which they are taken in the run. The assumption that there are finitely many accepting runs for every input string ensures that the above sum is well-defined.

Example 12. [Weighted automata over the Boolean semiring] When the semiring is the Boolean semiring, then a weighted automaton is the same as a nondeterministic automaton. Therefore, the functions computed by such weighted automata are exactly the regular languages. $\square$

Just as regular languages arise as a special case of weighted automata in the Boolean semiring, the rational relations will arise for a different choice of semiring, namely the semiring of regular languages. This is shown in the following example.

Example 13. [Weighted automata over the semiring of regular languages] When the semiring is the semiring of regular languages, then weighted automata are the same as rational relations, i.e. a relation

$$R \subseteq A^* \times B^*$$

is rational if and only if the function

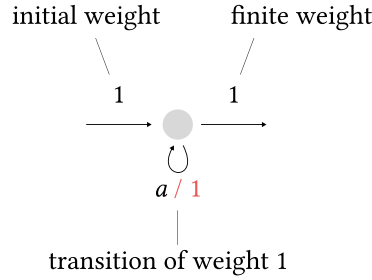
$$w \in A^* \mapsto \{ v \in B^* \mid wRv \}$$

is computed by a weighted automaton with outputs in the semiring of regular languages. This is not hard to see. To go from a rational relation to a weighted automaton, we simply use outputs on transitions that are singleton regular languages. In the opposite direction, a transition

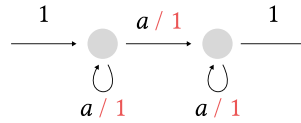
$$p \xrightarrow{a/L} q$$

in a weighted automaton can be replaced by a copy of an NFA that recognises the language L . $\square$

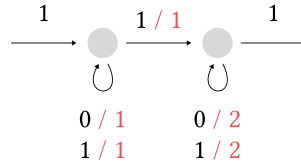
Example 14. [Weighted automata over $\mathbb{N}$] Let us give some examples of weighted automata over the semiring of natural numbers. Consider first the following automaton, which has a one-letter input alphabet:



This weighted automaton has exactly one run of weight 1 for each input word, and therefore it computes the constant function $w \mapsto 1$. If we want to compute the length of the input string, then we can use the following weighted automaton, which has one run of weight 1 for each input position:



Finally, consider the following weighted automaton with input alphabet $\{0, 1\}$:



For each position with label 1 in the input string, this automaton has a run whose weight is 2^i , where i is the distance until the end of the input string. Therefore, this automaton maps an input string to the number that is represented by it in binary notation. $\square$

B.3.2 Decidable equivalence

In this section, we leverage the connection with weighted automata to decide equivalence of rational functions. To do this, we will use decidability of equivalence for weighted automata over the field of rationals.

Theorem B.3.3. *Given two weighted automata over the field of rationals, it is decidable whether they compute the same function.*

The above result is well-known, but in the interest of completeness, we give a proof of it in the next section. We have stated it for the field of rationals, which is the field

that we will use in our applications to string-to-string transducers. Nevertheless, the same proof will work for any other field, as long as field elements can be represented in a finite way and field operations can be computed. Let us now show how to use the above result to decide equivalence of rational functions.

Theorem B.3.4. *The equivalence problem $f = g$ is decidable for rational functions.*

Proof. We reduce the problem to equivalence for weighted automata over the field of rationals. For this reduction, we need three observations: (a) output strings can be injectively represented using rational numbers; (b) weighted automata are closed under pre-composition with rational functions; and (c) equivalence is decidable for weighted automata over the field of rationals. Here is a picture of the reduction:

$$\begin{array}{ccc} A^* & \xrightarrow{f} & B^* \xrightarrow{\iota} \mathbb{Q} \\ & \xleftarrow{g} & \end{array}$$

In the picture, f and g are two rational functions, and ι is the injective weighted automaton from observation (a). By injectivity, the equivalence problem $f = g$ has the same answer as the equivalence problem $f \cdot \iota = g \cdot \iota$. By the closure from observation (b), the latter is an instance of equivalence for weighted automata, and hence decidable by observation (c). The decidability from observation (c) is a well-known result, but we give a self-contained proof in the next section for completeness. It remains to prove the two observations (a) and (b).

For observation (a) about an injective representation, we can use the weighted automaton from Example 14, which returns the natural number (a special case of a rational number), that is represented by the input string in binary (or any other base, for input alphabets with more than two letters). The only obstacle is leading zeros in the input string; to circumvent this obstacle the weighted automaton can begin by prepending the digit 1.

Observation (b) is proved in the following lemma. We prove the lemma for arbitrary semirings, and not just the field of rationals, since the more general result will be useful in the future.

Lemma B.3.5. *Weighted automata (over any semiring, not necessarily the field of rationals) are closed under pre-composition with rational functions, i.e. the composition*

$$\begin{array}{ccc} A^* & \xrightarrow{f} & B^* \xrightarrow{h} \mathbb{S} \\ \underbrace{\quad} & \underbrace{\quad} & \\ \text{a rational} & \text{a weighted} & \\ \text{function} & \text{automaton} & \end{array}$$

is necessarily a weighted automaton.

Proof. We use a product construction. To make this construction correct, we want to ensure that there is a one-to-one correspondence between accepting runs of the product automaton and runs of the second weighted automaton. This will be achieved by making the first automaton (for f) unambiguous, and appropriately aligning the two runs in the product constructions.

By Lemmas B.2.4 and B.2.5, we assume that the automaton for the rational function f is: (a) unambiguous; and (b) for every nonempty input string, each transition used in the unique accepting run consumes exactly one input letter. Condition (a) will be useful to ensure that there is no double counting, and condition (b) will be used to align the runs of the two automata in the product construction.

The weighted automaton for the composition is now constructed as follows. The states of the new weighted automaton in the conclusion of the theorem are pairs (state of f , state of h), plus two extra states $\vdash$ and $\dashv$. The initial states are $\vdash$ and every pair (p, q) that is initial on both coordinates, in the corresponding automata. The state $\dashv$ is the unique final state. Transitions between states that are pairs are of the form

$$(p, q) \xrightarrow{a/s} (p', q')$$

where a is an input letter and $s \in \mathbb{S}$ is the sum of weights of runs in the weighted automaton that have the following properties: the source state is q , the target state is q' , the input string is a , and the letter is consumed in the last transition. To reach the unique final state from pair states, we have transitions of the form

$$(p, q) \xrightarrow{\varepsilon, s} \dashv,$$

where s is the sum of weights for runs in the weighted automaton that have the following property: the source state is q , the target state is final, and the input string is empty. Finally, we need a transition that takes care of the empty input string, which is of the form

$$\vdash \xrightarrow{\varepsilon, s} \dashv,$$

where s is the output of the weighted automaton on the input string $f(\varepsilon)$. □

□

In the proof above, we established that if a function is rational, then weighted automata are closed under pre-composition with it. This can be seen as a variant of continuity, except that instead of regular languages (i.e. weighted automata over the Boolean semiring), we are using weighted automata over the field of rationals. As it turns out, this variant of continuity is exactly the same as rationality.

Theorem B.3.6. *A string-to-string function $f : A^* \rightarrow B^*$ is rational if and only if weighted automata are closed under pre-composition with f .*

Proof. The left-to-right implication was shown in Lemma B.3.5. For the right-to-left implication, consider the semiring of regular languages, and the function

$$w \in B^* \mapsto \{w\} \in \mathbb{S}.$$

By closure under pre-composition, it follows that

$$w \in A^* \mapsto \{f(w)\} \in \mathbb{S}$$

is computed by a weighted automaton over the semiring of regular languages. By Example 13, this means that the function f is rational. □

B.3.3 Decidability of equivalence for weighted automata over a field

In this section, we show that equivalence of weighted automata over the field of rationals is decidable. We first observe that the essential problem is zeroness, i.e. equivalence with the zero function. This is because

$$f = g \quad \text{iff} \quad f - g = 0,$$

and weighted automata can be subtracted from each other. It remains to show decidability of the zeroness problem.

Theorem B.3.7. *The zeroness problem is decidable for weighted automata over the field of rationals².*

Proof. Assume that Q is the set of states in the weighted automaton, and A is the input alphabet. We begin by recasting the semantics of the weighted automaton in terms of vectors in $\mathbb{F}^Q$ and linear maps that act on them. Define the *initial vector*

$$q_0 \in \mathbb{F}^Q$$

as follows: on the initial states it has 1, and on the non-initial states it has 0. For each input letter $a \in A$, define its *update map*

$$\delta_a : \mathbb{F}^Q \rightarrow \mathbb{F}^Q$$

to be the linear map for which in the corresponding $Q \times Q$ matrix, the coefficient in cell (p, q) stores the sum of weights of runs in the automaton that go from state p to state q on the input string a , and which consume the letter a in the last transition. Define the *final map*

$$F : \mathbb{F}^Q \rightarrow \mathbb{F}$$

as follows: it is the linear map where the coefficient next to state q is the sum of weights of runs in the automaton that start in state q and end in a final state, and which consume no input letters. The data above was constructed so that if we take an input string, then the output of the weighted automaton on this input string is obtained as follows: start in the initial vector, then apply the update maps corresponding to the input letters, and then apply the final map.

Define a *reachable vector* to be a vector that can be obtained from the initial vector by applying a finite sequence of linear maps corresponding to input letters. The zeroness problem is equivalent to

(*) F outputs zero for all reachable vectors.

Since the map F is a linear map, the above is equivalent to

(**) F outputs zero for all linear combinations of reachable vectors.

²The proof works for any computable field, where field elements can be represented in a finite way and field operations can be computed.

Because the update maps are linear maps, the above is equivalent to:

(***) F outputs zero on the least subset of $\mathbb{F}^Q$ that contains the initial vector, and is closed under applying update maps and taking linear combinations.

The subspace (***) can be computed using a simple saturation procedure, where we start with a basis that contains only the initial vector, and we keep adding new basis vectors by applying update maps, until no new basis vectors can be added. This must terminate with at most $|Q|$ basis vectors. At the end, we check if the final map is zero on all basis vectors. $\square$

B.4 Machine independent characterisations

In this section, we present machine independent characterisations of transducers, such as Mealy machines and the rational functions. The idea is to describe a class of functions uniquely in terms of the semantic properties of the functions – such as continuity – and not in terms of some computational model. For Mealy machines, the machine independent characterisation will be straightforward, but it will be substantially harder for the rational functions. In fact, our characterisation for rational functions will require developing characterisations for other intermediate classes that lie between Mealy machines and rational functions.

B.4.1 Mealy machines

As a warmup, we give a machine independent characterisation of Mealy machines. We say that a function is prefix preserving if

$$w \text{ is a prefix of } v \implies f(w) \text{ is a prefix of } f(v).$$

Similarly, a length preserving function is one in which the length of the input string is the same as the length of the output string. Clearly Mealy machines have these two properties, and they are continuous. It turns out that these properties exactly characterise Mealy machines, as the following theorem shows.

Theorem B.4.1. *A function $f : A^* \rightarrow B^*$ is computed by a Mealy machine if and only if it is: (a) continuous; (b) prefix preserving; and (c) length preserving.*

Proof. Clearly a Mealy machine satisfies all conditions in the theorem. Let us now look at the converse. For each output letter $b \in B$, consider the language

$$\{ w \in A^* \mid f(w) \text{ ends with } b \}, \quad (3)$$

which is regular by continuity. Because the function preserves the length and prefixes, we know that each input position contributes exactly one output letter, and furthermore, we can determine which one using finite state control, thanks to the regularity of the languages (3) defined above. Formally speaking, the state space of the Mealy machine is the product of the state spaces of all languages from (3). $\square$

Using the above theorem, we show that the Mealy machines, seen as a subset of the rational functions, are decidable.

Theorem B.4.2. *One can decide if a rational function is computed by a Mealy machine.*

Proof. It is enough to show that we can check if a rational function satisfies the three conditions in Theorem B.4.1. Continuity is for free, since rational functions are necessarily continuous. Let us discuss the other two conditions.

Lemma B.4.3. *One can decide if a rational function is length-preserving.*

Proof. We give two arguments. The first argument is based on semilinear sets. Since we only sketch this argument, we do not recall the definition of semilinear sets. Consider the set of input/output length pairs

$$\{ (|w|, |f(w)|) \mid w \in A^* \} \subseteq \mathbb{N} \times \mathbb{N}.$$

This set is semilinear and can be computed using the Parikh image. The function is length-preserving if and only if this set is contained in the diagonal $\{ (n, n) \mid n \in \mathbb{N} \}$. Since the diagonal is semilinear and containment is decidable for semilinear sets, we get a proof of the lemma. This concludes the first argument.

Let us now present in more detail a second argument, which is more direct and does not use semilinear sets. Fix an NFA with output that computes f , and which has states Q . Define a *typing* for this automaton to be a function

$$\tau : Q \rightarrow \mathbb{Z}$$

such that for every run ρ of the automaton from an initial state to q , we have

$$|\text{output string of } \rho| = |\text{input string of } \rho| + \tau(q).$$

The typing, if it exists, is clearly unique.

Claim B.4.4. *The function f is length-preserving if and only if the typing exists, and all accepting states are mapped to zero.*

Proof. The right-to-left direction is immediate. For the opposite direction, we observe that: (1) if the typing does not exist, then one can find two runs that reach the same state q and have different differences between the output and input lengths, which means that the function cannot be length-preserving; and (2) if the typing exists, but some accepting state has nonzero type, then also the function cannot be length-preserving, since we can find a run that reaches this accepting state and has a difference between output and input lengths equal to this nonzero type. $\square$

Therefore, it remains to check if the conditions from the above claim are satisfied. The unique candidate for the typing can easily be computed, by looking at runs that reach every state. Once we have a candidate for the typing, we need to check if

$$|w| = |v| + \tau(q) - \tau(p) \quad \text{for every transition } p \xrightarrow{v/w} q$$

and all accepting states have type zero. This concludes the second argument for proving the lemma. $\square$

It remains to check the prefix-preserving property. Before doing this, we prove a characterisation of the length-preserving functions.

Lemma B.4.5. *If a rational function is length-preserving, then it is computed by an NFA with output where for each transition, the input and output strings have the same length.*

Proof. In this proof, it will be convenient to use possibly negative strings, which can be seen as living in the free group. Formally speaking, the free group over generators B , which is denoted by $B^{(*)}$, consists of strings over the alphabet

$$\{ b, b^{-1} \mid b \in B \}$$

which are reduced, i.e. they do not contain two consecutive letters that cancel each other out. The multiplication operation in the free group is defined by concatenation followed by reduction. The free group will also be used later in this chapter, in the proof of Theorem B.4.8.

Consider a rational function f that is length preserving and computed by some NFA with output. By Claim B.4.4, there must be some typing $\tau : Q \rightarrow \mathbb{Z}$ which maps all accepting states to zero. Define a new NFA with output as follows. The states are pairs (q, x) where x belongs to the free group $B^{(*)}$ and has length $\tau(q)$. This allows for negative lengths, since we assume that the inverse letters have length -1. The transitions of the automaton are defined by

$$\underbrace{(q, x) \xrightarrow{w/v} (p, y)}_{\text{is a transition in the new automaton}} \quad \text{if and only if} \quad \underbrace{q \xrightarrow{w/x^{-1}vy} p}_{\text{is a transition in the original automaton}}. \quad (4)$$

In the above, w and v are required to be normal strings, without negative letters, and also $x^{-1}vy$ must be a normal string, since it appears in a transition of the original automaton. These transitions in the new automaton are length-preserving, as required by the conclusion of the lemma, because

$$|w| = |x^{-1}vy| + \tau(q) - \tau(p) = |v|.$$

The initial states are those which use an initial state of the original automaton, and likewise for the accepting states. Observe that both initial and final states are annotated by empty strings, since these states have type zero: initial states have type zero by definition and final states have type zero by assumption that the automaton is length-preserving. The condition (4) that defines transitions in the automaton also extends to runs, and thus the new automaton computes the same function as the original automaton. $\square$

We now complete the proof of the theorem. Consider some NFA which computes a rational function. We first check if it is length-preserving, using Lemma B.4.3, and if it is, then we use Lemma B.4.5 to compute a new NFA in which all transitions are length-preserving. (This can be done because the proof of Lemma B.4.5 is constructive.) By possibly splitting transitions, we can assume without loss of generality that every transition consumes zero or one letters (both on input and output). As usual, we also assume that all states are productive, i.e. every state appears in some accepting run. The function computed by the automaton is *not* prefix-preserving if and only if one can find a pair of transitions

$$p_1 \xrightarrow{v_1/w_1} q_1 \quad p_2 \xrightarrow{v_2/w_2} q_2$$

such that the following three conditions are satisfied:

1. the inputs v_1 and v_2 are the same; and
2. the outputs w_1 and w_2 are different; and
3. p_1 and p_2 can be reached from initial states using a common input string.

Since each transition consumes at most one letter, it follows from conditions 1 and 2 that the inputs and outputs of the two transitions use exactly one letter. To check if a pair of transitions with the above conditions exists, one can enumerate all pairs of transitions, and use a product construction to check if the two states p_1 and p_2 can be reached from initial states using a common input string. $\square$

B.4.2 Sequential functions

Our ultimate goal in this chapter is to characterise the rational functions. Before we do that, however, we will characterise an intermediate model that lies between Mealy machines and rational functions, which is called the *sequential functions*. These are defined like Mealy machines, except that the transition function has type

$$Q \times A \rightarrow Q \times B^*,$$

which means that a transition can produce an output string of variable length, instead of exactly one letter.

Theorem B.4.6. *A function $f : A^* \rightarrow B^*$ is sequential if and only if it: (a) is continuous; (b) is prefix preserving; and (c) has the following bounded increase property:*

$$\overbrace{\sup_{\substack{w \in A^* \\ a \in A}} |f(wa)| - |f(w)|}^{\text{maximal increase in output length due to extending input with one letter}} < \infty.$$

Proof. Again, it is clear that sequential functions have the properties in the theorem. Let us prove the converse. Consider a function f that satisfies the three properties. Define the *derivative* of this function to be

$$A^+ \xrightarrow{f'} B^* \\ wa \mapsto (f(w))^{-1} \cdot f(wa).$$

This function tells us what is the contribution of the last letter to the output. The derivative is well-defined thanks to the prefix-preserving property, and it has finite image thanks to the bounded increase property. The output string of f can be computed by concatenating outputs of the derivatives

$$f(a_1 \cdots a_n) = f'(a_1) \cdot f'(a_1 a_2) \cdots f'(a_1 \cdots a_n).$$

Therefore, in order to show that f is a sequential function, it suffices to show that the derivative f' is computed by a finite automaton, i.e. for each of the finitely many output values, the corresponding set of inputs is a regular language. We do this in two steps.

- We first show that a finite automaton can determine the length of $f'(w)$. To do this, use the continuity assumption to compute the length

$$|f(w)| \mod k,$$

where k is some number that is bigger than the maximal output of the derivatives. This gives the length of the derivative, since the length of $f'(wa)$ is the unique number $\ell < k$ such that

$$|f(w)| + \ell \equiv |f(wa)| \mod k.$$

- Once we know the length of the derivative $f'(wa)$, we determine its output in the same way as for Mealy machines, i.e. we can use continuity to ask what are the last $|f'(wa)|$ letters of the output $f(wa)$.

□

B.4.3 Subsequential functions

In this section, we extend the characterisation of sequential functions by considering a model that adds two new features: (a) the function is partial, i.e. it might have undefined outputs; and (b) the function knows when the input string ends. The model is called the *subsequential functions*, and the corresponding transducer is defined like a sequential transducer, except that its syntax is extended with an *end-of-input function*, which is a partial function of type $Q \rightarrow B^*$. The end-of-input function is applied to the last state in the computation. If the result is undefined, then the entire output string is undefined, and otherwise the result is appended to the previously produced output string.

Example 15. [Append b] One of the advantages of the end-of-input function is that it allows producing nonempty output on an empty input. For example, the function $a^n \mapsto a^n b$ is subsequential. We use a single state, in which the input is copied, and then the end-of-input function adds one more letter b . □

The subsequential functions seem to be a minor variation on the sequential ones, and yet their machine independent characterisation will be considerably harder than the one for sequential functions. Nevertheless, this effort will pay off in the next section, where subsequential functions will be used as a crucial ingredient in the characterisation of rational functions.

One of the difficulties in subsequential functions is that the condition on preserving prefixes is no longer relevant. This condition will no longer be necessary: because of the end-of-input function, the functions will no longer be prefix-preserving, such as the function in Example 15. It will also not be sufficient, since any function can be turned into a partial function that is prefix preserving.

Example 16. [Endmarkers] Take any string-to-string function f . We can extend the input alphabet with a distinguished endmarker $\dashv$, and define a new function by

$$f_{\dashv}(w) \stackrel{\text{def}}{=} \begin{cases} f(v) & \text{if } w = v \dashv \text{ for some } v \text{ that does not use } \dashv \\ \text{undefined} & \text{otherwise.} \end{cases}$$

In the new function, the domain contains only words that end with the endmarker $\dashv$ and do not use it anywhere else. Such words are incomparable with respect to the prefix relation, and therefore the new function is prefix-preserving. This way, we can turn any continuous function into one that is partial and prefix-preserving, and as we have seen in the introduction, there are plenty of ill-behaved continuous functions. Also, this example shows that the end-of-input function is somehow less important than the partiality feature, since f is subsequential if and only if $f_{\dashv}$ is subsequential without using the end-of-input function. Nevertheless, the end-of-input function will make the characterisation of subsequential functions more elegant, and so we keep it in the definition. $\square$

Because of the difficulty highlighted in the above example, we need a variant of the condition on preserving prefixes. This variant will be based on modifying strings by minor variations on the suffix (such as removing an endmarker), and is formalised in the following definition.

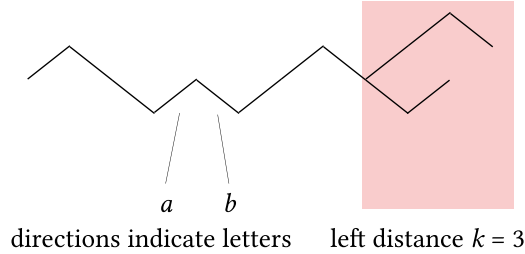
Definition B.4.7 (Left distance). *For two strings w_1 and w_2 , define their left distance*

$$\|w_1, w_2\|$$

to be the smallest k such that the strings can be decomposed as

$$w_1 = vv_1 \quad w_2 = vv_2 \quad \text{with } |v_1|, |v_2| \leq k$$

In the decomposition from the above definition, the string v will necessarily be the longest common prefix of w_1 and w_2 , as explained in the following picture:



The characterisation of subsequential functions will say that they are exactly the functions which are: (a) continuous; and (b) have a property called bounded variation, which is defined in terms of the left distance. The continuity condition is the same as in Definition .0.1, except that we use inverse images for partial functions in the natural way. The bounded variation property is actually the same as uniform continuity with respect to the left distance, but we prefer to avoid the word continuity to avoid a name clash with condition (a).

Theorem B.4.8. *A partial function $f : A^* \rightarrow B^*$ is subsequential if and only if it is continuous and has the following property:*

- **Bounded variation:** *for every w_1 and w_2 in A^* , we have*

$$\sup_w \|f(w w_1), f(w w_2)\| < \infty,$$

where w ranges over strings such that both $f(w w_1)$ and $f(w w_2)$ are defined.

Proof. The left-to-right direction is easy to see, and we only explain the proof of the right-to-left direction. Assume that f is a continuous function with bounded variation. We will show that it can be computed by a subsequential transducer.

We begin with a simple pumping argument which shows that if a string can be extended to a string in the domain of f , then a short extension can be used.

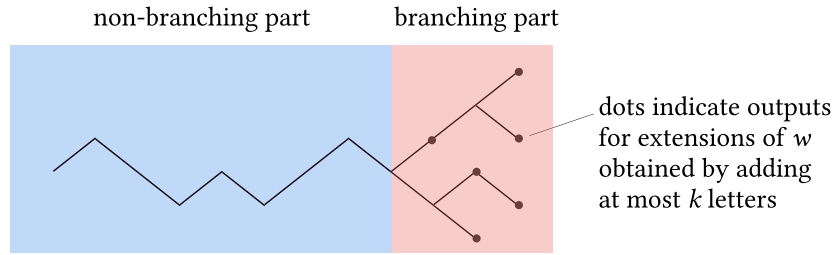
Claim B.4.9. *There is some $k \in \mathbb{N}$ such that for every $w \in A^*$, if there is some $v \in A^*$ such that wv is in the domain of f , then one can choose v so that it has length at most k .*

Proof. The domain is a regular language by continuity, and hence a pumping argument can be applied to make v short. $\square$

Fix the constant k from the above claim for the rest of this proof. Consider a string $w \in A^*$. We will be interested in the outputs of f on input strings that are obtained from w by extending it with at most k letters. (By definition of k , such short extensions are enough to enter the domain of the function, if this is possible at all.) Since these extensions are at bounded left distance from each other, it follows from the bounded variation property that the outputs will also be at bounded left distance from each other. Summing up, there is some $K \in \mathbb{N}$ such that for every $w \in A^*$, all strings in the set

$$\{ f(wv) \mid v \in A^* \text{ has length at most } k \} \quad (5)$$

are at left distance at most K from each other. This means that the set of strings in (5) can be visualised as a tree (where ancestors are prefixes), in which there is an initial non-branching part (of unbounded length), followed by some branching part of length at most K , as shown in the following picture:



Define $\alpha(w)$ to be the non-branching part for the input string w , which is formally speaking the longest common prefix of all strings in the set (5). The non-branching part might be undefined, but thanks to Claim B.4.9, it is defined if and only if w has some extension that is in the domain of f . In particular, if we think of the branching part as a string-to-string function

$$\alpha : A^* \rightarrow B^*,$$

then it is a partial function with a prefix-closed domain. Ultimately, we will establish that this function is subsequential.

Let us now discuss the branching part, which we denote by $\beta(w)$. Formally speaking, the branching part is the partial function

$$A^{\leq k} \xrightarrow{\beta(w)} B^*$$

$$v \mapsto f(wv) \text{ with the prefix } \alpha(w) \text{ removed.}$$

From the branching and non-branching parts, we can recover the output of f :

$$f(w) = \alpha(w) \cdot \underbrace{\beta(w)(\varepsilon)}_{\substack{\text{branching part evaluated} \\ \text{at the empty string}}} \quad (6)$$

The above is undefined if either of the two concatenated strings on the right-hand side of the equality is undefined, and it is defined otherwise. We will show that the branching part $\beta(w)$ has finitely many possible values and the right value can be selected by a finite automaton. Therefore, the function f will be sequential, because the contribution of the non-branching part $\alpha(w)$ is subsequential, and the contribution of the branching part $\beta(w)$ can be computed by using the end-of-input feature.

Computing the branching part. We begin by discussing the branching part, and only later we will discuss the non-branching part. The following claim shows that the contribution of the branching part can be computed using the end-of-input feature in a subsequential transducer.

Claim B.4.10. *The set of possible values of the branching part*

$$\{ \beta(w) \mid w \in A^* \}$$

is finite. Furthermore, for every β , the set of $w \in A^$ such that $\beta(w) = \beta$ is regular.*

Proof. The first part of the claim, about finitely many possible values, is a direct consequence of the fact that the branching part has length at most K , and thus there are finitely many possibilities for the corresponding tree. We now show the second part of the claim, about regularity. For every $v \in A^*$ of length at most k and every $u \in B^*$ of length at most K , the language

$$\{ w \in A^* \mid u \text{ is a suffix of } f(wv) \}$$

is regular by continuity. Therefore, a finite automaton which reads w can identify the last K letters of $f(wv)$ for all v of length at most k . This almost gives us the branching part $\beta(w)$, except that we do not know how the strings $f(wv)$ align with each other. To do this, we do a similar modulo-counting trick as in the proof of Theorem B.4.6: we determine the remainders

$$|f(wv)| \mod K + 1$$

for all v of length at most k . This gives us the relative alignment of the outputs. Indeed, if v_1 and v_2 have length at most k then the difference

$$|f(wv_1)| - |f(wv_2)|$$

is at most K , and therefore this difference is uniquely determined by its value modulo $K + 1$, which is known to the automaton. With these differences, we know how the strings $f(wv)$ align with each other, and hence we can determine the branching part $\beta(w)$. $\square$

Computing the non-branching part. We now show that the non-branching part $\alpha(w)$ can be computed by a subsequential transducer. For a nonempty input string $wa \in A^*$, let us define the offset as the difference

$$\delta(wa) \stackrel{\text{def}}{=} (\alpha(w))^{-1} \cdot \alpha(wa).$$

There is an important caveat in the definition of the offset: we do not necessarily know if $\alpha(w)$ is a prefix of $\alpha(wa)$. Therefore, the computation of the prefix might result in some negative letters, as in the following example:

$$\underbrace{(aa)^{-1}}_{\delta(wa)} = \underbrace{(abbaa)^{-1}}_{\alpha(w)} \cdot \underbrace{abb}_{\alpha(wa)}.$$

Formally speaking, the offset belongs to the free group that was defined in the proof of Lemma B.4.5. Using the offsets in the free group, we can compute the non-branching part as

$$\alpha(a_1 \cdots a_n) = \alpha(\varepsilon) \cdot \delta(a_1) \cdot \delta(a_1 a_2) \cdots \delta(a_1 \cdots a_n). \quad (7)$$

Formally speaking, the above equality is in the free group, i.e. we must first reduce the right-hand side before we can compare it to the left-hand side. We will worry about the reductions later, in Claim B.4.12. For now, let us show that the offsets can be computed by a finite automaton.

Claim B.4.11. *The set of possible values of the offset*

$$\{ \delta(w) \mid w \in A^+ \}$$

is finite. Furthermore, for every δ , the set of $w \in A^+$ such that $\delta(w) = \delta$ is regular.

Proof. Consider a nonempty string of the form wa . To get its offset, we need to compare the branching parts $\beta(w)$ and $\beta(wa)$, and how they are aligned. All this information can be computed by a finite automaton, using the same proof as in Claim B.4.10. $\square$

As explained in (7), the non-branching part $\alpha(w)$ is obtained by concatenating the offsets and then reducing them in the free group. We know that the ultimate result, after reduction, does not use any negative letters, since it is equal to the non-branching part $\alpha(w)$. The following claim shows that the negative letters can be eliminated from the transducer.

Claim B.4.12. *Let g be a partial sequential function whose output alphabet is*

$$\{ b, b^{-1} \mid b \in B \},$$

and whose domain is prefix closed. Consider the function

$$g'(w) = \text{reduced form of } g(w).$$

If all outputs of g' are in B^ , then it is a partial sequential function.*

Proof. The assumption that the domain is prefix-closed is true in our intended application, but it is not essential, since the argument can be easily adapted to work without this assumption. Because of the negative letters, it might be the case that extending the input results in a shorter output, i.e.

$$\{ |g'(wv)| - |g'(w)| \mid w, v \in A^* \} \quad (8)$$

might contain negative numbers. The crucial observation is that the above set can only contain finitely many negative numbers. Indeed, otherwise the sequential transducer would have a loop with output of negative length, and iterating this loop would violate the assumption of the claim that all reduced outputs are positive. Therefore, there is some constant M such that for every input string w , only the last M letters of $g'(w)$ can ever be affected by cancellations in extension of the input. This means that g' can be computed by a sequential transducer, which outputs $g'(w)$ without the last M letters, and keeps the remaining M letters in its state. $\square$

Putting everything together. We now put the above ingredients together to obtain the desired conclusion that f is subsequential. Thanks to Claim B.4.11 and (7), we know that there is a subsequential function with outputs in the free group that computes the non-branching part $\alpha(w)$. Thanks to Claim B.4.12, we can improve this function so that it does not use negative letters at all, i.e. the function $\alpha(w)$ itself is subsequential. Thanks to Claim B.4.10, there is a finite automaton whose state can be used to indicate the value of the branching part. By taking the product of this automaton with the subsequential function for $\alpha(w)$, we can get a subsequential transducer for f , thanks to the equality (6). $\square$

B.4.4 Rational functions

We finish this chapter with a machine independent characterisation of the rational functions. This characterisation is also based on bounded differences in the left distance.

Theorem B.4.13. *A function $f : A^* \rightarrow B^*$ is rational if and only if it is continuous, and the following equivalence relation on input strings has finite index:*

$$w_1 \sim w_2 \stackrel{\text{def}}{=} \sup_{w \in A^*} \|f(w w_1), f(w w_2)\| < \infty.$$

Proof. Since we have chosen to define rational functions as total functions, we assume that the function f is total. Nevertheless, the same proof would work for partial rational functions, since all the work related to undefined outputs is already done in the characterisation of subsequential functions.

The easy implication is from left-to-right. We have already established continuity for rational functions in Theorem B.1.5. To see that $\sim$ has finite index for a rational function, one can use bimachines: if w_1 and w_2 give the same state of the suffix automaton, then they will be equivalent.

The rest of the proof is devoted to the right-to-left direction. Assume then that f is continuous and $\sim$ has finitely many equivalence classes. We first observe that the relation $\sim$ in the statement of the theorem is a *left congruence*, which means that

$$w \sim w' \implies aw \sim aw' \quad \text{for every } w, w' \in A^* \text{ and } a \in A.$$

This congruence property holds because when we prepend a , the range of the supremum in the definition of $\sim$ becomes smaller. Let Q be the finite set of equivalence classes of $\sim$. Thanks to the congruence property, there is an automaton structure on equivalence classes, which goes from right-to-left: if $q \in Q$ is an equivalence class and a is an input letter, then there is a well-defined equivalence class $aq \in Q$, which is obtained by taking any word in q , prepending a , and taking the equivalence class.

To finish the proof, we will show that the function f can be computed in two steps, both of which are rational functions, and is hence rational as a composition of rational functions. In principle, the second step is a partial function, but the outputs of the first step will always fall in the domain of the second step.

1. In the first step, we annotate each input letter with the equivalence class under $\sim$ of the remaining suffix. If the input is $a_1 \cdots a_n$, then the output of the first step is the string

$$\xleftarrow{a_1} q_1 \xleftarrow{a_2} q_2 \cdots \xleftarrow{a_n} q_n \in (A \times Q)^*,$$

where q_i is the equivalence class of the suffix $a_{i+1} \cdots a_n$. This function is rational, since it can be computed by a finite automaton which runs on the input string from right to left, and outputs the annotation as it goes.

2. In the second step, we use the string from the first step to compute the output of the original function f . Formally speaking, the second step is a partial function

$$g : (A \times Q)^* \rightarrow B^*,$$

which is defined if and only if the input is consistent (i.e. it is a possible output of the first step), and in this case it returns the output of f on the original input string. We now use Theorem B.4.8 to show that g is subsequential. Let us check that the conditions in the theorem are satisfied. Continuity of g is an easy consequence of continuity of f . For the bounded variation property, we need to show that for all strings w_1 and w_2 over the input alphabet of g ,

$$\sup_w \|g(w w_1), g(w w_2)\| < \infty,$$

where w ranges over strings that make both outputs defined. Let v_1 and v_2 be the strings obtained from w_1 and w_2 by erasing the annotation with states Q . If w_1 and w_2 are inconsistent, or if they are consistent but the strings v_1 and v_2 are not equivalent under $\sim$, then the inequality above holds vacuously, since no string w can be found which makes both outputs defined. Otherwise, by definition of the function g , the supremum above is the same as

$$\sup_{v \in A^*} \|f(vv_1), f(vv_2)\|,$$

and is therefore bounded by the definition of $\sim$.

□

Example 17. [String reversal] Using the characterisation from the above theorem, we can show that the string reversal function is not rational. (We assume that the alphabet has at least two letters, since otherwise string reversal is the identity, and hence rational.) For string reversal, not only does the relation $\sim$ from the theorem have finite index, but in fact all strings are pairwise non-equivalent. Indeed, consider two different strings w_1 and w_2 . We want to show that

$$\sup_{w \in A^*} \|\text{reverse}(ww_1), \text{reverse}(ww_2)\| = \infty.$$

We can easily choose the string w so that the reversals differ in one of the first $|w_1| + 1$ letters, which is a constant when w_1 and w_2 are fixed. Since the string w can be made arbitrarily long, this will give unbounded distance. □

Part C: Regular functions

In this part, we move to the third step in the transducer ladder, namely the regular functions. Probably the quickest way to define the regular functions is in terms of prime functions, so we begin with that.

Example 18. [Map reverse and map duplicate] The map reverse function is obtained by applying the map lifting operation from Definition A.2.3 to string reversal. Here is an example of its application:

$$123\#45\#6789 \mapsto 321\#54\#9876.$$

Similarly, we define the map duplicate function, by lifting string duplication $w \mapsto ww$. Here is an example of its application:

$$123\#45\#6789 \mapsto 123123\#4545\#67896789.$$

Formally speaking these are not two functions, but a family of functions, one for each alphabet (the input and output alphabets are the same in this case). Using Theorem B.4.13, one can show that none of these functions is rational. $\square$

In Example 17 we have shown that string reversal is not rational. It follows that map reverse is not rational either, since it can be used to implement string reversal by choosing an input string without separators. A similar argument can be made to show that neither duplicate nor map duplicate are rational. Therefore, we get a new step in the transducer ladder by adding these two functions, as we do in the following definition.

Definition C.0.14 (Regular functions). *A string-to-string function is called regular if it can be obtained as a finite composition of functions, each of which is either:*

1. *a rational function;*
2. *one of the map reverse or map duplicate functions from Example 18.*

One could, of course, add just one of the two functions, or maybe even just duplicate and reverse without map. As we will see in this part, there is a good reason to consider the extension from the above definition. This is because the resulting class has an abundance of equivalent models, including two-way automata with output, streaming string transducers, a transducer variant of monadic second-order logic, and a functional programming language based on combinators. These models will be introduced in the next few chapters.

C.1 The prime regular functions

So far, we have defined the regular functions in terms of compositions of certain prime functions, namely the rational functions, map reverse and map duplicate. Before introducing new models for this class, in this chapter we establish the basic properties of the regular functions as defined.

Continuity and composition. Almost immediately from the definition, we recover two of our favourite properties for transducer classes.

Theorem C.1.1. *Regular functions are continuous and closed under composition.*

Proof. Composition is built into the definition, so we only argue about continuity. Since continuous functions are closed under composition, and rational functions are continuous by Theorem B.1.5, it remains to show that map reverse and map duplicate are continuous. We first show that reverse and duplicate (without map) are continuous, and then we extend this with the map combinator.

Lemma C.1.2. *String reversal and string duplication are continuous.*

Proof. This lemma is a classical exercise in automata theory³. We need to show that for every regular language L , the inverse images

$$\{ w \in A^* \mid \text{reverse}(w) \in L \} \quad \{ w \in A^* \mid ww \in L \}$$

are regular. We assume that L is given by a nondeterministic automaton. For string reversal, we modify the original automaton by reversing all transitions and swapping initial and accepting states. For string duplication, the inverse image is

$$\bigcup_q \{ w \in A^* \mid \begin{array}{l} \text{there is a run from an initial state to } q \text{ and} \\ \text{there is a run from } q \text{ to an accepting state} \end{array} \},$$

which is easily seen to be a regular language. □

It remains to lift the above lemma via map. We will show that the map lifting of every continuous function, not just reverse and duplicate, is continuous.

Lemma C.1.3. *If a string-to-string function is continuous, then the same is true for its map lifting.*

Proof. Consider the map lifting of some continuous function f . To prove the lemma, we need to show how an automaton $\mathcal{B}$ over the output alphabet of the map lifting is pulled back to an automaton $\mathcal{A}$ over the input alphabet. Given an input string,

$$w_1 \# \cdots \# w_n$$

the automaton $\mathcal{A}$ nondeterministically guesses states $p_1, q_1, \dots, p_n, q_n$ of the automaton $\mathcal{B}$ and checks that the following conditions are all satisfied:

³[MNR23, Problems 30 and 35]

1. the state p_1 is initial and the state q_n is accepting;
2. for every $i \leq n$, the automaton $\mathcal{B}$ can go from p_i to q_i when reading $f(w_i)$;
3. for every $i < n$, the automaton $\mathcal{B}$ can go from q_i to p_{i+1} when reading $\#$.

The first and third conditions can easily be checked by an automaton. The second condition can also be checked by an automaton thanks to continuity of f , since it corresponds to an inverse image of a regular language. $\square$

$\square$

Equivalence. We can also show that equivalence is decidable for regular functions.

Theorem C.1.4. *Equivalence is decidable for regular functions.*

Proof. We use the same approach as in the equivalence algorithm for rational functions in Theorem B.3.4, which is a reduction to equivalence for weighted automata. As in the case of rational functions, it is enough to show that if we post-compose a regular function f with a weighted automaton g over the field of rationals

$$A^* \xrightarrow{f} B^* \xrightarrow{g} \mathbb{Q},$$

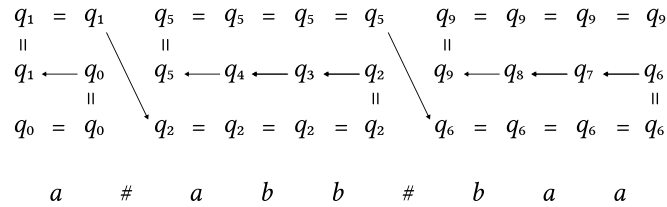
then the resulting function is also a weighted automaton, and can be computed. By choosing the weighted automaton g to be injective, we can faithfully represent the regular function f as a weighted automaton, thus reducing the equivalence problem for regular functions to the equivalence problem for weighted automata over a field.

Consider the class of functions which can be post-composed with weighted automata over a field, i.e.

$$\{ A^* \xrightarrow{f} B^* \mid \text{if } B^* \xrightarrow{g} \mathbb{Q} \text{ is a weighted automaton, then so is } f \cdot g. \}$$

We need to show that this class contains all regular functions. Since this class is easily seen to be closed under composition, and it contains all rational functions by Theorem B.3.6, it is enough to show that it contains map reverse and map duplicate.

- **Map reverse.** On each block between two separator symbols, the automaton for the post-composition $f \cdot g$ runs in reverse, and remembers the states at the beginning and end of the block. This is implemented using triples of states, as in the following picture:



(As we will see in the next chapter, this construction corresponds to a two-way automaton that computes the map reverse function.) Assuming that the original weighted automaton g had states Q , the new weighted automaton (for the post-composition $f \cdot g$) has states Q^3 . We assume that the original automaton consumes at most one input letter in each transition, but ε -transitions are still allowed. The transitions are defined by

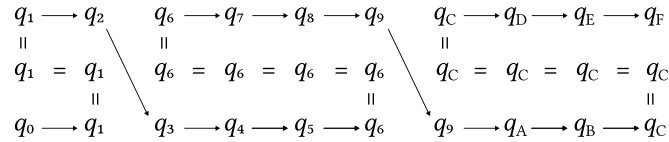
$$\underbrace{(x, q, y) \xrightarrow{a/x} (x, p, y)}_{\text{in the new automaton}} \iff \underbrace{p \xrightarrow{a/x} q}_{\text{in the original automaton}}.$$

for a which is not the separator symbol $\#$, and by

$$\underbrace{(p, y, y) \xrightarrow{\#/x} (x, x, q)}_{\text{in the new automaton}} \iff \underbrace{p \xrightarrow{a/x} q}_{\text{in the original automaton}}.$$

The initial states are those which have an initial state on the first coordinate, and the accepting states are those which have an accepting state on the third coordinate. Note that this weighted automaton will multiply the weights of the original run in a different order, but this is not an issue since multiplication in a field is commutative. More generally, this construction would work for any commutative semiring. For non-commutative semirings, the construction cannot work, since we have established in Theorem B.3.6 that only rational functions can be closed under post-composition with weighted automata over arbitrary semirings, and map reverse is not rational.

- **Map duplicate.** The construction is similar to the one for map reverse, and illustrated in the following picture:



The details of the construction are left to the reader. Note that for a transition in the new automaton, its weight will be obtained by multiplying the weights of two transitions in the original automaton.

□

In the above proof, we showed that regular functions can be post-composed with weighted automata over the field of rationals, and the proof also extends to any commutative semiring. We conjecture that the converse also holds, thus giving a characterisation in the same spirit as Theorem B.3.6.

Conjecture C.1.5. *A string-to-string function $f : A^* \rightarrow B^*$ is regular if and only if it is closed under post-composition with weighted automata over all commutative semirings $\mathbb{S}$, i.e.*

$$B^* \xrightarrow{g} \mathbb{S} \text{ is a weighted automaton} \quad \implies \quad A^* \xrightarrow{f \cdot g} \mathbb{S} \text{ is a weighted automaton.}$$

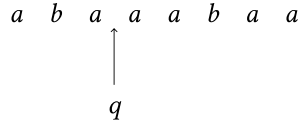
Note that continuity, in the sense of Definition .0.1, is a special case of the post-composition closure in the above conjecture, when one considers weighted automata over the Boolean semiring. It is also possible that the conjecture holds already for a fixed semiring, such as the field of rationals.

Exercises

Exercise C.1.1. In Definition C.0.14, we use `map reverse` and `map duplicate` over arbitrary alphabets of the form $A + 1$. Show that the definition remains the same if we only consider A with two letters.

C.2 Two-way transducers

In this chapter, we define a transducer model for the regular functions, which is based on two-way automata, in which the head of the automaton can move in both directions. A configuration of the transducer consists of an input string, a state, and a head position, as in the following picture:



Note that the head points to a gap between positions, instead of a position. Usually, one defines two-way transducers (or Turing machines, which are a more powerful cousin) so that the head points to a position instead of a gap, but we choose the gap view so that it is more consistent with one-way automata. A configuration can be formally seen as an element of

$$\underbrace{A^*}_{\substack{\text{input} \\ \text{before} \\ \text{the head}}} \times \underbrace{Q}_{\text{state}} \times \underbrace{A^*}_{\substack{\text{input} \\ \text{after} \\ \text{the head}}}.$$

The transducer starts in an initial configuration, where the state is the chosen initial state, and the head is to the left of the entire input string. Next, in each step, the transducer looks at: (a) the letter to the left of the head, (b) the current state, and (c) the letter to the right of the head. Based on this information, it can either produce an output string and halt, or it can produce an output string, change its state, and move the head left or right. Note that the letters in (a) and (c) might be undefined, since the head can be at the leftmost or rightmost position, and hence they can be seen as elements of $A + 1$, where 1 is a special symbol that stands for the undefined letter. The formal definition is given below.

Definition C.2.1. *A two-way transducer consists of the following data:*

- *a finite input alphabet A ;*
- *a finite output alphabet B ;*
- *a finite set of states Q , with an initial state $q_0 \in Q$;*
- *a transition function*

$$\underbrace{(A+1)}_{\substack{\text{letter} \\ \text{left of} \\ \text{head}}} \times \underbrace{Q}_{\text{state}} \times \underbrace{(A+1)}_{\substack{\text{letter} \\ \text{right of} \\ \text{head}}} \xrightarrow{\delta} \underbrace{B^*}_{\substack{\text{produce an} \\ \text{output string} \\ \text{and halt}}} \overset{\text{or}}{+} \underbrace{(Q \times B^* \times \{-1, 1\})}_{\substack{\text{if the transducer does not halt,} \\ \text{then it chooses a new state,} \\ \text{produces some output, and} \\ \text{moves the head left or right.}}.$$

We now define the semantics of the transducer in more detail. The *configuration graph* is a directed edge labelled graph where vertices are

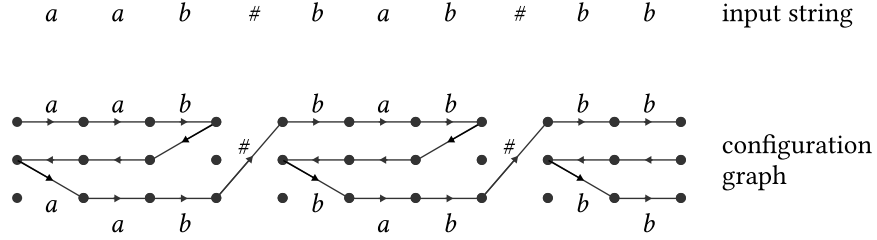
$$V = \underbrace{(A^* \times Q \times A^*)}_{\text{configurations}} + \underbrace{1}_{\text{halt}}$$

and the edges represent one step of computation, with

$$c \xrightarrow{w} d$$

saying that the transition applied in configuration c leads to d (which is defined in the natural way) and produces output w . By determinism, every vertex has at most one outgoing edge. (There might be no outgoing edge in case the transducer chooses to fall off the input, by moving left in the leftmost position or right in the rightmost position.) The output for an input string $w \in A^*$ is obtained by taking the corresponding initial configuration (ε, q_0, w) , and reading the labels on the path from this configuration to the halting vertex. In particular, we require that this path exists, i.e. the transducer does not loop or fall off the tape.

Example 19. Consider the map duplicate function from Example 18. This function is computed by a two-way transducer that has three states, and whose configuration graph (restricted to the input string) is shown in the following picture:



(The reader might recognise this picture from the proof of Theorem C.1.4.) A very similar transducer works for the map reverse function, except that the output is produced during the movement in the left direction. $\square$

C.2.1 Continuity

This section shows that two-way transducers are continuous, i.e. inverse images of regular languages are regular. Ultimately, we will show that two-way transducers are the same as the regular functions, and hence they must be continuous by Theorem C.1.1. However, the direct proof of continuity is worth the effort, since it will introduce some useful techniques.

Theorem C.2.2. *Every function computed by a two-way transducer is continuous.*

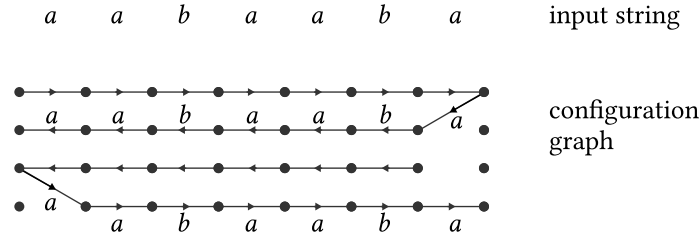
Proof. We want to show that if f is a two-way transducer and L is a regular language, then their composition (which corresponds to an inverse image)

$$A^* \xrightarrow{f} B^* \xrightarrow{L} \{\text{yes, no}\} \quad (9)$$

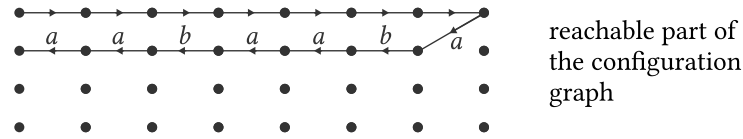
is a regular language. If we take a DFA that recognises the language L , then we can apply a straightforward product construction, yielding a two-way automaton (not transducer) which recognises the inverse image. Therefore, this theorem boils down to showing that two-way automata can only recognise regular languages⁴.

Later on in this section, we will want to prove that two-way transducers are closed under composition, which is the extension of (9) in which the language L is replaced by another two-way transducer. We try to phrase the present proof in a way which is amenable to that extension. The proof has two steps: (a) we first use a rational function to compute the configuration graph of f on the input string $w \in A^*$; and then (b) we use the configuration graph to check membership of $f(w) \in L$.

Computing the configuration graph. For an input string, we will be interested in its configuration graph, which is the configuration graph of the transducer restricted to configurations that come from the input string. We have seen a picture of such a configuration graph in Example 19. However, that picture was a bit misleading, since it only contained reachable configurations (except for isolated vertices). Here is a more illuminating picture, which shows a transducer that reverses the input string if the last letter is a , and otherwise keeps it unchanged:



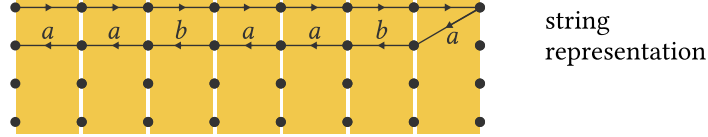
A local inspection of the above configuration graph will not tell us which configurations are reachable. For this reason, we will be interested in the reachable part of the configuration graph, as in the following picture:



⁴Turning a two-way automaton into a one-way automaton was one of the original motivations for introducing nondeterministic automata by Rabin and Scott in [RS59]. The same problem was solved independently by Shepherdson in [She59], who used a direct construction without nondeterminism. In Note 4 of Shepherdson's paper one can also see a mention of the transducer variant, and thus this is probably the first appearance of two-way transducers in the literature.

The reachable part is a single path, which goes from the initial configuration to a halting configuration.

We will show that the reachable part of the configuration graph can be computed by a rational function. In order to do this, we need to represent it as a string over a finite alphabet. The idea is to slice the graph into smaller graphs, one for each input letter, as in the following picture:



Formally speaking, the alphabet for the string representation, call it C , consists of bipartite graphs, where the vertices are two copies of Q , edges are directed and labelled with output strings, each vertex has at most one outgoing edge, and this edge must go to the other copy of Q . This alphabet is finite, since we use only labels of edges that arise from transitions of the transducer. There is an edge case for the empty input string, in which the configuration graph looks like this:



Therefore, the alphabet C contains a special letter for this configuration graph.

Lemma C.2.3. *The function which maps an input string to the string representation of its reachable configuration graph is rational.*

Proof. The main observation is that configuration graphs form a regular language, i.e.

$$\{ w \in C^* \mid w \text{ is a string representation of a reachable configuration graph} \} \quad (10)$$

is a regular language. To see this, we need to check that the letters are consistent with the transition function of the two-way transducer (which is a simple local consistency check) and that all arrows are reachable. For the latter property, about reachability of arrows, it is easier to consider the complement (since regular languages are closed under complementation). The complement can be checked by guessing a subset of arrows that is disconnected from the others.

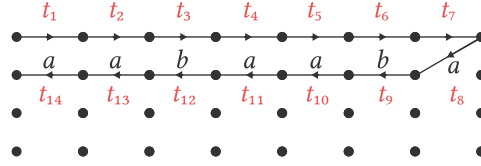
Once we know that the language (10) is regular, the lemma follows. The function in the lemma is computed by an NFA with output, which nondeterministically guesses the output, and then checks if it is correct using the language (10). There is a unique correct guess, and hence the function is rational. $\square$

The following lemma shows that once we have computed the reachable configuration graph, we can check if the output string in this graph belongs to L .

Lemma C.2.4. *The following language is regular:*

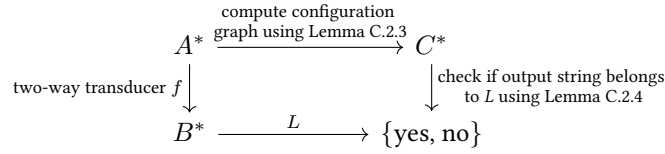
$$\{ w \in C^* \mid \begin{array}{l} w \text{ is a string representation of a reachable configuration graph} \\ \text{and the corresponding output string belongs to } L \end{array} \}$$

Proof. The automaton checks that the input is indeed a reachable configuration graph (which can be done as shown in the previous lemma), and then it guesses a labelling of the edges in the graph by transitions of an automaton for L , as in the following picture (with transitions in red):



Once the transitions have been guessed, the automaton checks if they represent an accepting run, which is a purely local property. $\square$

By putting the above two lemmas together, we complete the proof, as explained in the following diagram:



The right-down part describes a regular language by continuity of rational functions, and hence the down-right path also describes a regular language, thus proving continuity of the two-way transducer f . $\square$

C.2.2 Closure under composition

We now prove that two-way transducers are closed under composition. This is a non-trivial result, because we cannot simply use a product construction, since the heads of the two transducers that are composed can move in different directions.

Theorem C.2.5. *Functions computed by two-way transducers are closed under composition, i.e.*

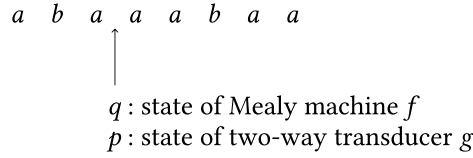
$$\text{2way} \cdot \text{2way} \subseteq \text{2way}.$$

Proof. We will prove the theorem by gradually advancing up the transducer ladder. We will first show that two-way transducers are closed under pre-composition with Mealy machines, then we will extend this to pre-composition with rational functions, and finally we will show pre-composition with two-way transducers, as required by the theorem.

Lemma C.2.6. *Functions computed by two-way transducers are closed under precomposition with Mealy machines, i.e.*

$$\text{Mealy} \cdot \text{2way} \subseteq \text{2way}.$$

Proof. A natural idea would be to use a product construction, in which the two-way transducer for the composition $f \cdot g$ keeps a pair of states as in the following picture:



If the two-way transducer wants to move to the right, then we can easily update both states, by using the transition functions in the two automata. However, if the two-way transducer wants to move to the left, then we have a problem: we do not know how to update the state of the Mealy machine, since there might be several candidates which go to state q by reading the letter a . One way to solve this problem was proposed by Hopcroft and Ullman⁵, using a clever construction in which a transducer retraces its steps. However, we will not need this construction, since we can leverage the Krohn-Rhodes Theorem, which says that every Mealy machine can be decomposed into prime Mealy machines. In light of this theorem, it is enough to show pre-composition with primes, i.e.

$$\begin{aligned}
 (\text{Reversible Mealy}) \cdot \text{2way} &\subseteq \text{2way} \\
 (\text{Flip-flop Mealy}) \cdot \text{2way} &\subseteq \text{2way}.
 \end{aligned}$$

For the reversible Mealy machines, we can simply use the product construction, since the Mealy machine is deterministic in both directions. It remains to deal with the case of flip-flops. For flip-flops, we adapt the product construction as follows. If the two-way transducer wants to move right, the pair of states can be easily updated. If it wants to move to the left, there are two cases: (i) the letter to the left of the head does not change the state of the Mealy machine; or (ii) the letter to the left of the head resets the state of the Mealy machine to some fixed state. In case (i), there is no problem. Finally, in case (ii), we can execute a subroutine which keeps on moving to the left until it reaches another resetting letter, stores the corresponding state, and then returns to the right to the original resetting letter. $\square$

Corollary C.2.7. *Functions computed by two-way transducers are closed under precomposition with rational functions, i.e.*

$$\text{Rational} \cdot \text{2way} \subseteq \text{2way}.$$

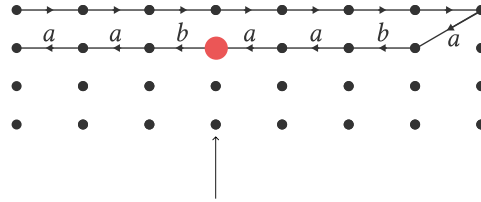
⁵This construction was introduced in [HU67, Lemma 3]. A description of the same construction can also be found in the lecture notes [Boj25, Lemma 17.4].

Proof. As explained in the proof of Theorem B.2.6, every rational function is a composition of: (a) Mealy machines, (b) a right-to-left variant of Mealy machines, (c) string homomorphisms, and (d) appending a separator. Pre-composition under (a) was shown in the previous lemma, and pre-composition under (b) can be shown by a symmetric argument. Pre-composition under (c) and (d) are easy constructions that are left to the reader. $\square$

We are now ready to finish the proof of the theorem, by showing that the composition

$$A^* \xrightarrow{f} B^* \xrightarrow{g} C^*$$

can also be computed by a two-way transducer. By Lemma C.2.3, we can compute the configuration graph of f using a rational function. Since two-way transducers are closed under pre-composition with rational functions by Corollary C.2.7, we can compute the configuration graph of f using a two-way transducer. Finally, by Lemma C.2.4, it is enough to show that the output $g(f(w))$ can be computed by a two-way transducer which looks at the configuration graph of f on w . Once the configuration graph is present, we can do this easily using a product construction, which is explained in the following picture:



product automaton remembers:

- state of f (red circle); and
- state of g

$\square$

A corollary of the above theorem is that two-way transducers are contained in the regular functions from Definition C.0.14.

Corollary C.2.8. *If a function is computed by a two-way transducer, then it is regular.*

Proof. Two-way transducers can compute all rational functions by Corollary C.2.7, and they can compute map reverse and map duplicate by Example 19. Finally, they are closed under composition thanks to Theorem C.2.5. $\square$

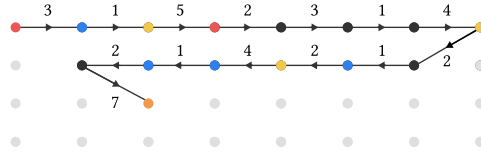
Later in this chapter we will show the converse inclusion, which is that two-way transducers can be decomposed into the prime rational functions (map reverse, map duplicate, and rational functions).

C.2.3 Decidability of equivalence

In the previous chapter, we have shown that equivalence is decidable for regular functions, as defined by compositions of certain prime functions. We will show that two-way transducers are the same as regular functions, and hence the equivalence algorithm from the previous chapter will apply. Nevertheless, it is worth pointing out that the proof from the previous chapter can be adapted to give an equivalence algorithm that works directly on two-way transducers, without going through the prime decomposition. To see this, consider a composition

$$A^* \xrightarrow{f} B^* \xrightarrow{g} \mathbb{Q}$$

of a two-way transducer followed by a weighted automaton. The composed function can be computed directly by a weighted automaton, in which each run describes a labelling of the configuration graph of f by transitions of g , as in the following picture, in which states of the weighted automaton are represented using colours:



The weight of a transition in the new automaton, which corresponds to a column in the above picture, is the product of transition weights of the original weighted automaton g along the column. For example, the second column in the above picture has weight $14 = 1 \times 2 \times 7$. It is helpful in the construction to assume that the weighted automaton g does not use ε -transitions. Such transitions can be eliminated using the same technique as in Lemma B.2.4. The details are left to the reader.

C.2.4 Decomposition into prime functions

A consequence of the closure under composition for two-way transducers was that all regular functions are computed by two-way transducers. In this section, we show the converse, thus establishing:

Theorem C.2.9. *Two-way transducers compute exactly the regular functions.*

Proof. The only part that we need to show is the left-to-right inclusion, i.e. that every two-way transducer is regular, i.e. it can be decomposed into prime functions. We begin by collecting some closure properties of the regular functions.

Lemma C.2.10. *If $f, g : A^* \rightarrow B^*$ are regular functions, then so are:*

- *their map liftings; and*
- *their concatenation $w \mapsto f(w) \cdot g(w)$; and*

- the following conditional function

$$w \in A^* \mapsto \begin{cases} f(w) & \text{if } w \in L \\ g(w) & \text{otherwise.} \end{cases} \quad \text{where } L \subseteq A^* \text{ is regular.}$$

Proof. Since map commutes with composition, as we have remarked in the proof of Lemma A.2.4, it is enough to show that the map liftings of the prime functions are regular. Rational functions are closed under map lifting, so we only need to check the map lifting of map reverse and map duplicate, i.e. we have a double map lifting. In the double map lifting, there are two kinds of separators, which – up to a rational rewriting – can be represented using nested brackets (with two levels of nesting):

$$[[123, 45, 6], [78, 9]] \xrightarrow{\text{map map rev}} [[321, 54, 6], [87, 9]]$$

Once we see the double map lifting this way, it is clear that it is a regular function, since it can be implemented using the usual map lifting and a rational rearrangement of brackets.

Before dealing with the remaining two items in the lemma, about concatenation and conditionals, we introduce an auxiliary construction.

Claim C.2.11. Consider two regular functions

$$f_1 : A_1^* \rightarrow B_1^* \quad f_2 : A_2^* \rightarrow B_2^*$$

with disjoint input and output alphabets. The following function $f_1 + f_2$ is also regular:

$$w \in (A_1 + A_2)^* \mapsto \begin{cases} f_1(w) & \text{if } w \text{ uses only letters from } A_1 \\ f_2(w) & \text{if } w \text{ uses only letters from } A_2 \\ \perp & \text{otherwise, where } \perp \text{ is a string using both } B_1 \text{ and } B_2. \end{cases}$$

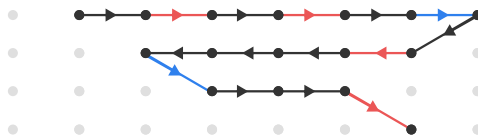
Proof. The construction in the claim is easily seen to be compatible with composition of functions. Therefore, it is enough to show it in the special case in which one of the functions is the identity function, and the other one is a prime function. When the prime function is rational, this is easy to see. When the prime function is map reverse or map duplicate, the argument is also straightforward and left to the reader. (If the input is in the part that should be unchanged, then we can insert a separator before every letter so that it can be recovered after map duplicate or map reverse.) $\square$

Using the above claim, we implement concatenation of regular functions as follows:

$$w \xrightarrow{\text{append } \#} w\# \xrightarrow{\text{map duplicate with } \# \text{ not being a separator}} w\#w\# \xrightarrow{\text{use disjoint alphabet for second copy and remove extra } \#} w\#\textcolor{red}{w} \xrightarrow{\text{map}(f+g) \text{ with } \# \text{ being a separator}} f(w)\#g(w).$$

Finally, a rational function can erase the separator and remove the colour information. A similar idea applies to the conditionals: we can first use a rational function to use an appropriate copy of the input alphabet (depending on membership in the condition language L), and then we can apply the function $f + g$. $\square$

In the proof, we will be using snake graphs, which are pictorial descriptions of a run of a two-way transducer. Here is a picture of a snake graph, in which the colours of the arrows represent output letters (black represents the empty output):



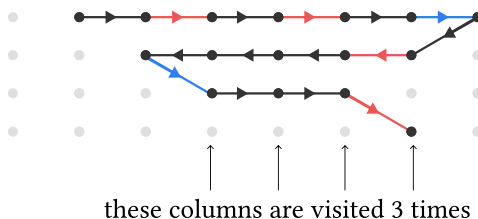
Formally speaking, a *snake graph* with states Q , length n and alphabet B is a directed graph with edges labelled by $B + 1$ that satisfies the following properties:

- each vertex consists of a row $q \in Q$ and a column $i \in \{0, 1, \dots, n\}$;
- all edges in the graph are on a single directed path;
- edges can only go between adjacent columns.

Similarly to configuration graphs in the proof of Theorem C.2.2, snake graphs can be represented as strings over a finite alphabet C , once the set of states is fixed. Define the *output* of a snake graph to be the concatenation of the labels on the edges. Using the output of snake graphs, every two-way transducer f can be decomposed into two stages, as in the following diagram:

$$A^* \xrightarrow{\text{compute snake graph}} C^* \xrightarrow{\text{output of snake graph}} B^*$$

Since the first function is rational (and therefore also regular) by the same argument as in Lemma C.2.3, it remains to prove that the second function is regular. The second function is partial, since the input might not represent a snake graph. To make it total, we assume that if a string in C^* does not represent a snake graph, then its output is empty. To prove that the output of snake graph can be computed by a regular functions, we use induction on a parameter of snake graphs that we call the width: this is the maximal number of times that the snake graph visits a single column. Here is a picture:

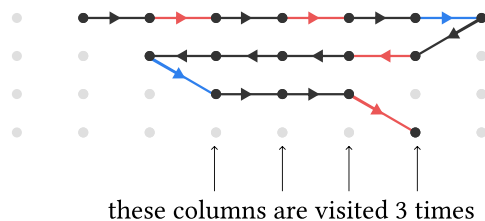


The maximal width of a snake graph is $|Q|$, and hence the following lemma will be sufficient to prove the theorem.

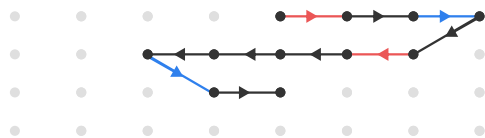
Lemma C.2.12. *Let C be the alphabet used to represent snakes with state Q and output alphabet B . For every $k \in \{1, \dots, |Q|\}$, the following function is regular:*

$$w \in C^* \mapsto \begin{cases} \varepsilon & \text{if } w \text{ does not represent a snake graph of width } \leq k \\ \text{otherwise, the output of the snake graph.} \end{cases}$$

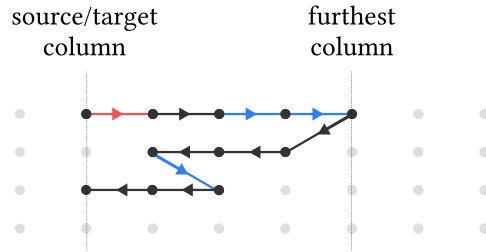
Proof. The function in the lemma is partial, since the input might not represent a snake graph. Therefore, we assume that if the input is not a snake graph, then the output is the empty string. The lemma will be proved by induction on a parameter of snake graphs that we call the width: this is the maximal number of times that the snake graph visits a single column, as explained in the following picture:



Looping snake. We begin by proving the induction step for a special case of a snake, in which the source and target are in the same column, as in the following picture:



A snake with this property is called a *looping snake*. If the column with the source and target is visited at least three times then we can decompose the snake into a concatenation of a constant number of snakes in which the source and target are the only vertices in their column. Each of the factors in this concatenation can be computed by a rational function, and the concatenation itself can be implemented by a regular function thanks to Lemma C.2.10. We are left with the case where the source and target vertices of the looping snake are the only vertices in their column. Among the columns visited by the snake, consider the one that is the furthest away from the source, as in this picture:



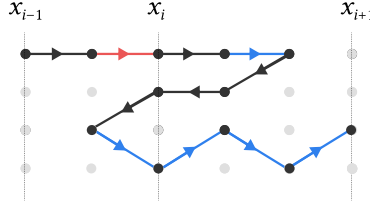
In order to reach this furthest column for the first time, the snake cannot use its full width, since it needs to return. Therefore, if we partition the snake into two parts: before and after the first visit to the furthest column, then both parts have smaller width than the original snake. This partition can be computed by a rational function, the outputs for the parts can be computed by induction assumption, and the results can be combined using the concatenation construction in Lemma C.2.10. This completes the proof of the induction step in the case of a looping snake.

General case. Having dealt with the special case of looping snakes, we now deal with the general case. We can assume without loss of generality that the source column is before the target column. Indeed, using a regular language we can check if the source column is after the target column, and if it is, then we can reverse the snake, which does not change the output. (Formally speaking, when reversing a snake, we need to change the direction of the arrows in the letters that represent the snake.) Also, we need to use here the cases construction from Lemma C.2.10 to check if the snake needs to be reversed.

From now on, we assume that the source column is before the target column. This means that the snake generally progresses in the left-to-right direction, but it might make loops along the way. In the rest of this proof, we will decompose a snake into parts which are looping and parts which are progressing. The looping parts are defined as follows: for a column x that is visited by the snake, define its *loop part* to be the part of the snake that starts in the first visit to x and ends in the last visit to x . The output of the loop part can be computed by a regular function, as we have explained above. To discuss the progressing part, we will be interested in special columns,

$$x_1 < \cdots < x_n,$$

which are called *record-breakers*. The first record-breaker x_1 is the source column. Assuming that the record-breaker x_i has been defined, we define a new record-breaker x_{i+1} as follows. Consider the rightmost column visited by the loop part of x_i , and then take the next column. (If the loop part of x_i is empty, then we simply take the next column after x_i .) If the next column is visited by the snake, then it becomes the new record-breaker x_{i+1} , otherwise the sequence of record-breakers ends at x_i . Here is a picture:



For each record-breaker x_i , define its *progress part* to be the part of the snake that starts in the last visit to x_i and ends in the first visit to the next record-breaker x_{i+1} (or the end of the snake if $i = n$). To compute the output of the progress part, we can use the induction assumption on smaller width, since every column visited by the progress part is also visited by the loop part, except for the last column which is visited only once. We now complete the proof of the induction step, which is done in several stages that are regular functions:

1. In the first stage, we first mark the record-breakers, i.e. we compute the string

$$w_0 \# w_1 \# \cdots \# w_n,$$

where w_i is the part of the input string between the record-breakers x_{i-1} and x_i . This stage can be implemented by a rational function, since a nondeterministic automaton with output can guess the record-breakers, and then check that they satisfy the conditions in the definition.

2. In the output of the first stage, each separator $\#$ represents a record-breaker. In the second stage, we compute for each separator the part of the input string that is around it, up to the nearest separator. This is represented by the following string

$$(w_0 \# w_1)(w_1 \# w_2) \cdots (w_{n-1} \# w_n).$$

In the above string, the parentheses are part of the output. As we will see later in the proof, the part $w_{i-1} \# w_i$ will contain both the loop and progress parts of the i -th record-breaker. This stage can be implemented by a regular function, which uses map duplicate to procure the two copies of each w_i , and then uses rational functions to arrange parentheses and remove the extra copies of w_0 and w_n .

3. For each block $(w_{i-1} \# w_i)$ from the previous stage, we will need two copies, one for the loop and one for the progress. For this reason, we duplicate each block from the previous stage, as represented by the string

$$((w_0 \# w_1)(w_0 \# w_1))((w_1 \# w_2)(w_1 \# w_2)) \cdots ((w_{n-1} \# w_n)(w_{n-1} \# w_n)).$$

4. We now show that the loop and progress parts of the i -th separator are contained in the block $w_{i-1} \# w_i$. This will finish the proof, since the outputs of these parts can be computed (by induction assumption for the progress part, and

by the previous argument for the loop part), and regular functions are closed under applying the map combinator thanks to Lemma C.2.10. The loop part of the i -th record-breaker is contained in $w_{i-1} \# w_i$ because after visiting this record-breaker, the previous one is never visited. For the progress part, the argument is similar: the progress part never returns to its source (since otherwise such a return would be part of the corresponding loop) and it stops immediately upon visiting its target, which is the next record-breaker.

This completes the proof of the induction step in the general case, and hence the proof of the lemma. $\square$

$\square$

C.3 Streaming string transducers

So far we have proved that the regular functions are exactly those which can be computed by two-way transducers. In this section, we describe a one-way model for the regular functions, which is called *streaming string transducers* (sst's). The general idea is that the transducer processes the input string in a single left-to-right pass, and uses registers to store intermediate results, which are strings over the output alphabet. Before giving a formal definition of the model, we give some examples to illustrate the main ideas behind it.

Example 20. [Reverse] Consider the reverse function

$$\{a, b\}^* \xrightarrow{\text{reverse}} \{a, b\}^*.$$

To compute this function, we use a streaming string transducer that has one register X and one state q . This register starts with the empty string, and input letters are prepended to its content, with transitions of the form

$$\underbrace{q \xrightarrow{a|X \mapsto aX} q}_\text{when reading input } a, \text{ the state } q \text{ is not changed} \\ \text{and the register } X \text{ is updated by prepending } a \quad q \xrightarrow{b|X \mapsto bX} q.$$

After reading the whole input, the content of the register X is the reverse of the input, and this is also the output of the transducer. $\square$

Example 21. [Map reverse] Consider the map reverse function

$$\{a, b, \#\}^* \xrightarrow{\text{map reverse}} \{a, b, \#\}^*.$$

For this function, we also use one state q , but this time we have two registers. One register X stores the reverse of the current block, and the other register Y stores the output produced for the previous blocks. The transitions are as follows:

$$\underbrace{q \xrightarrow{a| \begin{smallmatrix} X \\ Y \end{smallmatrix} \mapsto \begin{smallmatrix} aX \\ Y \end{smallmatrix}} q}_\text{upon reading input } a, \\ \text{letter } a \text{ is prepended to } X \\ \text{and } Y \text{ is left unchanged} \quad \underbrace{q \xrightarrow{b| \begin{smallmatrix} X \\ Y \end{smallmatrix} \mapsto \begin{smallmatrix} bX \\ Y \end{smallmatrix}} q}_\text{upon reading input } b, \\ \text{letter } b \text{ is prepended to } X \\ \text{and } Y \text{ is left unchanged} \quad \underbrace{q \xrightarrow{\#| \begin{smallmatrix} X \\ Y \end{smallmatrix} \mapsto \begin{smallmatrix} \varepsilon \\ YX\# \end{smallmatrix}} q}_\text{upon reading input } \#, \\ \text{register } X \text{ is cleared} \\ \text{and its content is appended to } Y$$

After reading the whole input, the output is obtained as the concatenation XY . $\square$

Example 22. [Iterated duplication] Using a register update of the form $X \mapsto XX$, we could implement a function with exponential output size. This will be forbidden in our model, by requiring that the register updates are nonduplicating, which means intuitively that each register from the old valuation can be used at most once in the new valuation. $\square$

Example 23. [Map duplicate] As we have explained in the previous example, we cannot copy registers. Nevertheless, there is some bounded duplication that is possible. For example, if we want to compute the map duplicate function, we can simply

have two registers X_1 and X_2 for the current block, which are updated independently. Then, the two registers are appended onto Y upon reading the update

$$\begin{array}{c} X_1 \\ X_2 \\ Y \end{array} \mapsto Y \overset{\varepsilon}{\#} \overset{\varepsilon}{X_1 X_2}.$$

□

We now give a formal definition of the model of streaming string transducers. As explained in the above examples, the transducer model is based on updating registers that store strings over the output alphabet. Therefore, we begin by explaining in more detail how such register updates are formalised.

Registers and register updates. Suppose that we have a set $\mathcal{X}$ of register names and an output alphabet B . During its computation, the transducer will maintain a register valuation of type $\mathcal{X} \rightarrow B^*$, i.e. the registers will store strings over the output alphabet. To update the register valuation, we will use register updates of the form

$$u : \mathcal{X} \rightarrow (\mathcal{X} + B)^*,$$

which are *copyless* in the following sense: each register name from $\mathcal{X}$ can appear in at most one string of the form $u(X)$, and if it appears, then it appears exactly once in that string. In other words, if we concatenate all the strings $u(Y)$ ranging over $Y \in \mathcal{X}$, then each register name from $\mathcal{X}$ appears at most once in the resulting string. A copyless register update can be applied to a register valuation, yielding a new register valuation. The copyless restriction ensures that for each register update u there is some $k \in \mathbb{N}$ such that if the old register valuation had combined length n , then the new one after applying u has combined length at most $n + k$.

Transducer definition. Having defined register updates, we can now give a formal definition of streaming string transducers. The general idea is that this is a deterministic finite automaton, which processes the input string in a single left-to-right pass, and which uses register updates to compute the output string.

Definition C.3.1 (SST). *A streaming string transducer (SST) consists of:*

- a finite input alphabet A and a finite output alphabet B ;
- a finite set Q of states, with an initial state $q_0 \in Q$;
- a finite set $\mathcal{X}$ of register names;
- a transition function

$$\delta : Q \times A \rightarrow Q \times (\text{copyless register updates}),$$

- a final output function

$$F : Q \rightarrow (B + \mathcal{X})^*.$$

In the final function, there is no copyless restriction. Nevertheless, we could impose such a restriction without changing the expressive power of the model, since the final function is applied once, and therefore any duplication could be treated by computing the appropriate registers in several independent copies. An sst as in the above definition computes a function of type $A^* \rightarrow B^*$ which is defined as follows. After reading an input string $w \in A^*$, the transducer is in a configuration, which consists of a state $q \in Q$ and a register valuation $\eta : \mathcal{X} \rightarrow B^*$. In the initial configuration, the initial state is used, and all registers are empty. Upon reading an input letter, the transition function is applied to determine a new state and a register update which is applied to the current register valuation. After reading the whole input string, the output is obtained by applying the final output function to the state at the end of the computation, and then substituting each register name for its contents in the final configuration.

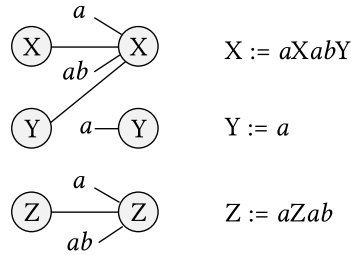
C.3.1 Equivalence with regular functions

In this section we show that the sst model is equivalent to the ones previously described in this part of the book.

Theorem C.3.2. *Streaming string transducers compute exactly the regular functions.*

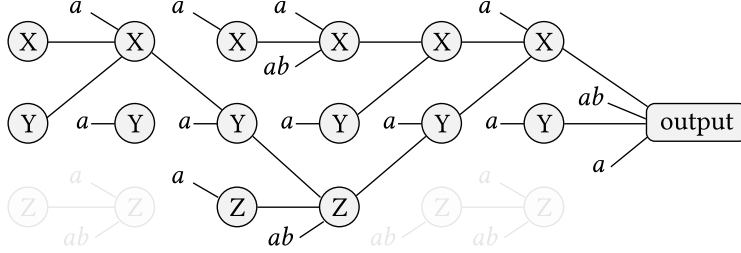
Proof. We begin with the easier implication of the theorem, which is that functions computed by sst's are regular. The converse implication, that regular functions are computed by sst's, is more involved, and relies on the decomposition results for the various classes of transducers that we have proved before.

sst to regular. Let us first show that every sst computes a regular function. Since the regular functions are the same as two-way transducers by Theorem C.2.9, we will prove this by converting an sst into an equivalent two-way transducer. A single register update of the sst can be visualised as a forest (of depth 1), as in the following picture:



The fact that the picture above is a forest, i.e. that the registers in the left column of the above picture have outdegree at most one, is a consequence of the copyless restriction on the register updates. If we compose all the register updates that are applied when reading an input string of length n , and then we add the final output update, then we

get a tree of depth $n + 1$, as explained in the following picture (the unreachable parts are erased from the tree):



We use the name *register flow tree* for this tree. This tree can be computed by a rational function, under a suitable string representation, similar to the one for the configuration graph in Section C.2.1. Since two-way transducers are closed under pre-composition with rational functions by Theorem C.2.5, it remains to show that there is a two-way transducer which inputs a (string representation of) a register flow tree, and outputs the corresponding output string. This is easy to see, since the output string is produced by traversing the register flow tree in a depth-first search, which can easily be implemented by a two-way transducer.

Regular to sst. In this part of the proof, we leverage the decomposition results on the various transducer classes that we have proved before. By definition, a regular function is a composition of prime functions, namely rational functions, map reverse, and map duplicate. We will show that sst's are closed under post-composition with these prime functions, i.e.

$$\text{sst} \cdot \text{primes} \subseteq \text{sst}.$$

This will imply that sst's are closed under post-composition with all regular functions, and hence sst's contain all regular functions. Among the prime functions we find the rational functions, and we will further decompose them using Theorem B.2.6 into: Mealy machines, right-to-left Mealy machines, homomorphisms, and the end-of-string function $w \mapsto w\#$. We now show how to post-compose sst's with these prime functions.

- **Homomorphisms and end-of-string.** Consider a composition

$$A^* \xrightarrow[\text{sst}]{f} B^* \xrightarrow{g} C^*,$$

where g is either a homomorphism or the end-of-string function. We want to show that the composition $f \cdot g$ is computed by an sst. This is proved by a straightforward modification of the sst for f . In the case of a homomorphism, we apply the homomorphism when adding the letters to the registers, while in the case of the end-of-string function, we simply add the letter $\#$ to the output in the final output function.

- **Map reverse.** Let us now give an SST for a composition

$$A^* \xrightarrow[\text{SST}]{f} (B+1)^* \xrightarrow{\text{map rev}} (B+1)^*.$$

To do this, we modify the SST f as follows. For a string stored in a register X of this SST, we will be interested in three parts, as illustrated in the following example:

$$\begin{array}{ccc} \text{before first} & & \text{after last} \\ \text{separator} & \text{from first separator to last separator} & \text{separator} \\ \hline \underbrace{abcd}_{\text{before first separator}} \# \underbrace{abc\#abcdd\#abbc\#abcca\#}_{\text{from first separator to last separator}} \underbrace{aaabdd}_{\text{after last separator}}. \end{array}$$

There is an edge case where the middle part is empty, i.e. there is no separator; in this case we assume that the first part is the whole string. If there is one separator only, then the middle part is $\#$.

In the new SST computing the composition of f with map reverse, each register X of f is replaced by three registers X_1, X_2, X_3 that correspond to these three parts. The invariant, which compares the values of registers X_1, X_2, X_3 in the new SST with the value of register X in the original SST, is the following:

$$\begin{array}{ccc} X_1 \text{ stores first} & X_2 \text{ stores middle part with map reverse applied} & X_3 \text{ stores third} \\ \text{part reversed} & & \text{part reversed} \\ \hline \underbrace{cdba}_{X_1} \quad \underbrace{\#cbba\#ddcba\#dcba\#accba\#}_{X_2} \quad \underbrace{dddbbaaa}_{X_3}. \end{array}$$

We now describe how the register updates are modified in the new SST. We only show the construction for atomic updates, which are:

$$\begin{array}{ccc} \text{prepend a letter} & \text{append a letter} & \text{concatenation} \\ \hline \underbrace{X \mapsto aX} & \underbrace{X \mapsto Xa} & \underbrace{X \mapsto YZ}. \end{array}$$

- **Prepending a letter.** If the original SST does a prepend operation $X \mapsto aX$ for a letter a that is not the separator symbol, then the new SST simulates this by adding the same letter to the left part in the opposite order, i.e. $X_1 \mapsto X_1a$. If the prepended letter is the separator symbol $\#$, then we need to move the left part into the middle part, which is implemented by the following update (updates are, as usual, simultaneous):

$$\begin{array}{l} X_1 \mapsto \varepsilon \\ X_2 \mapsto \#X_1X_2. \end{array}$$

- **Appending a letter.** The construction for appending letters is symmetric to the one for prepending, except that the third part is used instead of the first one.
- **Concatenation.** Suppose now that the original SST does a concatenation update $X \mapsto YZ$. There are several cases to consider depending on whether the middle part of Y and/or Z is empty or not. To manage

this case distinction, the new SST keeps in its state the information about which middle parts are nonempty. When both middle parts are nonempty, then the new SST does the following updates:

$$\begin{aligned} X_1 &\mapsto Y_1 \\ X_2 &\mapsto Y_2 Z_1 Y_3 Z_2 \\ X_3 &\mapsto Z_3. \end{aligned}$$

The remaining cases, when some middle parts are empty, are handled by similar updates, which are left to the reader.

- **Map duplicate.** The construction for map duplicate is similar to the previous one. This time each register of the original SST is split into five registers instead of three, since we need an extra copy for each of the first and last parts.
- **Mealy machines.** We are left with Mealy machines, and right-to-left Mealy machines. Since the construction is symmetric, we only do it for the normal, left-to-right Mealy machines. Consider a composition

$$A^* \xrightarrow[\text{SST}]{f} B^* \xrightarrow[\text{Mealy}]{g} C^*$$

of an SST followed by a Mealy machine. Similarly to the previous cases, we will modify the SST by integrating the actions of the Mealy machine into its register updates. We begin with an erroneous construction, which will help explain the underlying issue. For each state q in the state space Q of the Mealy machine, let g_q be the Mealy machine obtained from g by making q the initial state. A natural idea is to create a new SST f' , in which every register X of f is replaced by a family of registers $\{X_q\}_{q \in Q}$, subject to the following invariant: register X_q in the new SST stores g_q applied to the string stored in X by the original SST. However, this construction is not compatible with the copyless restriction. Indeed, if we want to simulate a concatenation update $X \mapsto YZ$ of the original SST, then the corresponding family of updates would be

$$X_q \mapsto Y_q Z_{q'},$$

where q' denotes the state of the Mealy machine after starting in q and reading the string in Y . However, it could be the case that the map $q \mapsto q'$ is not injective, and thus we would need to use several copies of a register $Z_{q'}$. To solve this issue, we will use the Krohn-Rhodes decomposition theorem, to decompose the Mealy machine into reversible machines and flip-flops, and do the construction for these simpler machines.

- **Reversible Mealy machines.** For reversible machines, the map $q \mapsto q'$ is injective, and thus the above construction works without any modification.

- **Flip-flop Mealy machines.** Consider now a flip-flop machine. In this case, we use a similar construction as for map reverse. For each register X of the original sst, we will care about two parts, depending on where the reset letters (those with a constant state transformation) are located, as explained in the following picture:

$$\begin{array}{c} \text{up to first reset} \\ \text{letter (in red)} \end{array} \quad \begin{array}{c} \text{remaining part} \end{array}$$

$$\overbrace{abbbababaaac}^{\text{up to first reset letter (in red)}} \overbrace{abababba\textcolor{red}{d}abababaaabcbbaabaaacbabbb}^{\text{remaining part}}.$$

In the edge case, where there is no reset letter in X , the first part is the whole string. For each register X in the original sst, the new sst will have $|Q| + 1$ registers: one register X_{right} for the right part, and for each state q of the Mealy machine, one register X_q which stores the left part of X transformed along f_q . It will also remember in its state: (a) the first constant letter in X ; and (b) the last constant letter in X . A concatenation operation $X \mapsto YZ$ in the original sst is simulated by the following updates (with q ranging over states of the Mealy machine) in the new sst:

$$\begin{aligned} X_q &\mapsto Y_q \\ X_{\text{right}} &\mapsto Y_{\text{right}} Z_p Z_{\text{right}}, \end{aligned}$$

where p is the state of the Mealy machine after reading Y concatenated with the left part of Z ; the relevant state p can be inferred from the information stored in the state of the new sst. This is a copyless update, as required. The updates for prepending/appendix letters are similar, and are left to the reader.

This completes the proof that sst's are closed under post-composition with the Mealy machines, and more generally, all prime functions used in the regular functions.

Having proved that sst's are closed under post-composition with all regular functions, we conclude that sst's contain all regular functions, and this completes the proof of Theorem C.3.2. $\square$

C.4 Logic

In this chapter, we describe a logical perspective on regular languages and transducers. The underlying theme is the famous automata-logic correspondence, where regular languages (and also transductions) are defined using monadic second-order logic.

C.4.1 Monadic second-order logic

In this section, we begin by describing how logic can be used to define languages. In the next sections, we extend this to defining transductions.

The general idea behind the logical perspective is to describe a property of a word using a formula which quantifies over its positions, and has access to the order on positions as well as their labels. For example, the formula

$$\underbrace{\forall x}_{\text{for every position } x} \quad \underbrace{\exists y}_{\text{there exists a position } y} \quad \underbrace{a(y)}_{\substack{\text{such that} \\ y \text{ has} \\ \text{label } a}} \quad \wedge \quad \underbrace{y \leq x}_{\text{and is before } x}$$

describes the language A^*a of strings that end with the letter a . The first logic that comes to mind is first-order logic, where formulas are constructed using

$$\underbrace{\exists x \quad \forall x}_{\text{quantification over positions}} \quad \underbrace{\neg \varphi \quad \varphi_1 \wedge \varphi_2 \quad \varphi_1 \vee \varphi_2}_{\text{Boolean combinations}} \quad \underbrace{x \leq y \quad a(x)}_{\text{atomic formulas for order and label tests}},$$

where the letter a is taken from the input alphabet. We use the name *first-order logic over A^** for this logic, where the finite alphabet A is a parameter of the logic. As we will see, formulas of this logic can only define regular languages.

Example 24. The language $(ab)^*$ is defined by the following formula

$$\bigwedge \left\{ \begin{array}{l} \underbrace{\forall x \exists y a(y) \wedge y \leq x}_{\text{string begins with } a} \\ \underbrace{\forall x \exists y b(y) \wedge y \geq x}_{\text{string ends with } b} \\ \underbrace{\forall x \forall y x + 1 = y \Rightarrow a(x) \wedge b(y) \vee b(x) \wedge a(y)}_{\text{string alternates between } a \text{ and } b} \end{array} \right.$$

In the above formula, the successor relation $x + 1 = y$ can be defined in terms of the order relation as follows:

$$x + 1 = y \stackrel{\text{def}}{=} x < y \wedge \neg \exists z x < z < y,$$

where $<$ is defined in terms of $\leq$ as usual. $\square$

As it turns out, first-order logic is not strong enough to define all regular languages. The classical example is parity: first-order logic over a one-letter alphabet

$\{a\}$ cannot define the regular language $(aa)^*$ of even-length strings. (A formal argument will be given at the end of this chapter.) For this reason, we extend first-order logic with an extra feature: the ability to quantify over sets of positions; the resulting logic is called *monadic second-order logic* (MSO). In MSO, we have two kinds of variables: first-order variables, which range over positions, and second-order variables, which range over sets of positions. We use the convention that first-order variables are denoted by lowercase letters x, y, z , and second-order variables by uppercase letters X, Y, Z . The logic MSO is obtained by extending first-order logic with the following features:

$$\underbrace{\exists X \quad \forall X}_{\substack{\text{quantification} \\ \text{over sets of positions}}} \quad \underbrace{x \in X}_{\text{set membership}}$$

Example 25. The language $(aa)^*$ can be defined in MSO by saying that the positions can be labelled with two colours (odd and even), so that the colours alternate along the string, the first position is odd, and the last position is even. The corresponding formula is a variant of the formula for $(ab)^*$ in Example 24:

$$\exists X \bigwedge \left\{ \begin{array}{l} \underbrace{\forall x \exists y y \in X \wedge y \leq x}_{\text{first position is in } X} \\ \underbrace{\forall x \exists y y \notin X \wedge y \geq x}_{\text{last position is not in } X} \\ \underbrace{\forall x \forall y x + 1 = y \Rightarrow (x \in X \Leftrightarrow y \notin X)}_{\text{string alternates between } X \text{ and not } X} \end{array} \right.$$

□

Theorem C.4.1. A language $L \subseteq A^*$ is regular if and only if it is definable in MSO.

Proof. We begin with the inclusion

$$\text{regular} \subseteq \text{MSO}.$$

To prove this inclusion, we show that the logic MSO is rich enough to formalise the definition of an accepting run. Indeed, consider a nondeterministic automaton with k transitions

$$q_1 \xrightarrow{a_1} p_1, \dots, q_k \xrightarrow{a_k} p_k$$

The corresponding MSO formula (which is a variant of the parity formula from Example 25) says that the input string can be labelled by transitions in a way that represents an accepting run (for readability, we use finite disjunctions $\bigvee_i$, whose ranges are de-

scribed below the formula):

$$\exists X_1 \dots \exists X_k \quad \wedge \left\{ \begin{array}{l} (1) \text{ every position is labelled by at least one transition:} \\ \forall x \bigvee_i x \in X_i \\ (2) \text{ consecutive transitions agree on the intermediate state:} \\ \forall x \forall y \ x + 1 = y \Rightarrow \bigvee_{(i,j)} x \in X_i \wedge y \in X_j \\ (3) \text{ the first transition has a source state that is initial:} \\ \forall x \exists y \ y \leq x \wedge \bigvee_i y \in X_i \\ (4) \text{ the last transition has a target state that is accepting:} \\ \forall x \exists y \ y \geq x \wedge \bigvee_i y \in X_i. \end{array} \right.$$

The disjunctions in the four items range as follows: (1) i ranges over all transitions; (2) (i, j) ranges over all pairs of transitions such that the target state of i is the source state of j ; (3) i ranges over all transitions that have an initial source state; and (4) i ranges over all transitions that have an accepting target state. These are finite disjunctions, and hence they correspond to repeated use of $\vee$, and not some kind of quantification. The formula also uses the successor relation $x + 1 = y$, which can easily be defined in terms of the order. The formula allows a position to be labelled by several transitions, but if the formula is true, then it can also be made true by a choice where each position is labelled by exactly one transition. It should be easy to see that the formula is true exactly in the words that admit at least one accepting run.

Let us now prove the other inclusion

$$\text{regular} \supseteq \text{MSO}.$$

By induction on the size of a formula of MSO, we will show that the corresponding language is regular. Although the outermost formula does not have free variables, its subformulas will have free variables, and therefore we need to formally associate a language also to formulas that have free variables. This is done by encoding the values of the variables as bits that are stored in the labels of positions. For a string $w \in A^*$ and subsets $X_1, \dots, X_k$ of its positions, let us define

$$w \otimes X_1 \otimes \dots \otimes X_k$$

to be the string over alphabet $A \times 2^k$ in which every position carries its original label from w , and also k bits of information which indicate the sets from $X_1, \dots, X_k$ that it belongs to. Using this notation, we can use languages to describe formulas with free variables. The following lemma, in the case of formulas that have no free variables, establishes the desired inclusion in the theorem.

Lemma C.4.2. *Let φ be an MSO formula over alphabet A , with*

$$\text{free variables of } \varphi \subseteq \underbrace{\{x_1, \dots, x_k\}}_{\text{first-order variables}}, \underbrace{\{X_1, \dots, X_\ell\}}_{\text{set variables}}.$$

Then the following language over alphabet $A \times 2^{k+\ell}$ is regular:

$$\{ w \otimes \{x_1\} \otimes \dots \otimes \{x_k\} \otimes X_1 \otimes \dots \otimes X_\ell \mid w \models \varphi(x_1, \dots, x_k, X_1, \dots, X_\ell) \}.$$

Proof. The regular language will always need to check that the input string is well-formatted, which means that the bits for the first-order variables are such that each variable selects exactly one position. This property is easily seen to be regular. The lemma is now proved by induction on the structure of the formula.

- Consider the atomic formula $x_i \leq x_j$. The corresponding language says that the input string is well-formatted, and the unique position selected by x_i is to the left of the unique position selected by x_j . This is easily seen to be regular. The other atomic formulas $x_i \in X_j$ and $b(x_i)$ are treated in the same way.
- For the Boolean connectives, use the fact that regular languages are closed under Boolean combinations. In the case of negation, we also need to check again if the input string is well-formatted.
- Consider an existential set quantifier $\exists X$. The corresponding automaton non-deterministically guesses for each position a bit 0 or 1 that is added to its label, and then runs the automaton from the induction assumption. For an existential first-order quantifier $\exists x$, we do the same, except that we need to ensure that only one position is selected. For the universal quantifiers, we use De Morgan's laws to reduce them to existential quantifiers and negation.

□

It is worth pointing out that the construction in the above lemma has a large computational cost. When we apply an existential quantifier, we create a nondeterministic automaton. When we apply complementation, we need to complement the automaton, which is done using determinisation. Therefore, whenever we alternate an existential quantifier with negation, the construction incurs an exponential blowup, thus leading to a tower of nested exponentials. This is indeed unavoidable, since one can show that the construction cannot be done so that it has a tower of exponentials of fixed height, independent of the formula. □

C.4.2 Rational functions in terms of logic

In the previous section, we showed how monadic second-order logic can be used to define languages. In this section, we extend the correspondence to cover rational functions. The relevant model is described in the following definition.

Definition C.4.3 (MSO relabelling). *An MSO relabelling consists of:*

1. *an input alphabet A ;*
2. *an output alphabet B ;*
3. *a finite set Φ of MSO formulas over the input alphabet, such that each formula $\varphi(x)$ in Φ has exactly one free variable, which is of first-order kind;*
4. *an output map of type $\Phi \rightarrow B^*$;*
5. *a string in B^* for the empty input.*

We require that for every input string and every position in it, there is a unique formula from Φ which is true in that position.

An MSO relabelling defines a function of type $A^* \rightarrow B^*$ as follows. If the input string is empty, then we use the string from the last item. Otherwise, if the input string is nonempty, then to each input position we associate the unique formula from Φ that is true in that position, and we label the output position corresponding to it by the same letter as the formula. The resulting string – with the outputs concatenated from left to right – is the output of the function.

Theorem C.4.4. *A string-to-string function is rational if and only if it is definable by an MSO relabelling.*

Proof. We begin with the inclusion

$$\text{rational} \subseteq \text{MSO relabellings}.$$

Consider a rational function $f : A^* \rightarrow B^*$. By Lemma B.2.4, we know that f is computed by a nondeterministic one-way transducer, which has the following properties: (1) for every input string there is exactly one accepting run; and (2) if the input string is nonempty, then every transition used in the accepting run consumes exactly one input letter. Let

$$\Delta \subseteq Q \times A \times B^* \times Q$$

be the finite set of transitions that are used in accepting runs over input strings; these transitions consume exactly one input letter. The following claim shows that we can define runs of the transducer in MSO.

Claim C.4.5. *For each transition $t \in \Delta$ there is an MSO formula $\varphi_t(x)$ over alphabet A such that for every $w \in A^*$ and every position x in w ,*

$$w \models \varphi_t(x)$$

if and only if x is a position where transition t is used in the unique accepting run on w .

Proof. The same construction as in the proof of $\text{regular} \subseteq \text{MSO}$ in Theorem C.4.1, except that we need to additionally assert that transition t is used in position x . $\square$

The set of formulas in the MSO relabelling is in one-to-one correspondence with the transitions:

$$\Phi = \{ \varphi_t(x) \text{ from Claim C.4.5} \mid t \in \Delta \}.$$

Because the transducer is unambiguous, each position is selected by a unique formula, as required in the definition of an MSO relabelling. The output function maps $\varphi_t(x)$ to the output string in the transition t . It is easy to check that the MSO relabelling defined in this way computes the same function f as the original transducer.

Let us now prove the other inclusion

$$\text{rational} \supseteq \text{MSO relabellings}.$$

Consider an MSO relabelling defined by a set Φ of formulas.

Claim C.4.6. *The following language over alphabet $A \times \Phi$ is regular:*

$$\{ (a_1, \varphi_1) \cdots (a_n, \varphi_n) \mid \begin{array}{l} \text{for every } i \in \{1, \dots, n\}, \text{ formula } \varphi_i \in \Phi \\ \text{is true in position } i \text{ of } a_1 \cdots a_n \in A^*. \end{array} \}.$$

Proof. Because each formula in Φ is a formula of MSO, it follows that the language in the statement of the claim is definable in MSO, the corresponding formula being

$$\bigwedge_{\varphi \in \Phi} \forall x (\varphi \text{ is in the label of } x) \Rightarrow x \text{ is selected by } \varphi \text{ after projecting letters to } A.$$

Thanks to Theorem C.4.1, the language is regular. $\square$

The language above can be seen as a length-preserving rational relation

$$R \subseteq A^* \times \Phi^*,$$

which for input $a_1 \cdots a_n$ outputs all possible choices of $\phi_1 \cdots \phi_n$ such that the corresponding zipped word belongs to the language from the above claim. Because of the unambiguity requirement in the definition of an MSO relabelling, we know that this relation is in fact a function. Therefore, in order to produce the output of the MSO relabelling, we can apply this rational function, followed by a homomorphism which maps each formula from Φ to the corresponding output string in B^* . Since rational functions are closed under composition, it follows that the function defined by the MSO relabelling is rational. $\square$

Exercises

Exercise C.4.1. Show that a function $f : A^* \rightarrow B^*$ is computed by a Mealy machine if and only if it is definable by an MSO relabelling with the following additional restrictions imposed on Definition C.4.3:

1. the string for empty input in item 5 is empty;
2. the output map in item 4 maps each formula to a one-letter string;
3. the formulas in item 3 depend only on the past, which means that $w \models \varphi(x)$ depends only on the prefix of w up to and including position x .

C.4.3 Regular functions in terms of logic

In this section, we define the logical model for the regular functions. This model is called *MSO transductions*. The idea is that we use formulas of MSO logic to define a new order on the input positions, and to change the labels. For example, the reverse function is defined by saying that the labels are the same as in the input string, but the order is reversed. A more sophisticated formula is needed to define the order

after applying map reverse, but this can still be done. For functions that create new positions (such as map duplicate), we will need an extra duplication feature in the model. To make this feature easier to use, as well as to prepare the ground for more advanced models of logical transduction in the next part, we begin by introducing an extended syntax for MSO logic.

Extended syntax for MSO logic. We will want to quantify over objects such as gaps in strings, configurations, or sets of configurations. For example, the set of gaps in a string of length n is $n + 1$, and therefore quantifying over a gap corresponds to either quantifying over a position, or otherwise considering the special case of the extra gap that is not associated with any position. Such quantification can be handled directly in MSO, by considering different cases inside formulas. Nevertheless, in order to handle such constructions in a succinct and yet precise way, we consider an extended notation for MSO logic, in which first-order variables can have types such as:

$$\underbrace{x : n + 1}_{\substack{\text{a variable} \\ \text{that ranges} \\ \text{over gaps}}} \quad \underbrace{x : Q \times (n + 1)}_{\substack{\text{a variable that} \\ \text{ranges over} \\ \text{configurations}}} \quad (11)$$

To implement this, we will use an extended syntax, in which first-order variables are assigned types that are polynomials of the form

$$\tau = n^{d_1} + \dots + n^{d_k}.$$

Example 26. Using a variable $x : n^2$, we can quantify over pairs of positions. To access the components of this variable, we will have a feature in the logic that will unpack it as a pair (x_1, x_2) of variables of type n . $\square$

Example 27. The summation in a polynomial is used to represent alternative variants. For example, the polynomial $n^0 + n^0$, which we denote by $1 + 1$, allows us to quantify over Boolean values. To access a variant type, we will have a feature in the logic that will allow us to identify which variant is used. $\square$

The polynomials are used as types for first-order variables. We extend this notation to second-order variables, so that we use variables such as

$$\underbrace{X : 2^{Q \times (n+1)}}_{\substack{\text{a variable that} \\ \text{ranges over} \\ \text{sets of configurations}}}.$$

For the second-order variables, the types will be of the form 2^τ , such that the polynomial τ is linear, i.e. it only uses monomials of degree zero or one. (The linear restriction corresponds to the fact that we are working with the monadic fragment of second-order logic.) We have the same formula constructors as in MSO logic, except that the

quantifiers are now typed:

$$\underbrace{\exists x : \tau \quad \forall x : \tau}_{\text{first-order quantifiers}} \quad \underbrace{\exists X : 2^\tau \quad \forall X : 2^\tau}_{\text{second-order quantifiers}} \quad \underbrace{\vee \wedge \neg}_{\text{Boolean connectives}} \quad \underbrace{x \leq y \quad a(x)}_{\substack{\text{atomic formulas} \\ \text{for order and labels}}} \quad \underbrace{x \in X}_{\text{set membership}}.$$

For the atomic formulas, the variables need to represent positions, i.e. their types must be n , while for set membership, the types need to match: if x has type τ then X must have type 2^τ . Finally, we add a feature for unpacking the polynomial types. For every variable x that has a type

$$\tau = n^{d_1} + \dots + n^{d_k},$$

and for every $i \in \{1, \dots, k\}$ we have an atomic formula

$$x = i(\underbrace{x_1, \dots, x_{d_i}}_{\text{these variables have type } n}),$$

which says that x uses the i -th monomial, and identifies the coordinates of it.

Example 28. If we have a Boolean variable $x : 1 + 1$ then we can check if it is true or false using the atomic formulas $x = 1()$ and $x = 2()$. $\square$

Here is a more sophisticated example of the logic, which computes the transitive closure of the one-step reachability relation of a two-way transducer.

Example 29. Consider a two-way transducer which has input alphabet A and states Q . If an input string has length n , then it has $n + 1$ gaps, with one gap before the input string and one gap after each position. Therefore, the set of configurations of this automaton is produced by the polynomial

$$\tau = Q \cdot (n + 1) = \underbrace{n + \dots + n}_{|Q| \text{ times}} + \underbrace{1 + \dots + 1}_{|Q| \text{ times}}.$$

We first observe that one-step reachability can be defined in MSO. This means that we can write a formula

$$\delta(x : \tau, y : \tau)$$

which says that the transducer can go in one step from configuration x to configuration y . This is done by unpacking the state and position in x , similarly for y , and then comparing them with respect to the definition of transitions in the transducer. The more interesting observation is that we can also define multi-step reachability. The corresponding formula

$$\delta^*(x : \tau, y : \tau)$$

says that for every set $X : 2^\tau$ of configurations that is closed under the one-step reachability relation we have

$$x \in X \implies y \in X.$$

Observe that we can quantify over sets of configurations, since the type τ has degree at most one, which corresponds to the fact that the number of configurations is linear in the length of the input string. $\square$

String-to-string MSO transductions. We now explain how MSO logic can be used to define regular string-to-string functions. The idea is that the positions of the output string are defined by some formula $\varphi(x : \tau)$, where τ is a linear polynomial. This means that we can duplicate each input position a constant number of times, and then possibly add a constant number of extra positions.

Definition C.4.7. A string-to-string MSO transduction is given by:

- an input alphabet A ;
- an output alphabet B ;
- a type $\tau \in \mathbb{N}[n]$, which is linear;
- a family of formulas

$$\underbrace{\varphi_{\text{univ}}(x : \tau)}_{\text{universe formula}} \quad \underbrace{\varphi_b(x : \tau)}_{\substack{\text{one letter formula} \\ \text{for each } b \in B}} \quad \underbrace{\varphi_{\leq}(x : \tau, y : \tau)}_{\substack{\text{order} \\ \text{formula}}},$$

which are MSO formulas (in extended syntax) over the alphabet A .

We require that for every input string $w \in A^*$, every element $x \in \tau(w)$ selected by the universe formula satisfies exactly one of the letter formulas, and that the order formula defines a linear order on the elements selected by the universe formula

The semantics of an MSO transduction is a function of type $A^* \rightarrow B^*$, which is defined as follows. For an input string $w \in A^*$, the output string is obtained by taking the elements of $\tau(w)$ that satisfy the universe formula, ordering them according to the order formula, and then labelling them according to the letter formulas. The assumption that the type τ is linear will ensure that the output has linear size, as required in a regular function. Later on in this book, we will consider an extension where the linearity requirement is dropped. Also, MSO transductions can be used to define functions on other structures, such as graphs or trees, but this is beyond the scope of this book.

Example 30. [Reverse as an MSO transduction] Let us show that string reversal over alphabet A can be defined as an MSO transduction. Since the positions in the output string are the same as the positions in the input string, we use the type $\tau = n$. The labels are inherited from the input string, so the letter formula $\varphi_a(x)$ is $a(x)$. Finally, the order is reversed, so the order formula is

$$\varphi_{\leq}(x : n, y : n) \stackrel{\text{def}}{=} x \geq y.$$

□

Example 31. [Outputs of constant length] Consider a string-to-string function

$$f : A^* \rightarrow B^*$$

in which all outputs have at most one letter. In this case, we can use the type $\tau = 1$. For each output letter $b \in B^*$, the label formula

$$\varphi_b(x : 1)$$

is an MSO formula that describes the regular language of input strings that are mapped to b . (The formula ignores the variable x , since it carries no information.) The order formula is “true”, since there is at most one position to order. The same construction would work for functions with outputs of length at most k , by using the type $\tau = 1 + \dots + 1$ with k copies of 1, so that all output positions can be covered. $\square$

Example 32. [Duplication as an MSO transduction] We describe an MSO transduction for the duplicating function $w \mapsto ww$. In this case, the type is $\tau = n + n$. For each letter $a \in A$ in the alphabet, the corresponding letter formula says that its argument is one of the copies of an input position with label a , i.e.

$$\varphi_a(x : \tau) \stackrel{\text{def}}{=} \exists y : n \quad a(y) \wedge (x = 1(y) \vee x = 2(y)).$$

The order formula uses the lexicographic order, with the first copy of the input string coming before the second copy:

$$\varphi_{\leq}(x : \tau, y : \tau) \stackrel{\text{def}}{=} \vee \begin{cases} \text{both from first copy:} \\ \exists x' : n \exists y' : n \quad x = 1(x') \wedge y = 1(y') \wedge x' \leq y' \\ \text{from different copies:} \\ \exists x' : n \exists y' : n \quad x = 1(x') \wedge y = 2(y') \\ \text{both from second copy:} \\ \exists x' : n \exists y' : n \quad x = 2(x') \wedge y = 2(y') \wedge x' \leq y' \end{cases}$$

$\square$

Theorem C.4.8. *String-to-string MSO transductions define exactly the regular functions.*

Proof. As our model for the regular functions, we use two-way transducers. We begin with the easier direction

$$\text{2way} \subseteq \text{MSO transductions}.$$

One approach would be to show that MSO transductions are closed under composition, see the exercises, which would reduce the proof to the case of prime functions. However, the proof for general two-way transducers is easy enough so that we do not need to decompose into prime functions. Indeed, we can simply use MSO to formalise the semantics of a two-way transducer. We assume without loss of generality that the two-way transducer outputs at most one letter in each transition; this can be ensured by using extra transitions. As the affine type, we use the type

$$\tau = Q \cdot (n + 1) = \underbrace{n + \dots + n}_{|Q| \text{ times}} + \underbrace{1 + \dots + 1}_{|Q| \text{ times}}$$

that was used in Example 29. The universe formula says that $x \in \tau(w)$ represents a reachable configuration such that the transducer outputs one letter in this configuration. This can be defined in MSO as explained in Example 29. The letter formulas identify the letter produced by the configuration, while the order is defined as in Example 29.

We now turn to the more difficult direction

$$\text{2way} \supseteq \text{MSO transductions}.$$

from MSO transductions to two-way transducers. We begin by reducing to the special case where the type τ is n , which will make the proof easier.

Lemma C.4.9. *We can assume without loss of generality that the type τ is n .*

Proof. The crucial observation is that types more complicated than n can be reduced away by using rational functions:

- (*) every MSO transduction can be obtained by composing a rational function with an MSO transduction that uses type n .

Once we have proved (*), the lemma will follow, since two-way transducers are closed under precomposition with rational functions. It remains to prove (*). Consider an MSO transduction that has type $\tau = \alpha n + \beta$. The rational function copies each input letter α times, and then appends β extra letters, as in the following example:

$$123 \mapsto 0^\alpha 1^\alpha 2^\alpha \#^\beta.$$

If f is this rational function, then there is a one-to-one correspondence between $\tau(w)$ and positions in $f(w)$. This correspondence and the structure of order and labels can be defined in MSO, and thus the output of the original MSO transduction can be obtained by a new MSO transduction that uses type n . $\square$

Thanks to the above lemma, we assume that the type τ is n , which means that the output positions are in one-to-one correspondence to some subset of the input positions. To design the two-way automaton, we use the following lemma.

Lemma C.4.10. *Let Φ be a finite set of MSO formulas over some alphabet A , which have either one or two free variables of type n . There is a letter-to-letter rational function*

$$f : A^* \rightarrow C^*$$

with the following property for every formula $\varphi \in \Phi$:

- *If φ has one free variable, then there is a subset $F_\varphi \subseteq C$ such that*

$$w \models \varphi(x : n) \iff F_\varphi \text{ contains the label of } x \text{ in } f(w).$$

- *If φ has two free variables, then there is a regular language $L_\varphi \subseteq C^+$ such that*

$$w \models \varphi(x : n, y : n) \iff L_\varphi \text{ contains the infix } [x..y] \text{ in } f(w).$$

Proof. For the formulas that have one free variable, there is no difficulty: the label in the output of f can simply store which positions are selected by which formulas, which can be done thanks to the results in the previous section. The more interesting case is that of a formula $\varphi(x : n, y : n)$ with two free variables. As we have shown in the proof of the previous section, the language

$$\{ w \otimes \{x\} \otimes \{y\} \mid w \in C^+ \text{ and } x \leq y \text{ are distinguished positions} \}$$

is recognised by some deterministic automaton, call it $\mathcal{A}$. In the proof, we will be discussing state transformations of this automaton. Consider some input string $w \in C^+$ with two distinguished positions $x \leq y$. In order to check whether

$$w \models \varphi(x, y)$$

holds, it is enough to know the state transformation of the string

$$w \otimes \{x\} \otimes \{y\}. \quad (12)$$

This state transformation can be obtained by composing three state transformations

$$\underbrace{w[..x-1] \otimes \emptyset \otimes \emptyset}_{\substack{\text{we write } \delta_{<x} \text{ for the} \\ \text{state transformation} \\ \text{of this part}}} \quad \underbrace{w[x..y] \otimes \{\text{first}\} \otimes \{\text{last}\}}_{\substack{\text{we write } \delta_{xy} \text{ for the} \\ \text{state transformation} \\ \text{of this part}}} \quad \underbrace{w[y+1..] \otimes \emptyset \otimes \emptyset}_{\substack{\text{we write } \delta_{>y} \text{ for the} \\ \text{state transformation} \\ \text{of this part}}}.$$

The letter-to-letter rational function f in the statement of the lemma labels each position x with the state transformations $\delta_{<x}$ and $\delta_{>x}$. Once we have the range $[x..y]$ of the relabelled string $f(w)$, we can compute the state transformation in (12) as follows: the state transformation $\delta_{<x}$ is stored in the first position, the state transformation $\delta_{>y}$ is stored in the last position, and the state transformation δ_{xy} can be computed by looking at the letters in the range $[x..y]$. All of this can be computed by a regular language L_φ , as required in the conclusion of the lemma. $\square$

Let Φ be the following set of formulas:

1. the letter formulas from the transduction;
2. unary formulas that select the first and last positions of the output order;
3. a formula which selects positions that belong to the output string, have a defined successor in the output order, and this successor is to the left in the input order;
4. the order formula $\varphi_{\leq}$ and its converse $\varphi_{\geq}$.

The formulas in the first three items have one argument, while the two formulas in the last item have two arguments. Apply Lemma C.4.10 to this set of formulas, yielding a letter-to-letter rational transduction

$$g : A^* \rightarrow C^*.$$

Since two-way transducers are closed under precomposition with rational transductions, it is enough to show that the output of the original MSO transduction f can be computed by a two-way transducer that inputs the output of g . In other words, the following diagram commutes

$$\begin{array}{ccccc} & & f & & \\ & \nearrow & & \searrow & \\ A^* & \xrightarrow{g} & C^* & \xrightarrow{h} & B^* \end{array}$$

for some two-way transducer h . The two-way transducer h is described in the following program. In the program, the transducer can ask questions about formulas with one argument from Φ , since the answer to such a question is encoded in the label of the current position.

1. If the input string is empty, output $f(\varepsilon)$. Otherwise, go to step 2.
2. Scan the input string from left to right, and in each position ask if it is the first position. The answer to this question can be determined by looking at the label in $g(w)$, thanks to Lemma C.4.10. Find the position x that is the first in the output order, and go to step 3.
3. Let x be the current position. Ask which of the label formulas is true in it, and produce this letter on the output. Ask if the position x is the last in the output order, and if so terminate the computation. Otherwise, go to step 4.
4. We know that x is not the last position in the output order. Ask if the successor of x , call it y , is to the left or right in the input order. If y is to the right, then start moving to the right, while running the automaton for the language $L_{\leq} \subseteq C^*$ that is associated to the order formula. The position y is the first one such that the infix $[x..y]$ belongs to $L_{\leq}$. If y is to the left, do the same, but using the language $L_{\geq}$ for the converse of the order formula. Go to step 3.

□

C.4.4 The first-order fragment

Recall the notion of an aperiodic string-to-string function from Definition A.2.7, and the corresponding condition on state transformations in Lemma A.2.11. The latter condition only talks about state transformations, and hence it can be meaningfully applied to deterministic automata without output: a DFA is called *aperiodic* if for every state transformation δ , the sequence $\delta^1, \delta^2, \dots$ eventually reaches a stable value. The following theorem shows that the characterisation in Theorem A.2.8, which talked about aperiodic Mealy machines and flip-flops, happens to identify the same class as first-order logic.

Theorem C.4.11. *A language is definable in first-order logic if and only if it is recognised by an aperiodic DFA.*

Proof. This theorem is stated in terms of languages, while Theorem A.2.8 was stated in terms of functions, but the two results are closely related, since languages and Mealy machines are essentially the same. In fact, we will use the correspondence between languages and functions in this proof. It just so happens that compositions of flip-flops are more naturally connected to functions, while first-order logic is more naturally connected to languages.

From aperiodicity to first-order logic. We begin with the implication

$$\text{aperiodic DFA} \subseteq \text{first-order definable}.$$

This is usually considered the more difficult implication in the theorem, but it so happens that we have already done the work for this implication, by proving the Krohn-Rhodes decomposition theorem and its aperiodic variant in Theorem A.2.8. Consider a regular language $L \subseteq A^*$, and let

$$f : A^* \rightarrow \{\text{yes}, \text{no}\}^*$$

be the corresponding Mealy machine, in which the n -th letter of the output string says if the first n letters of the input string belong to L . We prove the difficult implication in three following steps:

$$\begin{array}{c} L \text{ is computed by an aperiodic DFA} \\ \Downarrow \text{by definition} \\ f \text{ is computed by an aperiodic Mealy machine} \\ \Downarrow \text{Theorem A.2.8} \\ f \text{ is a composition of flip-flops} \\ \Downarrow \text{see below} \\ L \text{ is first-order definable} \end{array}$$

It remains to show the last step, i.e. if f is a composition of flip-flops, then L is first-order definable. For this, we introduce the notion of first-order definable Mealy machine, which means that for every output letter b , the language

$$\{ w \in A^* \mid f(w) \text{ ends with letter } b \}$$

is first-order definable. Flip-flop machines are easily seen to be first-order definable, and it is also easy to see – using substitution of formulas – that first-order definable Mealy machines are closed under composition. Hence, if f is a composition of flip-flops, then it is first-order definable, and therefore also L is first-order definable.

From first-order logic to aperiodicity. Let us now turn to the other implication of the theorem, which is

$$\text{aperiodic DFA} \supseteq \text{first-order definable}.$$

To prove this implication, we will associate to each first-order formula a corresponding aperiodic DFA. This will be based on a technique that is called *compositionality of logic*. To explain this technique, it will be easier to work with a more combinatorial representation of first-order formulas, which uses nested sets.

Definition C.4.12 (*k*-types). *For $k \in \{0, 1, \dots\}$, the k -type of a string $w \in A^*$, denoted by $tp_k(w)$, is defined inductively as follows:*

$$tp_0(w) = \emptyset$$

$$tp_{k+1}(w) = \{ (tp_k(w_1), a, tp_k(w_2)) \mid a \in A \text{ and } w = w_1aw_2 \}.$$

By definition, once the alphabet A is fixed there are finitely many k -types for each k , although the number grows very rapidly, with an exponential increase for each step $k \mapsto k + 1$. We now establish that k -types are closely related to first-order formulas. The number k will correspond to the *quantifier-rank* of a first-order formula, which is the maximal number of quantifiers that can be found along a branch in its syntax tree. Equivalently, the quantifier-rank of atomic formulas is 0, Boolean combinations do not increase the quantifier rank (i.e. one uses the maximal quantifier rank of the formulas in the Boolean combination), and quantifiers increment it by one. The following lemma establishes the connection between k -types and first-order formulas of quantifier rank at most k .

Lemma C.4.13. *Two strings in A^* have the same k -type if and only if they satisfy the same first-order formulas of quantifier rank at most k .*

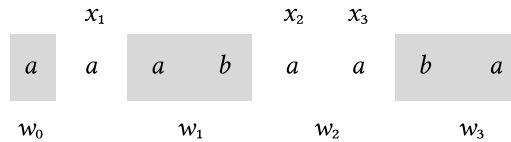
Proof. For our applications, we will not need the right-to-left implication, but we prove it anyway for the sake of completeness. For the right-to-left implication, we observe that for each possible k -type, the corresponding set of strings can be defined in first-order logic with quantifier rank k . The formula quantifies over a position x that corresponds to the letter a in a split $w = w_1aw_2$, and then uses formulas from the induction assumption to describe the types for the parts w_1 and w_2 . The formulas from the induction assumption need to have their quantification restricted to positions that are $< x$ for w_1 and $> x$ for w_2 , but this does not affect the quantifier rank.

Let us now prove the left-to-right implication. To prove this implication, we need a more refined statement that deals with free variables.

Claim C.4.14. *Let $\varphi(x_1, \dots, x_n)$ be a first-order formula of quantifier rank k . Then*

$$w \models \varphi(x_1, \dots, x_n) \quad \text{with } x_1 < \dots < x_n$$

depends only on the labels of the distinguished positions $x_1, \dots, x_n$ and the k -types of the strings $w_0, \dots, w_n$ between them, defined as in the following picture:



Proof. Induction on the structure of φ . The only interesting case is when the formula uses a quantifier, say an existential quantifier

$$\exists x_{n+1} \varphi(x_1, \dots, x_{n+1}).$$

The quantified variable will necessarily fall either on one of the distinguished positions or in some interval between them. If it falls on one of the distinguished positions x_i , then we can apply the induction assumption for the formula obtained from φ by replacing x_{n+1} with x_i . Otherwise, the quantified position falls in one of the intervals $w_0, \dots, w_n$, and we can use the available information and the induction assumption to check if it is true. $\square$

In the case of a formula without free variables, the information in the claim is the same as the k -type of the string, and therefore the claim implies the left-to-right implication of the lemma. $\square$

We now show that the k -types are compatible with operations on strings, which will allow us to define an automaton structure on them, and that the resulting automaton is aperiodic. (In the lemma below, a sequence is called aperiodic if it eventually reaches a stable value, i.e. from some point on all elements in the sequence are the same.)

Lemma C.4.15. *For every $k \in \{0, 1, \dots\}$ and $w, v \in A^*$, we have:*

1. **Refinement:** *if $k > 0$ then $tp_k(w)$ uniquely determines $tp_{k-1}(w)$;*
2. **Congruence:** *$tp_k(w)$ and $tp_k(v)$ uniquely determine $tp_k(wv)$.*
3. **Aperiodicity:** *the sequence $tp_k(w^1), tp_k(w^2), tp_k(w^3), \dots$ is aperiodic.*

Proof. For the induction basis of $k = 0$ there is nothing to do, since there is only one possible type. Let us now prove the induction step. Suppose that we have proved all three properties for k and we want to prove them for $k + 1$.

- **Refinement.** Choose a triple $tp_{k+1}(w)$ and use the congruence property for k to obtain $tp_k(w)$. The answer does not depend on the choice of triple.
- **Congruence.** By definition of types,

$$\begin{aligned} tp_{k+1}(wv) = & \{ (tp_k(w_1), a, tp_k(w_2v)) \mid a \in A \text{ and } w = w_1aw_2 \} \cup \\ & \{ (tp_k(wv_1), a, tp_k(v_2)) \mid a \in A \text{ and } v = v_1av_2 \}. \end{aligned}$$

By the congruence property for k , the first set in the union depends only on the set of triples in $tp_{k+1}(w)$ and the individual value $tp_k(v)$. Since the latter value depends only on $tp_{k+1}(v)$ by the refinement property, it follows that the first set depends only on the types $tp_{k+1}(w)$ and $tp_{k+1}(v)$. Since the same is true for second set, we obtain the congruence property for $k + 1$.

- **Aperiodicity.** By definition, the type $\text{tp}_{k+1}(w^n)$ is equal to the following set

$$\{ (\text{tp}_k(w^{n_1}w_1), a, \text{tp}_k(w_2w^{n_2})) \mid a \in A, w = w_1aw_2, n = n_1 + 1 + n_2 \}.$$

By the congruence and aperiodicity properties for k , the two sequences

$$\begin{aligned} & \text{tp}_k(w^1w_1), \text{tp}_k(w^2w_1), \text{tp}_k(w^3w_1), \dots \\ & \text{tp}_k(w_2w^1), \text{tp}_k(w_2w^2), \text{tp}_k(w_2w^3), \dots \end{aligned}$$

are both aperiodic. From this it follows that the set in the definition of $\text{tp}_{k+1}(w^n)$ must also reach a stable value, since the powers n_1 and n_2 can be capped to some fixed threshold.

□

Using the above lemma, we show that if a language is first-order definable, then it is recognised by an aperiodic DFA, thus completing the proof of the theorem. Let k be the quantifier rank of a first-order formula that defines the language. Define the set of states Q to be the set of possible k -types of strings in A^* . As we have remarked before, this is a finite set. By the congruence property in the above lemma, we can define an automaton structure on this set, since the k -type of wa depends only on the k -type of w and the letter a . By the aperiodicity property in the lemma, the automaton is aperiodic. Finally, since the language is defined by a first-order formula of quantifier rank at most k , it follows that strings that reach the same state cannot be distinguished by the language, and hence we can meaningfully define an accepting set of states in the automaton.

□

First-order definable rational functions. In the above theorem, we described the languages $L \subseteq A^*$ that can be defined in first-order logic. One can also lift this characterisation to string-to-string functions. For the rational functions, the corresponding fragment is the *first-order relabellings*, which are defined in the same way as the MSO relabellings from Definition C.4.3, except that instead of MSO formulas, we use first-order formulas. The corresponding automaton model is bimachines, in which both the prefix and suffix automata are aperiodic (call these aperiodic bimachines).

Theorem C.4.16. *A string-to-string function is a first-order relabelling if and only if it is computed by an aperiodic bimachine.*

Proof. We begin with the direction from aperiodic bimachines to first-order relabellings. Consider the prefix automaton, which is aperiodic. Thanks to Theorem C.4.11, we can describe runs of this automaton in first-order logic. In particular, for each transition t of the prefix automaton, we can write a first-order formula $\varphi_t(x)$ which selects input positions in which t is the transition used by the prefix automaton. Similarly, we can describe in first-order logic the transitions of the suffix automaton. By combining these formulas, we can describe in first-order logic the pairs (transition of the prefix automaton, transition of the suffix automaton) that are used on each input position, and using these pairs we can compute the corresponding parts of the output string.

Let us now prove the opposite direction, from first-order relabellings to aperiodic bimachines. Let k be the maximal quantifier rank of the first-order formulas used in the relabelling. Consider an input string $a_1 \cdots a_n \in A^*$. The output produced by the first-order relabelling on the i -th position depends on which formula from Φ is true in the i -th position. By compositionality, this depends only on the following three pieces of information:

$$\begin{array}{ccc} \text{(1)} & \text{(2)} & \text{(3)} \\ \text{\textit{k}-type of} & \text{this} & \text{\textit{k}-type of} \\ \text{this prefix} & \text{letter} & \text{this suffix} \\ \hline \underbrace{a_1 \cdots a_{i-1}} & \underbrace{a_i} & \underbrace{a_{i+1} \cdots a_n} . \end{array}$$

To retrieve this information, we will use the prefix and suffix automata in the bima-
chine. After reading a prefix of the form $a_1 \cdots a_i$, the prefix automaton will keep track
of information (1) and (2), i.e. the k -type of $a_1 \cdots a_{i-1}$ and the letter a_i . This can be
done by an aperiodic automaton thanks to Theorem C.4.11. Similarly, after reading
the suffix $a_{i+1} \cdots a_n$, the suffix automaton will keep track of information (3), i.e. the
 k -type of this suffix. Therefore, if we know the states of these two automata for a gap
of the form

$$\begin{array}{cc} \text{prefix} & \text{suffix} \\ \hline \underbrace{a_1 \cdots a_i} & \underbrace{a_{i+1} \cdots a_n}, \end{array}$$

with $i > 0$, then we have sufficient information to compute the string produced by
the relabelling on the i -th input position. $\square$

First-order definable regular functions. Finally, it would be useful to give a ma-
chine characterisation of the first-order transductions, i.e. the functions that can be
defined by MSO transductions in which all formulas are first-order. We only state the
appropriate result, and we leave the proof for a future edition of these notes.

Theorem C.4.17. *A string-to-string function is a first-order transduction if and only
if it can be obtained by composing: map reverse, map duplicate and first-order rational
functions (as in Theorem C.4.16).*

Part D: Polyregular functions

In this part, we move to the fourth and final (for this book) step in the transducer ladder, namely the polyregular functions. All transducer classes that we have seen before had linear output size, and the purpose of the polyregular functions is to achieve polynomial output size.

As was the case for the regular functions, probably the quickest way to define the polyregular functions is in terms of prime functions, so we begin with that. We will simply add one more prime function, which has quadratic output size.

Example 33. [Marked squaring] The marked squaring function is illustrated in the following example:

$$1234 \mapsto \underline{1}234 \ \underline{1}234 \ \underline{1}234 \ \underline{1}234.$$

More formally, if the input string has n letters, then it is copied n times, and in the i -th copy the first i letters are underlined⁶. In our example, we have spaces in the output for readability, but the function does not produce them, i.e. the output length is n^2 . (Using a variant with spaces would not change the class of polyregular functions.) As was the case for map reverse and map duplicate, marked squaring is not one function, but a family of functions, with one function of type

$$A^* \rightarrow (\underbrace{A + A}_{\substack{\text{two copies of the} \\ \text{input alphabet, one with} \\ \text{underlines and one without}}})^*$$

for each input alphabet A . $\square$

Definition D.0.18 (Polyregular functions). *A string-to-string function is polyregular if it can be obtained as a finite composition of functions, each of which is either regular, or the marked squaring function from Example 33. In other words:*

$$\text{polyregular} \stackrel{\text{def}}{=} (\text{regular} \cup \text{marked squaring})^*.$$

⁶The underlines might seem mysterious at first, but we will establish later in this book that they are necessary, and a version of the function without underlines would have smaller expressive power.

Of course, the regular functions can be further decomposed into primes, using prime decompositions of regular functions, rational functions, and Mealy machines. The above definition is pleasingly short, but it does not adequately convey the expressive power and algorithmic strength of polyregular functions. To do that, we will present several characterisations of the polyregular functions in this part, using models coming from different fields, such as automata or programming languages.

Before we do that, let us discuss our favourite three properties of transducer classes: continuity, closure under composition, and decidable equivalence. For closure under composition, there is nothing to do, since the class is defined in terms of composition. Decidable equivalence, regrettably, is not known and is a major open problem in this field. We are left with continuity.

Theorem D.0.19. *Polyregular functions are continuous.*

Proof. Since continuous functions are closed under composition, and regular functions are continuous by Theorem C.1.1, it remains to show that marked squaring is continuous. Consider then a composition

$$A^* \xrightarrow{\text{marked squaring}} (A + A)^* \xrightarrow{L} \text{Bool} \quad (13)$$

of marked squaring with a regular language L . We want to show that this composition, which corresponds to the inverse image of L under marked squaring, is a regular language. Suppose that the language L is recognised by a DFA $\mathcal{A}$ with states Q . For a string $w \in A^*$ and a position i in this string, let us write

$$\delta_w^i : Q \rightarrow Q$$

for the state transformation of the automaton $\mathcal{A}$ when reading the string that is obtained from w by underlining the first i input positions. We will decompose the language in (13) into two steps, such that the first step is a rational function, and the second step is a regular language. Composing these two steps will give a regular language, by continuity of rational functions, and will hence prove that the language in (13) is regular.

1. Suppose that the input is a string $w \in A^*$. In the first step, we replace each input position i by the state transformation δ_w^i . The result is a string which has the same length as w , but is over the alphabet of state transformations $Q \rightarrow Q$. We will argue below that this first step is a rational function.
2. If we compose the state transformations in the output of the first step, then we get the state transformation of the automaton $\mathcal{A}$ on the result of applying marked squaring to w . Therefore, in the second step, we: (a) compose all the state transformations in the output of the first step; and then (b) check if the resulting state transformation takes the initial state of $\mathcal{A}$ to an accepting state. The second step can be implemented by a DFA, whose states are state transformations, and is hence a regular language.

As we have explained, the composition of the above two steps gives the same language as the composition in (13), and hence proves regularity of the latter language.

To conclude the proof, we will explain why the first step is a rational function. We will compute it using an unambiguous one-way transducer with output, in which the states are pairs (δ, γ) of state transformations. The invariant is going to be that: (*) δ represents the underlined state transformation of the past, and γ represents the non-underlined state transformation of the future. The transitions are of the form

$$(\delta_1, \gamma_1) \xrightarrow{a/\delta_2 \cdot \gamma_2} (\delta_2, \gamma_2)$$

such that the following conditions are satisfied:

$$\delta_2 = \delta_1 \cdot \underbrace{\delta_a}_{\substack{\text{underlined state} \\ \text{transformation of } a \in A}} \quad \text{and} \quad \gamma_1 = \underbrace{\delta_a}_{\substack{\text{non-underlined state} \\ \text{transformation of } a \in A}} \cdot \gamma_2.$$

The initial states are those where δ is the identity transformation, and the accepting states are those where γ is the identity transformation. This automaton has a unique accepting run on every input string, which satisfies the invariant (*) described above. $\square$

D.1 For-transducers

We begin our discussion of polyregular functions with *for-transducers*, a model that was introduced in [Boj18, Section 3]. Roughly speaking, a for-transducer is an imperative program, which uses for loops to range over positions in the input string. We will use Python syntax for the code examples.

Example 34. [Marked squaring] Here is the source code of a for-transducer that implements the marked squaring function from Example 33. The program has two nested loops: the variable x in the outer loop ranges over copies of the input string, and the variable y in the inner loop ranges over positions inside a copy.

```
1 def markedSquare(w):
2     for x in positions(w):
3         for y in positions(w):
4             if y>x and w[y] == 'a':
5                 output('a')
6             if y<=x and w[y] == 'a':
7                 output('a')
8             if y>x and w[y] == 'b':
9                 output('b')
10            if y<=x and w[y] == 'b':
11                output('b')
```

This program contains the main capabilities of for-transducers: we can loop over input positions, we can test the labels and order on these positions, and we can output letters. $\square$

Apart from the features illustrated in the above example, the remaining features of for-transducers are loops that range over positions in the reverse order, and Boolean variables. We illustrate these features in the next two examples.

Example 35. [Reverse] To implement the reverse function, we use a loop that ranges over positions in the reverse order, as in the following program:

```
1 def reverse(w):
2     for x in positions_reverse(w):
3         if w[x] == 'a':
4             output('a')
5         if w[x] == 'b':
6             output('b')
```

$\square$

Finally, we can use Boolean variables, as illustrated in the following example.

Example 36. [Parity] The following for-transducer outputs the parity of the length of the input string:

```
1 def parity(w):
2     q = True # a Boolean variable
3     for x in positions(w):
4         q = not q
5     #at the end, we output the parity
```

```

6   if q:
7       output('even')
8   else:
9       output('odd')

```

Using k Boolean variables, we could compute the length modulo any number $\leq 2^k$. $\square$

The above examples illustrate all features of for-transducers. Nevertheless, we give below a more formal definition of the syntax of the programming language. In a for-transducer, there are two kinds of variables: position variables that range over positions in the input string, and Boolean variables that range over $\{\text{true}, \text{false}\}$. Apart from that, there is a special variable w that represents the input string, which is an argument of the for-transducer. There are two kinds of loops

`for x in positions(w)` `for x in positions_reverse(w)`

Such a loop creates a position variable x , which ranges over positions of the input string w in either forward or reverse order. The atomic statements are

$\underbrace{\text{output('a')}}_{\text{append letter } a \text{ to the output string}}$	$\underbrace{X = \text{true} \quad X = \text{false}}_{\text{assign a value to a Boolean variable}}$
--	---

It is important that there are no assignments to position variables, i.e. the position variables are read-only. A sequential composition $I; J$ of two for-transducers is also a for-transducer, and there is a conditional `if`, where the condition is a Boolean combination of tests of the following kinds:

$\underbrace{X}_{\text{value of a Boolean variable}}$	$\underbrace{x == y \quad x <= y}_{\text{equality and order tests on positions}}$	$\underbrace{w[x] == 'a'}_{\text{label of position}}$
---	---	---

We will not write variable declarations in the programs, with the types being implicit: the variables bound by the `for` loops are position variables, and the remaining ones are Booleans. The Booleans are initialised to false.

This completes the syntax of for-transducers. The semantics of a for-transducer is a string-to-string function which is defined in the expected way (we have presented the model as a fragment of Python, and the semantics are consistent). The output size of a for-transducer is polynomial – if it has at most k nested loops, then the output size will be $\mathcal{O}(n^k)$. This bound is not tight: one can show that the `map reverse` function – which has linear output size – needs two nested loops in order to be implemented by a for-transducer.

Example 37. The order test $x \leq y$ is in fact redundant, and it can be implemented using equality tests only: run a `for` loop of first-to-last kind, and check which of the variables x or y is seen first in the loop, with the result remembered in a Boolean variable. Similarly, we can implement a successor test $x = y + 1$. As we will see later, we can implement any test $\varphi(x_1, \dots, x_n)$ that can be defined using a formula of monadic second-order logic. $\square$

D.1.1 Equivalence with polyregular functions

In this section, we show that for-transducers are equivalent to polyregular functions, as defined in Definition D.0.18.

Theorem D.1.1. *A string-to-string function is polyregular if and only if it is computed by a for-transducer.*

Proof. We begin with the left-to-right inclusion

$$\text{polyregular} \subseteq \text{for-transducers}.$$

Before proving this inclusion, we would like to remark that it is non-trivial. In fact, already the fact that for-transducers can compute all regular functions is not obvious, at least if we think of regular functions as being computed by two-way transducers. Indeed, it is not immediately clear how a for-transducer can simulate the movement of the head in a two-way transducer, since the loops in a for-transducer are required to move in a single direction. As we will see, the problem can be solved by the decomposition theorems for regular and rational functions that we have proved in the previous parts of this book.

By the definition of the polyregular functions and the decomposition theorem for rational functions in Theorem B.2.6, we know that each polyregular function can be obtained by composing: (1) marked squaring; (2) map reverse; (3) map duplicate; (4) Mealy machines in both directions; (5) homomorphisms; and (6) the end-of-input function $w \mapsto w\#$. We will show that each of these functions can be computed by a for-transducer, and that for-transducers are closed under composition. Marked squaring was treated in Example 34. A for-transducer for map reverse is given in Figure 2, and a similar one can be written for map duplicate. Mealy machines – in both directions – can be implemented using the same construction as in Example 36, with the Boolean variables used to hold a state. Homomorphisms and the end-of-input function are straightforward to implement. We are left with closure under composition.

Before proving closure under composition, we begin by describing a normal form.

Definition D.1.2 (Prenex form). *A for-transducer is in prenex form if it can be written as*

```

1 for  $x_1$  in  $\tau_1$ :
2   for  $x_2$  in  $\tau_2$ :
3     ...
4       for  $x_k$  in  $\tau_k$ :
5          $\text{body}(x_1, \dots, x_k)$ 
6 epilogue
```

such that:

- each τ_i is either `positions(w)` or `positions_reverse(w)`;
- the subprograms `body` and `epilogue` do not use any loops;
- the subprogram `body` produces at most one output letter in each loop iteration.

```

1  def mapReverse(w):
2      #in this loop, x ranges over separator symbols
3      for x in positions(w):
4          if w[x]=='#':
5              #output the block ended by x, in reverse
6              #this variable will have four possible values,
7              #which are: 'before', 'separator', 'inside', and '
              after'
8              q = 'before'
9              for y in positions_reverse(w):
10                 if y == x:
11                     q = 'separator'
12                 if y < x and q == 'separator':
13                     q = 'inside'
14                 if q == 'inside' and w[y]=='#':
15                     q = 'after'
16                     output('#')
17                 if q == 'inside':
18                     output(w[y])
19             #special code for dealing with leftmost block
20             if q == 'inside':
21                 output('#')
22         #an extra loop to process the rightmost block
23         q = 'inside'
24         for y in positions_reverse(w):
25             if w[y]=='#':
26                 q = 'after'
27             if q == 'inside':
28                 output(w[y])

```

Figure 2: A for-transducer that implements the map reverse function. We use a variable *q* that can have four possible values; such a variable can be implemented using two Boolean variables.

The subprogram body is allowed to refer to the position variables $x_1, \dots, x_k$, but the subprogram epilogue is not. There is, however, some form of communication between the two programs, since both of these subprograms refer to the same Boolean variables, and the epilogue can be sensitive to the changes in the Boolean variables that were effected by the various iterations of the body. For example, the following program which checks for emptiness is in prenex form:

```

1 def isEmpty(w):
2     for x in positions(w):
3         q = True
4         if not q:
5             output('not_empty')
```

Lemma D.1.3. *Every for-transducer is equivalent to one in prenex form.*

Proof. The same kind of argument as when converting first-order formulas to prenex form. The epilogue is only needed for inputs of length at most one. For inputs with at least two letters, we can simulate sequential composition $I ; J$ using a for loop, with the first loop iteration running I , the second loop iteration running J , and the remaining iterations unused. To remember if we are in the first or second loop iteration, we use Boolean variables. $\square$

Using the prenex form, we can show that for-transducers are closed under composition.

Lemma D.1.4. *String-to-string functions computed by for-transducers are closed under composition.*

Proof. The construction is easiest to see if we assume that both composed for-transducers are in the prenex form of Lemma D.1.3, as follows:

First for-transducer

```

1 for x1 in  $\tau_1$ :
2     for x2 in  $\tau_2$ :
3         ...
4         for xk in  $\tau_k$ :
5             body1( $x_1, \dots, x_k$ )
6 epilogue1
```

Second for-transducer

```

1 for y1 in  $\sigma_1$ :
2     for y2 in  $\sigma_2$ :
3         ...
4         for y $\ell$  in  $\sigma_\ell$ :
5             body2( $y_1, \dots, y_\ell$ )
6 epilogue2
```

To simplify the notation, we assume below that there is no epilogue in the first for-transducer, i.e. `epilogue1` is empty; dealing with this epilogue is a simple exercise.

Our goal is to write a for-transducer for the composed function, in which the second for-transducer is executed on the output of the first for-transducer. In this composition, the variables $y_1, \dots, y_\ell$ of the second for-transducer range over outputs of the first one, which are in one-to-one correspondence with those tuples $(x_1, \dots, x_k)$ where the body of the first transducer produces an output letter. Therefore, each loop in the second transducer can be replaced by k loops, giving a for-transducer with $\ell \times k$ variables which looks like this (when multiplying kinds of loops, we assume that forward loops are represented by the number 1 and reverse loops are represented by the number -1):

Composed for-transducer

```

1  for z11 in  $\sigma_1 \cdot \tau_1$ 
2    for z12 in  $\sigma_1 \cdot \tau_2$ 
3      ...
4        for z1k in  $\sigma_1 \cdot \tau_k$ 
5          for z21 in  $\sigma_2 \cdot \tau_1$ 
6            for z22 in  $\sigma_2 \cdot \tau_2$ 
7              ...
8                for zℓk in  $\sigma_\ell \cdot \tau_k$ 
9                  body(z11, ..., zℓk)
10 epilogue2

```

The idea is that the new body is obtained from the body of the second for-transducer, by viewing each position variable y_i as a tuple $(z_{i1}, \dots, z_{ik})$. The details of the construction are left to the reader. One of these details is that we only care about iterations where all of the variables $y_1, \dots, y_\ell$ are defined, which means that for every $i \in \{1, \dots, \ell\}$, the body of the first for-transducer produces an output letter in the iteration corresponding to the valuation $(z_{i1}, \dots, z_{ik})$. $\square$

This completes the proof of the left-to-right inclusion in the theorem, since we have shown that for-transducers are closed under composition, and that they can compute all the prime polyregular functions.

Let us now move to the other inclusion

$$\text{for-transducers} \subseteq \text{polyregular}.$$

Thanks to Lemma D.1.3, it suffices to show that for-transducers in prenex form compute polyregular functions. Consider such a for-transducer, with its code being

```

1  for x1 in  $\tau_1$ :
2    for x2 in  $\tau_2$ :
3      ...
4        for xk in  $\tau_k$ :
5          body(x1, ..., xk)
6  epilogue

```

Each iteration of the body is called for some valuation of the position variables. Using the representation described in Section C.4, an input string $w \in A^*$ together with a

valuation of the variables $x_1, \dots, x_k$ can be represented as a string over an extended alphabet:

$$w \otimes x_1 \otimes \cdots \otimes x_k \in (A \times 2^k)^*.$$

We will simulate the for-transducer by a composition of the following functions:

1. In the first phase, we loop through all valuations of the variables $x_1, \dots, x_{k_s}$ in the order given by the nested for loops, and for each one we output the string representation of the valuation, as described above. The output of this phase is illustrated in the following picture, assuming that the input string is $abcd$ and there are two position variables x_1 and x_2 , of types left-to-right and right-to-left respectively:

[illegible]

The construction producing this output is by induction on the number of variables. If we have computed the output for i variables, corresponding to the outermost i loops, then the output for $i + 1$ variables is obtained as follows: apply marked squaring, and then use a rational function to filter out the ill-formatted blocks. The details are left to the reader. In particular, if the loop for the variable x_{i+1} is of the reverse kind, then we need a reverse version of marked squaring

$$abcd \mapsto abcd\ abcd\ abcd\ abcd.$$

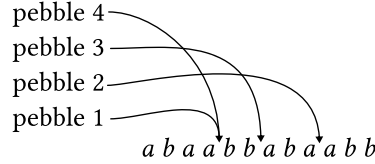
which can be implemented using marked squaring, two instances of reverse, and a rational function.

2. Once we have the output of the first phase, the output of the nested loops in the for-transducer can be computed by a rational function, which maintains the Boolean variables of the for-transducer in its state, scans each block (corresponding to one iteration of the body), and produces at most one output letter per block. The epilogue can then be implemented using one more pass, which only needs to scan one of the copies of the input string.

This completes the translation from for-transducers to polyregular functions, and therefore also the proof of Theorem D.1.1. $\square$

D.2 Pebble transducers

We now describe a second model for polyregular functions, which is called *pebble transducers*. A pebble transducer extends a two-way transducer by having not one head, but a stack of *pebbles* that point to the input string, as in the following picture:



Similarly to one-way and two-way transducers (but unlike for-transducers), our pebbles point to gaps between positions, and not positions. Importantly, the pebbles are organised in a stack, with pebble 1 at the bottom of the stack, and a pebble can only be modified by the transducer if it is at the top of the stack⁷. The stack can be potentially empty, and – more importantly – its height is bounded by a constant k that is given in the pebble automaton. (There are models with unbounded numbers of pebbles [EHS07].) We use the name *head* for the topmost pebble on the stack⁸. Apart from the pebble stack, the automaton has a state from a finite set. To update its configuration, the pebble transducer looks at its state, and the answers to the following *questions*, where $i \in \{1, \dots, k\}$ range over pebble numbers:

1. what are the two input letters adjacent to pebble i ?
2. which other pebbles are in the same place as pebble i ?

Formally speaking, if the input alphabet is A , then the set of possible answers to the above questions is described by the set

$$\sum_{\ell \in \{0, \dots, k\}} \prod_{i \in \{1, \dots, \ell\}} \overbrace{(A+1) \times (A+1)}^{\substack{\text{the two letters} \\ \text{adjacent to the} \\ \text{pebble, with 1 meaning} \\ \text{that there is no letter}}} \times \overbrace{P\{1, \dots, \ell\}}^{\substack{\text{the set of pebbles} \\ \text{in the same place} \\ \text{as pebble } i}}.$$

Based on the current state and the answers to these questions, the pebble transducer deterministically chooses a new state and an *action* from the following set:

1. output letter a ;
2. move the head by one position to the left/right;

⁷ Stack discipline is what ensures that the model is finite-state, which was originally proved in [LH96, Theorem 4.2]. Without the stack discipline the model would be the same as multi-head automata, which correspond to logarithmic space [Iba71, Corollary 3.5].

⁸Like [MSV03] but unlike [LH96, EM02], we count the head as just another pebble. Therefore, a transducer with k pebbles in our notation becomes, in the notation of [LH96, EM02], a transducer with $k - 1$ pebbles and a head. The motivation for our choice is that a transducer with k pebbles in our notation will have output size $\mathcal{O}(n^k)$.

3. push the first input position onto the pebble stack;
4. pop the topmost pebble from the pebble stack;
5. terminate.

Formally speaking, a *configuration* of the pebble transducer is defined to be an input string, together with a state and a stack $x_1, \dots, x_\ell$ of gaps in the input string whose height ℓ is in $\{0, \dots, k\}$.

The transducer begins in the configuration where the stack is empty and the state is a designated initial state. Next, it keeps on updating its configuration according to the above transition rules. The resulting sequence of configurations is called the *run* of the pebble transducer, and it stops when the “terminate” action is executed. The *output string* is defined to be the sequence of output letters that are produced by output actions during the run, in the order that they were executed. The output string is undefined if one of the following errors occurs: (a) the run never terminates; or (b) the head moves out of the input string; or (c) the stack bound is exceeded, by either popping in an empty stack or pushing pebble $k + 1$. We will only care about pebble transducers which define total functions, which means that for every input string none of the errors (a) – (c) occur. We use the name k -pebble transducer for a pebble transducer where the bound on the stack height is k . In the case of $k = 1$, we recover two-way transducers.

D.2.1 Continuity

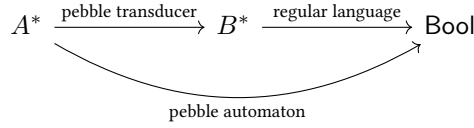
In Theorem D.2.4 later in this section, we will show that pebble transducers compute the same string-to-string functions as for-transducers, and therefore they are continuous and closed under composition. Nevertheless, it is instructive to give a direct proof of continuity, which does not rely on the equivalence with for-transducers, and which will also explain the relevance of the stack discipline. We begin by showing that without stack discipline, the model would not be continuous.

Example 38. [Importance of stack discipline] Consider the non-regular language $\{a^n b^n \mid n \in \mathbb{N}\}$. We will show that this language can be computed by an extension of pebble transducers that does not have stack discipline. (The transducer will output “yes” or “no” depending on whether the input string is in the language or not.) The transducer begins by placing the pebble one at the beginning of the input string, and pebble two at the end. Then, the transducer enters a loop, such that in each iteration pebble one is moved one step to the right, and pebble two is moved one step to the left, until both pebbles meet. We require that, before meeting, the labels traversed by pebble one have label a and the labels traversed by pebble two have label b . Formally speaking, this transducer needs a third pebble, which shuttles between the first two pebbles to implement the loop. Stack discipline is violated by this transducer, since pebble one is moved without removing pebble two. As we have remarked in the footnote on stack discipline, without stack discipline the model can compute the same functions as deterministic Turing machines with logarithmic space. $\square$

We now show that with stack discipline, the model becomes continuous.

Theorem D.2.1. *Pebble transducers compute continuous functions.*

Proof. In the proof, it will be simpler to deal with *pebble automata*, which are the variant of pebble transducers that compute languages, i.e. string-to-Boolean instead of string-to-string. A pebble automaton is defined like a pebble transducer, except that instead of outputting letters, at some point during its run it makes a decision which is either “accept” or “reject”. We first observe that the composition of a pebble transducer with a regular language is a pebble automaton, as explained in the following diagram:



This is easy to show, by using a product of the pebble transducer with a DFA that recognises the regular language. Therefore, in order to prove the theorem, it is enough to show that pebble automata recognise only regular languages. (They also recognise all regular languages, since they generalise DFA.)

Consider a pebble automaton that has k pebbles and states Q . If the input string has length n , then the set of configurations can be described by the polynomial

$$\tau(n) = \sum_{\ell \in \{0, \dots, k\}} Q \times (1 + n)^\ell. \quad (14)$$

The main observation in the proof is that reachability of configurations can be defined in MSO, as stated in the following lemma.

Lemma D.2.2. *Let τ be as in (14). There is an MSO formula*

$$\varphi(s : \tau, t : \tau)$$

which holds if there is a run that begins in configuration s and ends in configuration t .

Before proving the lemma, we use it to conclude the proof of the theorem. By the lemma, the language recognised by a pebble automaton is defined by an MSO formula which says that there is a run from the initial configuration to an accepting configuration. Since MSO can only define regular languages thanks to Theorem C.4.1, it follows that the language recognised by the automaton is regular, as required in the theorem. It remains to prove the lemma.

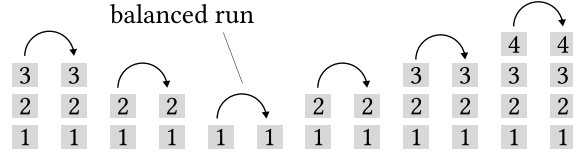
Proof. The most natural idea would be to define one-step reachability using a formula ψ , and to then express reachability as a fixpoint, as follows

$$\underbrace{\forall X : 2^\tau}_{\substack{\text{for every} \\ \text{set } X \text{ of} \\ \text{configurations}}} \underbrace{(s \in X \wedge \forall c : \tau \forall d : \tau \psi(c, d) \wedge c \in X \Rightarrow d \in X)}_{\substack{\text{if } X \\ \text{contains} \\ \text{the source} \\ \text{and is closed under taking transitions}}} \Rightarrow \underbrace{t \in X}_{\substack{\text{then } X \\ \text{contains} \\ \text{the target}}}$$

Unfortunately, this approach does not work directly, since we cannot quantify over sets of type 2^τ . This is because τ contains tuples of positions (if the automaton has $k >$

1 pebbles), and monadic second-order logic can only quantify over sets of individual positions. To work around this issue, we will have to appeal to the stack discipline.

The proof is based on the analysis of special runs, which we call *balanced runs*: this is a run in which the source and target have the same height, and the top pebble is never popped (but it can be moved). Every run of the pebble automaton can be decomposed into a sequence of at most $4k$ runs, each of which is either a single transition, or a balanced run, as explained in the following picture, which shows the evolution of the stack:



Since the decomposition has constant length, it remains to show that we can define in MSO runs which are either a single transition, or a balanced run. For single transitions, this is achieved by simply formalising the semantics of a pebble automaton in the logic. The interesting case is that of balanced runs.

When formalising balanced runs, we will think of a configuration with stack of height $\ell > 0$ as being a pair of the form (x, y) where

$$\underbrace{x : (1 + n)^{\ell-1}}_{\text{prefix of the stack that has height } \ell - 1} \quad \underbrace{y : Q \times (1 + n)}_{\text{the head}} \quad \text{we write } \sigma \text{ for this type}$$

The point of this decomposition is that a balanced run modifies y but not x . The following claim shows that balanced runs can be defined in MSO, and thus concludes the proof of the lemma.

Claim D.2.3. *For every $\ell \in \{1, \dots, k\}$, there is an MSO formula*

$$\varphi_\ell(x : (1 + n)^{\ell-1}, y_1 : \sigma, y_2 : \sigma)$$

which holds if and only if there is a balanced run from (x, y_1) to (x, y_2) .

Proof. The proof is by induction on ℓ , but in the opposite order: the induction starts with $\ell = k$ and proceeds down until $\ell = 1$. The base case is proved in the same way as the induction step, so we will only describe the induction step. Define a *basic balanced run* to be a balanced run which is either: (a) a single transition that moves pebble ℓ ; or (b) a run that starts by pushing pebble $\ell + 1$, then does a balanced run, and finishes with popping pebble $\ell + 1$. A balanced run is a composition of basic balanced runs. Using the induction assumption (if $\ell < k$), we can write an MSO formula

$$\psi(x : (1 + n)^{\ell-1}, y_1 : \sigma, y_2 : \sigma)$$

which says that there is a basic balanced run from (x, y_1) to (x, y_2) . To finish the proof of the claim, we will formalise in MSO the composition of basic balanced runs, using a fixpoint similar to the one at the beginning of the proof of the lemma.

$$\underbrace{\forall Y : 2^\sigma}_{\substack{\text{for every} \\ \text{set } Y \text{ of} \\ \text{head} \\ \text{positions}}} \underbrace{(y_1 \in Y \wedge \forall c : \sigma \forall d : \sigma \psi(x, c, d) \wedge c \in Y \Rightarrow d \in Y)}_{\substack{\text{if } Y \\ \text{contains} \\ \text{the source} \quad \text{and is closed under taking basic balanced runs}}} \Rightarrow \underbrace{y_2 \in Y}_{\substack{\text{then } Y \\ \text{contains} \\ \text{the target}}}.$$

The difference with respect to the erroneous fixpoint construction at the beginning of the lemma is that the type σ is linear, and therefore quantification over sets of type 2^σ is allowed in MSO, as explained below:

$$2^\sigma = 2^{Q \times (1+n)} = \overbrace{(2^1 \times 2^n) \times \cdots \times (2^1 \times 2^n)}^{\substack{\text{quantifying over a subset of } Q \times (1+n) \text{ is like} \\ \text{quantifying over } |Q| \text{ bits and } |Q| \text{ subsets of } n}}.$$

This completes the proof that balanced runs can be defined in MSO. $\square$

As we have remarked at the beginning of the proof of the lemma, every run can be decomposed into a constant number of runs, each of which is either a single transition or a balanced run. Since both of these can be defined in MSO, it follows that reachability can be defined in MSO, as required in the lemma. $\square$

$\square$

D.2.2 Equivalence with for-transducers

We now prove that pebble transducers compute the same string-to-string functions as for-transducers, and therefore they compute exactly the polyregular functions.

Theorem D.2.4. *Pebble transducers and for-transducers compute the same string-to-string functions.*

One direction is clear: a for-transducer can easily be simulated by a pebble transducer, with one pebble for each nested for loop, and with the state used to store the values of the Boolean variables and the currently executed instruction.

The interesting direction is simulating a pebble transducer with a for-transducer. The difficulty is that a pebble transducer can move its head both ways, while a loop in a for-transducer has a fixed direction, which is either first-to-last or last-to-first. Therefore, converting a pebble transducer into a for-transducer can be seen as a sort of “untwisting”, to use the terminology of [BGMP17]. A variant of untwisting was already presented when decomposing two-way transducers into primes in Theorem C.2.9, and our proof will draw heavily on the ideas behind that decomposition. It is worth pointing out that this untwisting will increase the depth of recursion. For example, map reverse is a function with linear output, which can be implemented by a pebble transducer with one pebble, but the corresponding for-transducer in Figure 2 has three nested loops.

To transform a pebble transducer into a for-transducer, we will show that a for-transducer can be used to produce the entire computation of a pebble transducer, represented as a list of configurations. A configuration of a pebble transducer can be represented as a string over an extended alphabet, as explained in the following example:

$$\underbrace{q}_{\text{state}} \text{ } abaa \textcolor{red}{x_1} x_3 ababa \textcolor{red}{x_2} \text{ } ababaa \textcolor{red}{x_4} abababa.$$

pebble i
is letter x_i

More formally, the *string representation* of a configuration with ℓ pebbles is a string in

$$Q \cdot (A \cup \{x_1, \dots, x_\ell\})^*$$

where each pebble name from $\{x_1, \dots, x_\ell\}$ appears exactly once, and if there are consecutive letters that represent pebbles, then they are listed in increasing order. Define the *string representation* of a run to be the concatenation of the string representations of the configurations used in the run.

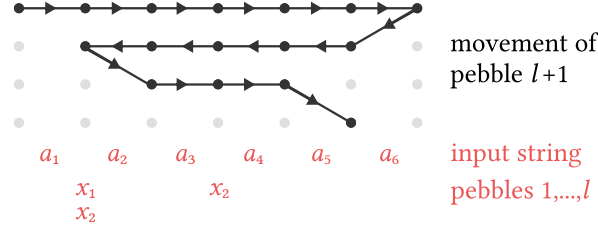
To prove that the computation of a pebble transducer can be produced by a for-transducer, we will appeal to the tree structure in a run of a pebble transducer, which we now explain. For a configuration with stack of height ℓ that appears in this run, its *parent* is defined to be the most recent earlier configuration with stack of height $\ell - 1$. If we assume that the initial configuration is the unique configuration of height 0 in every run, then this parent relation defines a tree structure on the configurations, with the initial configuration being the root. The assumption that the initial configuration is the unique configuration of height 0 in every run can be made without loss of generality, since any pebble transducer can be easily transformed into one that satisfies this assumption. The main technical ingredient in the proof of Theorem D.2.4 is the following lemma, which says that the children of a configuration in this tree structure can be computed by a for-transducer.

Lemma D.2.5. *For every k -pebble transducer there is a for-transducer which inputs a string representation of a configuration, and outputs all children of this configuration, represented as the concatenation of their string representations in order of execution.*

Before proving the lemma, we show how to use it to conclude the proof of the theorem. Functions computed by for-transducers are closed under composition, which was shown in Lemma D.1.4. They are also closed under the map combinator, which is left as an exercise for the reader. Thanks to these closure properties and the lemma, we can use a for-transducer to output all configurations used in a run: first we output the initial configuration, then we append to it the list of its children, then we append (using map) to each child a list of all of its children, and so on k times, until we have output all configurations in the run. Once we have a list of all configurations, the output string can be produced using a rational function which scans each configuration and outputs the letter (if any) that is produced when its transition is executed. It remains to prove the lemma.

Proof of Lemma D.2.5. Fix some k -pebble transducer for the rest of the proof. Suppose that the input is a configuration with stack of height ℓ . If $\ell = k$, then there is nothing

to do, since the stack is already at full height and there can be no children. Suppose now that $\ell < k$. The children of the input configuration can be described as a direct graph, in which an edge represents a run between two configurations of height $\ell + 1$ that are not separated by any configuration of height $\ell + 1$. Here is a picture of this graph, in which $\ell = 3$.



The picture also contains additional annotation (in red), which gives us the input string and the positions of pebbles $\{1, \dots, \ell\}$ that stay fixed throughout the run. We use the name *child configuration graph* for this graph. The child configuration graph can be represented as a string over a fixed finite alphabet, whose length is the same as the input string. This string contains the red annotation of the input string together with the positions of pebble $\ell + 1$ in each child. Using this string representation, it is meaningful to talk about a for-transducer which outputs the child configuration graph.

Claim D.2.6. *There is a for-transducer which inputs the string representation of a configuration, and outputs the string representation of its child configuration graph.*

Proof. In fact, we will show that the function in the claim is already a rational function, and therefore a special case of a for-transducer. To prove this, as our model for rational functions we use unambiguous one-way transducers with output. The transducer will simply guess the output string, and then check if it is correct. To complete the proof, it remains to show that the set of string representations of child configuration graphs is a regular language. Since regular languages are exactly those that can be defined in mso by Theorem C.4.1, it is enough to show that set of string representations of child configuration graphs is definable in mso. This follows from Lemma D.2.2, in which we showed that reachability of configurations can be defined in mso. $\square$

Thanks to the above claim, in order to complete the proof of the lemma, it is enough to show that there is a for-transducer which inputs the string representation of the child configuration graph, and outputs the concatenation of the string representations of the configurations that are stored in this graph, as stated in the following claim.

Claim D.2.7. *There is a for-transducer which inputs the string representation of a child configuration graph, and outputs the concatenation of the string representations of the corresponding child configurations.*

Proof. This is essentially the same proof as in Lemma C.2.12, in which we transformed a configuration graph into its output string. The difference is that in this claim, for each vertex in the child configuration graph we need to output the entire child configuration, which requires producing a copy of the input string. In particular, the transformation has quadratic output size.

As in Lemma C.2.12, the lemma is proved by induction on the width of the input graph, i.e. the maximal number of times that a given input gap is visited. In particular, in the inductive argument we will use subgraphs of the child configuration graph, in which the input is restricted by removing edges. In the induction basis, when the width is 1, then the path in the child configuration graph goes in one direction only, and the output can be produced by a for-transducer that has an outer loop which traverses the snake, and an inner loop which is used to copy the input string. (Depending on the direction of the path, the outer loop is either first-to-last or last-to-first.) In the induction step, we first deal with the case of looping snakes, and then we decompose a general snake into a sequence of loops, exactly as we did in the proof of Lemma C.2.12. $\square$

By composing the for-transducers in Claims D.2.6 and D.2.7, we get a for-transducer which inputs the string representation of a configuration, and outputs the concatenation of the string representations of its children, as required in the lemma. $\square$

References

- [BGMP17] Félix Baschenis, Olivier Gauwin, A. Muscholl, and G. Puppis. Untwisting two-way transducers in elementary time. *Logic in Computer Science*, 2017.
- [Boj18] Mikołaj Bojańczyk. Polyregular Functions. *CoRR*, abs/1810.08760, 2018.
- [Boj25] Mikołaj Bojańczyk. An automata toolbox. Lecture notes available at mimuw.edu.pl/~bojan/paper/automata-toolbox-book, 2025.
- [EHS07] J. Engelfriet, H. Hoogeboom, and Bart Samwel. Xml transformation by tree-walking transducers with invisible pebbles. *ACM SIGACT-SIGMOD-SIGART Symposium on Principles of Database Systems*, 2007.
- [EM02] J. Engelfriet and S. Maneth. Two-way finite state transducers with nested pebbles. *International Symposium on Mathematical Foundations of Computer Science*, 2002.
- [HU67] J. E. Hopcroft and J. D. Ullman. An approach to a unified theory of automata. In *8th Annual Symposium on Switching and Automata Theory (SWAT 1967)*, pages 140–147, Oct 1967.
- [Iba71] O. Ibarra. Characterizations of some tape and time complexity classes of turing machines in terms of multihead and auxiliary stack automata. *Journal of computer and system sciences (Print)*, 1971.
- [LH96] Noa Lewenstein and D. Harel. Complexity results for two-way and multi-pebble automata and their logics. *Theoretical Computer Science*, 1996.
- [MNR23] Filip Murlak, Damian Niwiński, and Wojciech Rytter. *200 problems on languages, automata, and computation*. Cambridge University Press, 2023.
- [MSV03] Tova Milo, Dan Suciu, and V. Vianu. Typechecking for xml transformers. *Journal of computer and system sciences (Print)*, 2003.
- [RS59] Michael O. Rabin and Dana Scott. Finite automata and their decision problems. *IBM J. Res. Develop.*, 3:114–125, 1959.
- [She59] J. C. Shepherdson. The reduction of two-way automata to one-way automata. *IBM Journal of Research and Development*, 3(2):198–200, April 1959.